

University of Rajshahi · Dept. of Computer Science & Engineering

# CSE4121

## Object Oriented Design and Design Patterns

Probable Exam Questions with Complete Answers

English Q&A · 58 questions · UML diagrams · compiled Java code

### What is inside

**Every** question from the 2021, 2023 and 2024 finals, CT-1 and CT-2, **every** question of the two model papers (PDF 1), and extra questions on topics taught but never asked. The “Asked in” line of each question tells you where it came from:

**2024 Q2(a)** = final exam question    **CT-2 Q1** = class test    **Set-1 Q5(a)** = model paper    **New** = predicted.

All 36 Java programs were compiled and executed; the printed output under each program is the real output.

### Scenario names used

**DUMS** = Dhaka Urban Management System (garbage collection, complaints, housing, billing, traffic).

**AAS** = Agricultural Automation System (irrigation, harvesters, sprayers, soil sensors, weather).

# Contents

---

|          |                                                                           |           |
|----------|---------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Section A — OO Analysis, Modelling and CRC Cards</b>                   | <b>1</b>  |
| Q1       | Design before coding; conceptual vs technical design . . . . .            | 1         |
| Q2       | Is UML unnecessary when IDEs can generate code? . . . . .                 | 1         |
| Q3       | Benefits of representing real-world entities as objects . . . . .         | 2         |
| Q4       | CRC cards for the DUMS smart-city portal . . . . .                        | 3         |
| Q5       | CRC cards for AAS smart irrigation . . . . .                              | 5         |
| Q6       | Software design vs software architecture . . . . .                        | 6         |
| Q7       | Object-oriented modelling; entity, control and boundary objects . . . . . | 6         |
| Q8       | Trade-offs between client needs and developer scope . . . . .             | 7         |
| Q9       | Evolution of programming languages towards OOP . . . . .                  | 8         |
| Q10      | Abstraction with DUMS and AAS examples . . . . .                          | 8         |
| <b>2</b> | <b>Section A — The Four Design Principles</b>                             | <b>10</b> |
| Q11      | TrafficLight: abstraction and encapsulation . . . . .                     | 10        |
| Q12      | Black-box encapsulation, visibility symbols, information hiding . . . . . | 12        |
| Q13      | Decomposition: association, aggregation, composition (DUMS) . . . . .     | 12        |
| Q14      | DUMS waste component: full class diagram and Java skeleton . . . . .      | 15        |
| Q15      | Aggregation vs composition . . . . .                                      | 18        |
| Q16      | FarmingMachine hierarchy: generalization for reuse . . . . .              | 18        |
| Q17      | The Whale extends Animal problem . . . . .                                | 21        |
| Q18      | SolarIrrigationPump cannot refuel . . . . .                               | 22        |
| Q19      | SmartDroneSprayer: interface inheritance . . . . .                        | 23        |
| <b>3</b> | <b>Section A — Design Complexity and Code Quality</b>                     | <b>26</b> |
| Q20      | Coupling and cohesion . . . . .                                           | 26        |
| Q21      | ServiceManager: evaluate and redesign (DUMS) . . . . .                    | 26        |
| Q22      | Sensor class with a control flag . . . . .                                | 29        |
| Q23      | SmartPhone: coupling, cohesion and separation of concerns . . . . .       | 31        |
| Q24      | Conceptual integrity . . . . .                                            | 34        |
| Q25      | OrderProcessor code smell; comments as a code smell . . . . .             | 35        |
| Q26      | Single Responsibility Principle . . . . .                                 | 36        |
| Q27      | Switch-statement smell and the Open/Closed Principle . . . . .            | 38        |
| Q28      | Message chains and the Law of Demeter . . . . .                           | 40        |
| <b>4</b> | <b>Section A — Behavioural Modelling with UML</b>                         | <b>42</b> |
| Q29      | Sequence diagram: changing a TV channel with a remote . . . . .           | 42        |
| Q30      | Sequence diagram: citizen reports uncollected garbage . . . . .           | 43        |
| Q31      | State diagram for a DUMS Complaint . . . . .                              | 44        |
| Q32      | UML state diagrams and model checking . . . . .                           | 45        |
| <b>5</b> | <b>Section B — Design Pattern Foundations</b>                             | <b>46</b> |
| Q33      | What are design patterns and why learn them? . . . . .                    | 46        |

|          |                                                                           |           |
|----------|---------------------------------------------------------------------------|-----------|
| Q34      | Categories: creational, structural and behavioural patterns . . . . .     | 47        |
| <b>6</b> | <b>Section B — Creational Patterns</b>                                    | <b>49</b> |
| Q35      | Convert ConfigurationManager into a Singleton . . . . .                   | 49        |
| Q36      | Database Singleton; shared logging service . . . . .                      | 51        |
| Q37      | Thread-safe SmsGateway Singleton . . . . .                                | 53        |
| Q38      | Factory pattern for DUMS notifications . . . . .                          | 55        |
| Q39      | Abstract Factory for AAS device families . . . . .                        | 58        |
| Q40      | Builder for a DUMS HousingApplication . . . . .                           | 61        |
| <b>7</b> | <b>Section B — Structural Patterns</b>                                    | <b>65</b> |
| Q41      | Adapter pattern: problem, real-world example, UML, media player . . . . . | 65        |
| Q42      | Adapter for an IoT soil-moisture sensor (AAS) . . . . .                   | 67        |
| Q43      | Mobile-wallet payment adapter; object vs class adapter . . . . .          | 69        |
| Q44      | Facade pattern: scenario, steps and facilities . . . . .                  | 72        |
| Q45      | Composite pattern for City, Zone, Ward and Bin . . . . .                  | 74        |
| Q46      | Bridge pattern for DUMS alerts . . . . .                                  | 77        |
| Q47      | Filter (Criteria) pattern for DUMS complaints . . . . .                   | 78        |
| Q48      | Decorator pattern for layered notifications . . . . .                     | 80        |
| <b>8</b> | <b>Section B — Behavioural Patterns and MVC</b>                           | <b>83</b> |
| Q49      | Observer: underlying principle; Facebook followers . . . . .              | 83        |
| Q50      | Observer for DUMS complaint status notifications . . . . .                | 86        |
| Q51      | Observer for an AAS weather station . . . . .                             | 88        |
| Q52      | Chain of Responsibility: bank cheque authorisation . . . . .              | 91        |
| Q53      | Chain of Responsibility: university approval hierarchy . . . . .          | 93        |
| Q54      | Chain of Responsibility: DUMS complaint escalation . . . . .              | 95        |
| Q55      | Command pattern for the AAS farmer remote app . . . . .                   | 97        |
| Q56      | Mediator pattern for a smart traffic intersection . . . . .               | 101       |
| Q57      | MVC for a maintainable web application . . . . .                          | 104       |
| Q58      | Pattern identification drill . . . . .                                    | 107       |

## CHAPTER 1

# Section A — OO Analysis, Modelling and CRC Cards

### Q1. Design before coding; conceptual vs technical design

Asked in: Set-1 Q1(a) · New (Slides M1)

Marks: 3

“Good design begins before a single line of code is written.” Explain this statement. Differentiate between *conceptual design* and *technical design* with reference to the waste-collection module of the Dhaka Urban Management System (DUMS).

### Why design comes before coding

Requirements say *what* the system must do; code says *how* the computer does it. Design is the bridge between the two. Coding without design is like building a house without a plan: work starts fast, but walls must be broken later.

- **Mistakes are cheap in design.** Moving a box in a diagram costs minutes; changing a class that twenty other classes already call costs days of rewriting and retesting.
- **The right structure is chosen first.** Design fixes the objects, their responsibilities and their relationships, so coding becomes translation instead of invention.
- **Teams can work in parallel.** Agreed interfaces let different developers build different modules that still fit together.
- **Quality is planned.** Performance, security, maintainability and extensibility are considered while alternatives are still open.
- **DUMS example.** If the truck-routing code is written first and reads the bin-sensor database table directly, adding a second sensor vendor later breaks routing. A design with a `FillLevelSensor` abstraction between them prevents this.

### Conceptual design vs technical design

|                          | Conceptual design                                                                                                                                                                                                           | Technical design                                                                                                                                                                                                                                                                                       |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Question</b>          | Which major components are needed and what does each do?                                                                                                                                                                    | How is each component built and how do components communicate?                                                                                                                                                                                                                                         |
| <b>Level</b>             | High level, independent of technology                                                                                                                                                                                       | Low level, technology-specific                                                                                                                                                                                                                                                                         |
| <b>Artefacts</b>         | Conceptual mock-ups, component lists, CRC cards                                                                                                                                                                             | UML class, sequence and state diagrams, APIs, data structures, algorithms, database schema                                                                                                                                                                                                             |
| <b>Audience</b>          | Client, analysts, developers                                                                                                                                                                                                | Developers, testers                                                                                                                                                                                                                                                                                    |
| <b>DUMS waste module</b> | Components: smart bin, collection zone, garbage truck, collection scheduler, citizen app. Responsibilities: a bin reports its fill level; the scheduler sends the nearest free truck; a citizen reports an overflowing bin. | Classes <code>GarbageBin</code> , <code>FillLevelSensor</code> , <code>CollectionScheduler</code> ; a bin sends a reading every 10 minutes; the scheduler uses a shortest-path algorithm on the road graph; table <code>bins(bin_id, zone_id, level)</code> ; endpoint <code>POST /complaints</code> . |

Conceptual design is done first. Technical design then keeps splitting each conceptual component into smaller components until each one is small enough to be coded.

### Q2. Is UML unnecessary when IDEs can generate code?

Asked in: CT-1 Q1

Marks: CT

Some developers argue that UML diagrams are unnecessary because modern IDEs can gener-

ate code and even draw class diagrams automatically. Critically evaluate this argument. What problems may arise if a large system such as DUMS is developed without any design model? (*paraphrased from CT-1*)

### What the argument gets right

IDEs really can generate getters/setters, constructors and class skeletons, and can reverse-engineer a class diagram from existing code. For a tiny program, drawing diagrams first may be unnecessary.

### Why the argument is wrong for real systems

1. **An IDE generates code only from decisions already made.** It cannot decide which classes should exist, which class owns which responsibility, or whether a relationship is aggregation or composition. That thinking *is* design, and UML is the tool for it.
2. **Reverse-engineered diagrams show what the code *is*, not what it *should be*.** A generated diagram of a badly designed system faithfully shows a badly designed system.
3. **Communication.** UML is a common visual language for clients, analysts, developers and testers. A class diagram of ten boxes explains what three thousand lines of code hide.
4. **Big picture and dynamics.** Multiplicities, whole–part lifetimes, message order (sequence diagrams) and object life cycles (state diagrams) are invisible when you read one source file at a time.
5. **Early validation.** Design reviews and model checking find missing states, deadlocks and wrong interactions before any code exists.
6. **Maintenance and onboarding.** New team members understand the architecture from diagrams in hours instead of weeks.

### Problems of building DUMS without a design model

- **Duplicated and inconsistent classes:** the housing team writes Resident, the complaint team writes Citizen for the same person.
- **God classes:** a CityManager that handles housing, garbage and traffic — low cohesion and tight coupling.
- **Integration failure:** billing identifies a person by an integer ID while complaints use the NID string.
- **Poor extensibility:** adding a new service (e.g. water-logging relief) requires edits in many places.
- **High cost of change:** missing requirements are discovered after coding; testing is difficult; no documentation exists.

**Conclusion:** UML and IDEs are complementary — design with UML (kept lightweight), implement and generate with the IDE, and keep the key diagrams updated.

#### Q3. Benefits of representing real-world entities as objects

**Asked in:** 2024 Q1(a) · 2023 Q1(a)

**Marks:** 3

Explain the benefit of representing real-world entities — such as Citizen, MunicipalService and Complaint in the Dhaka Urban Management System — as objects in your code.

### Idea

An object bundles **identity**, **state** (attributes) and **behaviour** (methods), exactly as a real-world entity has an identity, properties and things it can do.

| Object           | State                             | Behaviour                                   |
|------------------|-----------------------------------|---------------------------------------------|
| Citizen          | nid, name, phone, address         | login(), reportComplaint(), payBill()       |
| MunicipalService | serviceType, department, crewList | acceptRequest(), schedule(), updateStatus() |
| Complaint        | id, category, location, status    | assign(), resolve(), escalate()             |

## Benefits

1. **Natural mapping (small representational gap).** City officers talk about citizens and complaints; the code has Citizen and Complaint. Requirements are easy to trace to code and domain experts can review the design.
2. **Encapsulation and data integrity.** A complaint's status can change only through assign() and resolve(), so an invalid status such as "RESOLVED before ASSIGNED" can be blocked in one place.
3. **Abstraction.** Only attributes relevant to DUMS are modelled (a citizen's NID, not their favourite colour), keeping the model simple.
4. **Modularity and reuse.** The same Citizen class is reused by the complaint, billing and housing modules.
5. **Maintainability.** A change in the escalation rule is made inside Complaint (or its handler) without touching billing code.
6. **Extensibility through inheritance and polymorphism.** WasteCollectionService and RoadMaintenanceService can extend MunicipalService, and new services are added without rewriting old ones.
7. **Testability and team work.** Each object can be unit-tested and developed by different people.

```

1 public class Complaint {
2     private final String id;
3     private String status = "SUBMITTED";           // state is protected
4     public Complaint(String id) { this.id = id; }
5     public void assign() { status = "ASSIGNED"; } // behaviour guards the state
6     public void resolve() {
7         if (!status.equals("ASSIGNED"))
8             throw new IllegalStateException("must be assigned first");
9         status = "RESOLVED";
10    }
11 }

```

### Q4. CRC cards for the DUMS smart-city portal

Asked in: 2024 Q1(b) · 2023 Q1(b) ATM · 2021 Q1(c)

Marks: 4.75 (2021: 3+2)

A city resident logs into the smart city portal using their ID and password, which are verified against the municipal database. The resident can then report waste-collection issues, pay utility bills, request road maintenance or track service status. Explain how CRC cards can be used to isolate and define the system components — attributes, functionalities and interactions — for classes such as Citizen, MunicipalService, Complaint and Payment. What are the advantages of CRC cards?

## What a CRC card is

A **Class–Responsibility–Collaborator** card is a small index card used in *conceptual* design. The **class name** is written on top, the **responsibilities** (what the class knows and does) on the left, and the **collaborators** (classes it needs to fulfil those responsibilities) on the right.

## How CRC cards isolate the components

1. **Find candidate classes from nouns:** resident, ID, password, portal, municipal database, waste issue, bill, road maintenance, status.
2. **Find responsibilities from verbs:** log in, verify, report, pay, request, track.
3. **Write one card per class.** “Knows ...” responsibilities become *attributes*; “does ...” responsibilities become *functionalities*; collaborators become *interactions*.
4. **Role-play the scenario:** move the cards on a table step by step (resident logs in → portal asks authentication service → database checks credentials ...). Any step that no card can perform reveals a missing class or responsibility.
5. **Refine:** split a card that is overloaded (low cohesion), merge cards that do almost nothing, then convert the cards into a UML class diagram (collaborators become associations).

## CRC cards for the scenario

| Citizen                        |                 |
|--------------------------------|-----------------|
| Responsibilities               | Collaborators   |
| Know NID, name, phone, address | SmartCityPortal |
| Log in with ID and password    | Complaint       |
| Choose a service               | Payment         |
| Report a complaint             |                 |
| Pay a utility bill             |                 |
| Track own requests             |                 |

| AuthenticationService (control)      |                   |
|--------------------------------------|-------------------|
| Responsibilities                     | Collaborators     |
| Verify ID and password               | MunicipalDatabase |
| Create a login session               | SmartCityPortal   |
| Lock account after repeated failures |                   |

| Complaint                                                 |                   |
|-----------------------------------------------------------|-------------------|
| Responsibilities                                          | Collaborators     |
| Know id, type (waste/road), location, description, status | Citizen           |
| Get assigned to a service                                 | MunicipalService  |
| Update and report its status                              | MunicipalDatabase |

| Payment                             |                   |
|-------------------------------------|-------------------|
| Responsibilities                    | Collaborators     |
| Know bill no., amount, method, date | Citizen           |
| Validate the amount                 | MunicipalDatabase |
| Process payment through gateway     | PaymentGateway    |
| Generate receipt                    |                   |

| SmartCityPortal (boundary)  |                       |
|-----------------------------|-----------------------|
| Responsibilities            | Collaborators         |
| Accept ID and password      | Citizen               |
| Show the service menu       | AuthenticationService |
| Forward the chosen option   | Complaint, Payment    |
| Display receipts and status | MunicipalService      |

| MunicipalDatabase                       |                       |
|-----------------------------------------|-----------------------|
| Responsibilities                        | Collaborators         |
| Store citizen credentials and profiles  | AuthenticationService |
| Store complaints, payments and requests | Complaint, Payment    |
| Return stored records                   |                       |

| MunicipalService                          |                   |
|-------------------------------------------|-------------------|
| Responsibilities                          | Collaborators     |
| Know department (waste, road, utility)    | Complaint         |
| Accept a complaint or maintenance request | MunicipalDatabase |
| Schedule truck or repair crew             |                   |
| Update request status                     |                   |

| StatusTracker (control)           |                  |
|-----------------------------------|------------------|
| Responsibilities                  | Collaborators    |
| Fetch current status of a request | Complaint        |
| Show status history               | MunicipalService |
|                                   | SmartCityPortal  |

## Advantages of CRC cards

- **Cheap and low-tech** — paper cards, no tool, easy to throw away and rewrite.
- **Responsibility-driven thinking** — designers think about behaviour, not only data.

- **Keeps classes cohesive** — a small card cannot hold too many responsibilities; an overflowing card must be split.
- **Finds missing classes early** through role-play of scenarios.
- **Encourages team discussion** and involves non-programmers (clients, analysts).
- **Direct path to UML** — classes become boxes, collaborators become associations.

**2023 variant: ATM scenario (insert card, enter PIN, deposit / withdraw / balance)**

| Class                  | Responsibilities                                                                       | Collaborators                   |
|------------------------|----------------------------------------------------------------------------------------|---------------------------------|
| <b>ATM</b> (bound-ary) | Read card, accept PIN, show menu, dispense cash, print receipt                         | CardReader, Bank, CashDispenser |
| <b>Bank</b>            | Verify card and PIN, authorise transactions                                            | Account, ATM                    |
| <b>Account</b>         | Know number and balance; deposit; withdraw if balance allows                           | Bank, Transaction               |
| <b>Transaction</b>     | Know type, amount, time; execute deposit / withdrawal / balance inquiry; record itself | Account, ATM                    |
| <b>CardReader</b>      | Read card number; eject or retain card                                                 | ATM                             |
| <b>CashDispenser</b>   | Know cash available; dispense notes                                                    | ATM                             |

**Q5. CRC cards for AAS smart irrigation**
**Asked in:** Set-1 Q1(b) · New

**Marks:** 4.75

In an Agricultural Automation System (AAS), soil-moisture sensors in every field send readings every 10 minutes. When the moisture of a field falls below its threshold, the irrigation controller checks the weather forecast; if no rain is expected, it opens the valve of that field and informs the farmer through the mobile app. Using CRC cards, identify the classes with their responsibilities and collaborators.

**Candidate classes**

Nouns: *sensor, field, reading, threshold, irrigation controller, weather forecast, valve, farmer, mobile app*. “Threshold” is an attribute of Field, not a class; “forecast” is obtained through a WeatherService.

**SoilMoistureSensor**

| Responsibilities                                                                         | Collaborators                         |
|------------------------------------------------------------------------------------------|---------------------------------------|
| Know sensor id and its field<br>Measure soil moisture<br>Send a reading every 10 minutes | SensorReading<br>IrrigationController |

**SensorReading**

| Responsibilities                         | Collaborators |
|------------------------------------------|---------------|
| Know field id, moisture value, timestamp | Field         |

**Field**

| Responsibilities                                                                         | Collaborators          |
|------------------------------------------------------------------------------------------|------------------------|
| Know id, crop, moisture threshold, its valve<br>Tell whether moisture is below threshold | SensorReading<br>Valve |

**IrrigationController (control)**

| Responsibilities                                                                                                                                                     | Collaborators                                           |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| Receive readings<br>Compare with the field threshold<br>Ask for the forecast<br>Decide whether to irrigate<br>Open or close the valve<br>Request farmer notification | Field<br>WeatherService<br>Valve<br>NotificationService |

**WeatherService (boundary)**

| Responsibilities                                                                       | Collaborators        |
|----------------------------------------------------------------------------------------|----------------------|
| Fetch forecast for a location from the external API<br>Report whether rain is expected | IrrigationController |

**Valve**

| Responsibilities                          | Collaborators        |
|-------------------------------------------|----------------------|
| Open<br>Close<br>Report open/closed state | IrrigationController |

| NotificationService                                              |                 |
|------------------------------------------------------------------|-----------------|
| Responsibilities                                                 | Collaborators   |
| Compose the irrigation message<br>Deliver it to the farmer's app | FarmerMobileApp |

| FarmerMobileApp (boundary)                                       |                                             |
|------------------------------------------------------------------|---------------------------------------------|
| Responsibilities                                                 | Collaborators                               |
| Show alerts to the farmer<br>Allow manual start/stop<br>override | NotificationService<br>IrrigationController |

## Role-play check

(1) SoilMoistureSensor creates a SensorReading and sends it to IrrigationController. (2) The controller asks Field whether the value is below its threshold. (3) If yes, it asks WeatherService whether rain is expected. (4) If no rain, it tells the field's Valve to open. (5) It asks NotificationService to inform FarmerMobileApp. Every step is performed by some card, so the set of classes is complete.

### Q6. Software design vs software architecture

Asked in: 2021 Q1(b) · 2023 Q1(c) · 2024 Q1(c) · Set-1 Q1(c)

Marks: 1–2

What is the difference between software design and software architecture?

|                       | Software architecture                                                                                   | Software design                                                                             |
|-----------------------|---------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| <b>Scope</b>          | The high-level structure of the whole system: major components/subsystems and how they connect          | The internal structure of each component: classes, methods, relationships, algorithms       |
| <b>Decisions</b>      | Architectural style (layered, client-server, MVC, microservices), technology, deployment, communication | Class responsibilities, design patterns, data structures, interfaces of classes             |
| <b>Driven by</b>      | Quality attributes: scalability, availability, security, performance                                    | Functional correctness, cohesion, coupling, reusability                                     |
| <b>Cost of change</b> | Very expensive; decided early                                                                           | Cheaper; can be refactored                                                                  |
| <b>Made by</b>        | Architects                                                                                              | Developers / designers                                                                      |
| <b>DUMS example</b>   | Complaint, Billing and Housing as separate services behind an API gateway, each with its own database   | Inside the Complaint service: Complaint, ComplaintHandler chain, Observer for notifications |

**One-line answer:** architecture is the *city plan* (which buildings and roads exist and how they connect), while design is the *floor plan* of each building.

### Q7. Object-oriented modelling; entity, control and boundary objects

Asked in: 2021 Q1(a) · Set-2 Q1(a)

Marks: 1.75 / 3

What is object-oriented modelling? Classify the following AAS classes as entity, control or boundary objects, with a reason for each: Field, Crop, SoilSensorReading, IrrigationScheduler, HarvestPlanner, FarmerMobileScreen, WeatherAPIGateway.

## Object-oriented modelling

Object-oriented modelling is the process of representing a problem and its solution as a set of **interacting objects** — each with attributes, behaviour and relationships — using models (CRC cards, UML class, sequence and state diagrams) *before* writing code. It starts with a conceptual model of the real-world domain and refines it into a technical model that maps directly to classes. Benefits: models are close to the real world, they are easy to discuss with clients, and they translate directly to an OO language.

### The three kinds of design objects

**Entity objects** represent domain concepts and data (often persistent). **Boundary objects** sit at the

edge of the system and interact with actors or external systems (screens, APIs). **Control objects** coordinate the work of a use case and hold its business logic.

| Class               | Type            | Reason                                                                         |
|---------------------|-----------------|--------------------------------------------------------------------------------|
| Field               | <b>Entity</b>   | Real farm concept with id, area, crop and moisture threshold                   |
| Crop                | <b>Entity</b>   | Domain data: name, growth stage, water needs                                   |
| SoilSensorReading   | <b>Entity</b>   | Stored record of one measurement (value, time, field)                          |
| IrrigationScheduler | <b>Control</b>  | Decides when and where irrigation runs; coordinates sensors, fields and valves |
| HarvestPlanner      | <b>Control</b>  | Computes harvest timing from crop and weather data                             |
| FarmerMobileScreen  | <b>Boundary</b> | User interface between the farmer (actor) and the system                       |
| WeatherAPIGateway   | <b>Boundary</b> | Interface to an external weather system                                        |

#### Exam tip

Entity–boundary–control map naturally to MVC: entity  $\approx$  Model, boundary  $\approx$  View, control  $\approx$  Controller.

### Q8. Trade-offs between client needs and developer scope

**Asked in:** Set-2 Q1(b),(c) · New (Slides M1)

**Marks:** 4.75 + 1

The client of DUMS wants every citizen to see the real-time location of all garbage trucks, while the development team has limited time, budget and server capacity. (b) Discuss the trade-offs between client needs and developer scope, with reference to software quality attributes. (c) Differentiate between functional and non-functional requirements with a DUMS example of each.

#### (b) The two sides

The **client** wants features, usability, quick delivery and low cost. The **developers** are limited by time, budget, skills, technology and the need to keep the system maintainable. Design is the art of balancing these, because improving one quality attribute usually costs another.

#### Quality attributes affected by “live truck map for everyone”

- **Performance and scalability:** 300 trucks sending GPS every 5 s, shown to hundreds of thousands of citizens, means huge server and network load.
- **Availability:** the live map must not slow down critical functions like complaint submission.
- **Security and privacy:** exact live location of vehicles and drivers can be misused.
- **Usability:** a crowded city-wide map may confuse users more than a simple arrival time.
- **Maintainability and cost:** real-time infrastructure (web sockets, map tiles) is complex and expensive to maintain.

#### Options and their trade-offs

| Option                                                | Gains                                      | Costs                                                  |
|-------------------------------------------------------|--------------------------------------------|--------------------------------------------------------|
| Full real-time map of all trucks                      | Best experience, exactly what client asked | Highest cost and risk; late delivery; privacy concerns |
| Updates every 60 s, only trucks of the citizen's zone | Most of the value; load cut drastically    | Slightly stale positions                               |
| Only an arrival-time window ("7–8 AM")                | Cheap, private, simple                     | Less engaging                                          |
| Phased release (ETA → zone map → live map)            | On-time first release; learn real usage    | Client waits for the full feature                      |

**Recommended approach:** prioritise requirements with the client (must/should/could), build a quick prototype, agree on *measurable* quality targets (e.g. "position at most 60 s old, page loads within 2 s for 50,000 concurrent users"), and record the decision.

### (c) Functional vs non-functional requirements

A **functional requirement** states *what* the system must do — "A citizen can submit a garbage complaint with a photo and location." A **non-functional requirement** states *how well* it must do it or a constraint on it — "The complaint page must load within 2 seconds for 10,000 concurrent users and all personal data must be encrypted."

#### Q9. Evolution of programming languages towards OOP

Asked in: New (Slides M2)

Marks: 3

Briefly describe how programming languages evolved from early languages to object-oriented languages. What problems of earlier languages did object orientation solve?

1. **Early languages (COBOL, Fortran).** Programs were long sequences of statements working on **global data**. Any line could change any variable, so when data was corrupted it was very hard to find which part of the program did it — the *global data problem*.
2. **Block-structured / procedural languages (ALGOL 68, Pascal, C).** Introduced procedures with **local variables** and scope. Data could be kept inside a block, and programs were divided into functions (top-down decomposition). But data and the functions operating on it were still separate, and shared data still had to be passed around or kept global.
3. **Abstract data types.** Programmers began grouping a data structure with the operations allowed on it (e.g. a stack with push/pop), hiding the representation.
4. **Object-oriented languages (C++, Java).** Made the abstract data type a language feature: the **class**. Data and behaviour are bundled (encapsulation), access is controlled with `private/public`, classes can extend others (inheritance) and objects can be used through a common type (polymorphism).

**Problems solved:** uncontrolled access to shared data, poor mapping of code to real-world concepts, difficulty of reusing code, and difficulty of changing large programs, because a change can now be localised inside one class.

#### Q10. Abstraction with DUMS and AAS examples

Asked in: New (Slides M2)

Marks: 3

What is abstraction? Why does abstraction depend on context? Explain with DUMS and AAS examples and a UML class diagram.

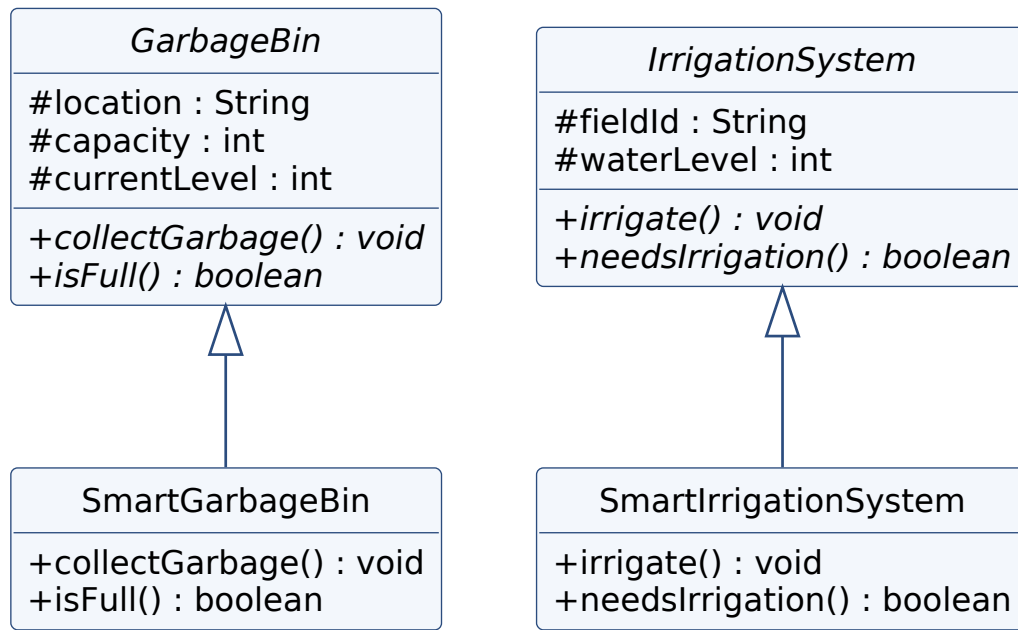
### Definition

Abstraction is the design principle of **keeping only the essential characteristics** of a concept that

matter for the problem and ignoring the rest. In OO design it produces classes whose attributes and methods describe *what* an object is and does, not every detail.

## Context

The “essential” details depend on the purpose. A *garbage bin* in DUMS needs location, capacity and current fill level; a bin manufacturer’s inventory system instead needs material, colour and price. The same real object gives different abstractions in different contexts.



Abstract classes capture what every bin / irrigation system must do; subclasses decide how.

## Explanation

**GarbageBin** exposes `isFull()` and `collectGarbage()`; the city dashboard does not care whether fullness is measured by an ultrasonic or a weight sensor. **IrrigationSystem** exposes `needsIrrigation()` and `irrigate()`; a scheduler can use any irrigation system without knowing its pumps. Abstract methods state *what* must be done; each concrete subclass supplies *how*. Abstraction therefore reduces complexity and lets implementations change without affecting clients.

## Section A — The Four Design Principles

### Q11. TrafficLight: abstraction and encapsulation

Asked in: Set-2 Q2(a) · New

Marks: 3.5

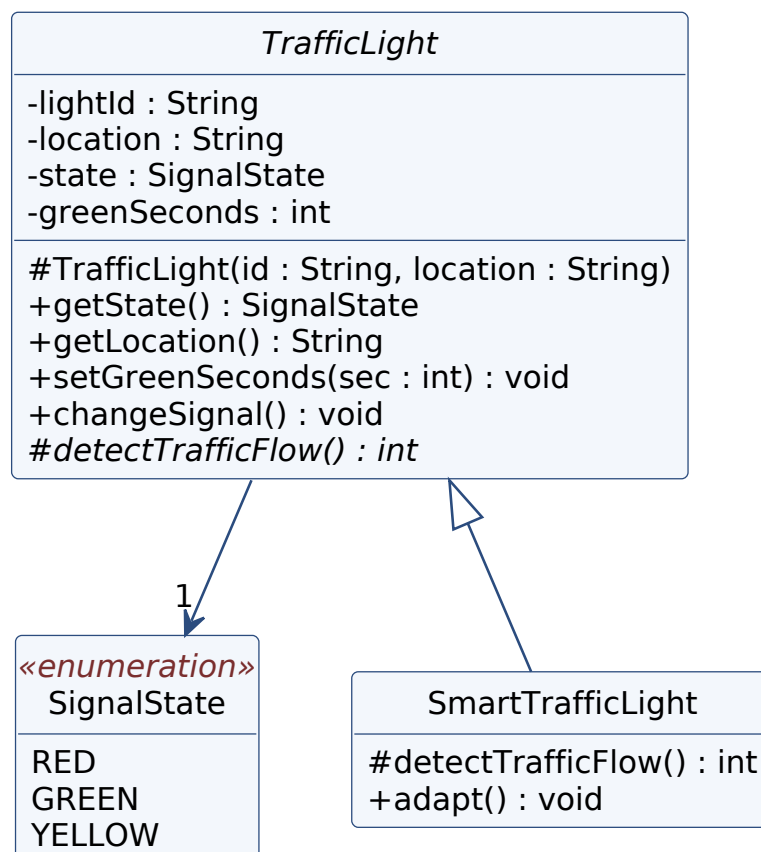
Design a TrafficLight class for DUMS applying abstraction and encapsulation: the signal state must never take an invalid value, the green duration must stay between 10 and 120 seconds, and the way traffic flow is detected must be hidden from other classes. Draw the UML class diagram and write Java code.

#### Applying abstraction

For DUMS traffic control the essential data are the light id, its location, its current signal state and its green duration; the essential behaviour is changing the signal and adapting to traffic flow. Details such as bulb wattage or manufacturer are ignored.

#### Applying encapsulation

- All fields are private. The state is an enum, so no invalid value like “BLUE” can exist.
- There is **no setter for the state**. The only way to change it is `changeSignal()`, which enforces the legal order RED → GREEN → YELLOW → RED.
- `setGreenSeconds()` validates its argument (10–120 s).
- `detectTrafficFlow()` is protected abstract: other classes cannot call it and do not know whether a camera or a loop sensor is used.



UML: private data (–), public interface (+), hidden protected hook (#).

## Java: Q\_TrafficLight.java

```

1  enum SignalState { RED, GREEN, YELLOW }
2
3  abstract class TrafficLight {
4      private final String lightId;
5      private final String location;
6      private SignalState state = SignalState.RED;    // hidden: no direct write access
7      private int greenSeconds = 30;
8
9      protected TrafficLight(String lightId, String location) {
10         this.lightId = lightId;
11         this.location = location;
12     }
13
14     public SignalState getState() { return state; } // getter: read-only view
15     public String getLocation() { return location; }
16
17     public void setGreenSeconds(int sec) {           // setter with validation
18         if (sec < 10 || sec > 120)
19             throw new IllegalArgumentException("green time must be 10-120 s");
20         greenSeconds = sec;
21     }
22
23     public void changeSignal() {                     // only legal order RED -> GREEN ->
24         YELLOW -> RED
25         switch (state) {
26             case RED:    state = SignalState.GREEN; break;
27             case GREEN:  state = SignalState.YELLOW; break;
28             case YELLOW: state = SignalState.RED;   break;
29         }
30         System.out.println(lightId + " @ " + location + " -> " + state
31             + (state == SignalState.GREEN ? " for " + greenSeconds + " s" : ""));
32     }
33
34     protected abstract int detectTrafficFlow();      // HOW flow is measured is hidden
35 }
36
37 class SmartTrafficLight extends TrafficLight {
38     SmartTrafficLight(String id, String location) { super(id, location); }
39     @Override protected int detectTrafficFlow() { return 42; } // vehicles/min from
40     camera
41     public void adapt() { setGreenSeconds(detectTrafficFlow() > 40 ? 60 : 30); }
42 }
43
44 public class Q_TrafficLight {
45     public static void main(String[] args) {
46         SmartTrafficLight light = new SmartTrafficLight("TL-17", "Shahbag");
47         light.adapt();
48         light.changeSignal();
49         light.changeSignal();
50         try {
51             light.setGreenSeconds(500);
52         } catch (IllegalArgumentException e) {
53             System.out.println("Rejected: " + e.getMessage());
54         }
55         // light.state = SignalState.GREEN;  <-- compile-time error: state is private
56     }
57 }

```

**Program output (compiled and run)**

```

1 TL-17 @ Shahbag -> GREEN for 60 s
2 TL-17 @ Shahbag -> YELLOW
3 Rejected: green time must be 10-120 s

```

**Q12. Black-box encapsulation, visibility symbols, information hiding****Asked in:** 2024 Q2(b) · Set-1 Q2(b) · Set-2 Q3(c)**Marks:** 1 / 1 / 1.75

(a) “The computation steps taken within a class never need to be known by any other class, as long as they are able to access the interface.” What type of encapsulation occurs here? (b) List the UML visibility symbols and the equivalent Java access modifiers. (c) Differentiate between information hiding and encapsulation.

**(a) Black-box encapsulation**

This is **black-box encapsulation**: a class is treated as a black box — other classes know only its interface (method names, parameters, return values), while its internal data and computation steps are hidden and can change freely. Example: a caller uses `bill.calculateAmount()` without knowing the tariff slabs used inside.

**(b) Visibility**

| UML | Java                | Meaning                    | Class | Package | Subclass | World |
|-----|---------------------|----------------------------|-------|---------|----------|-------|
| +   | public              | accessible everywhere      | yes   | yes     | yes      | yes   |
| #   | protected           | class, package, subclasses | yes   | yes     | yes      | no    |
| ~   | (default / package) | same package only          | yes   | yes     | no       | no    |
| -   | private             | inside the class only      | yes   | no      | no       | no    |

**(c) Information hiding vs encapsulation**

|                                    | Encapsulation                                                                                                      | Information hiding                                                                                    |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <b>What it is</b>                  | A <b>mechanism</b> : bundling data and the methods that operate on it into one unit (class) and controlling access | A <b>design principle</b> : hide design decisions that are likely to change behind a stable interface |
| <b>Focus</b>                       | Grouping + access modifiers                                                                                        | Protecting the rest of the system from change                                                         |
| <b>Relation</b>                    | A way to achieve information hiding                                                                                | The goal that encapsulation serves                                                                    |
| <b>Possible without the other?</b> | Yes — a class with public fields is encapsulated but hides nothing                                                 | Can also be achieved with modules, interfaces, packages                                               |
| <b>DUMS example</b>                | GarbageBin keeps level private with getter                                                                         | Callers use <code>isFull()</code> and never learn whether an ultrasonic or weight sensor is used      |

**Q13. Decomposition: association, aggregation, composition (DUMS)****Asked in:** 2021 Q2(a) · 2023 Q2(a) · 2024 Q2(a) · CT-1 Q2**Marks:** 5 / 6 / 4.75

The design principle of decomposition takes a whole thing and divides it into different parts. There are three types of relationships in decomposition which define the interaction between the whole and the parts: association, aggregation and composition. Explain those relationships and

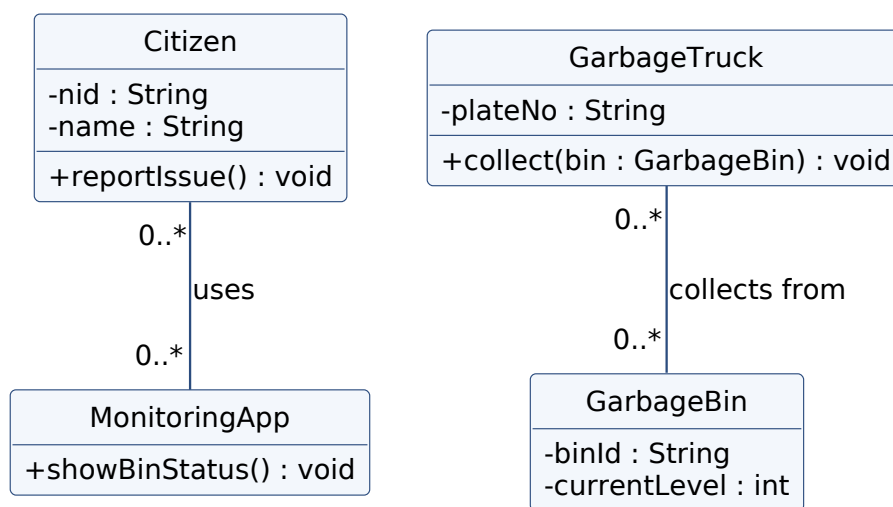
draw the UML diagram for each of them based on the Dhaka Urban Management System (use real-world examples as well).

## Decomposition

Decomposition breaks a complex whole into smaller parts that are easier to understand, build and maintain. The parts may be fixed or dynamic in number, may or may not live as long as the whole, and may or may not be shared. These differences give three relationships.

### 1. Association — “uses-a / knows-a”

A loose relationship in which two objects interact but **neither owns the other**; their lifetimes are independent. UML: a plain line (an arrow shows navigability), with multiplicities. Real world: a student plays sports. DUMS: a citizen uses the monitoring app; a garbage truck collects from many bins and a bin is emptied by many trucks.



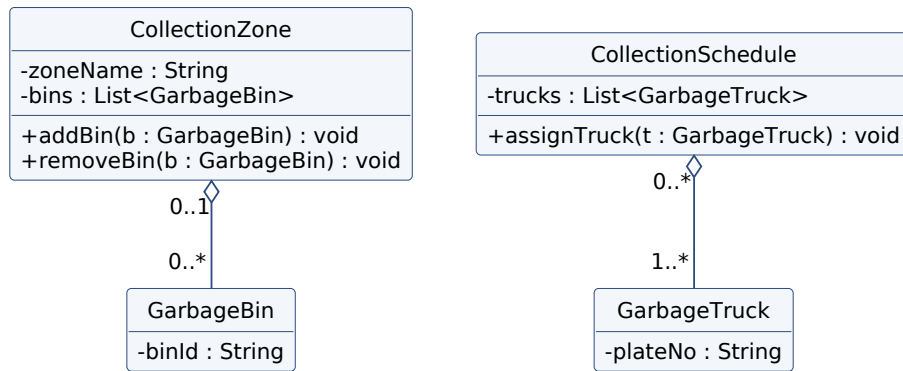
Association in DUMS

```

1 class GarbageTruck {
2     public void collect(GarbageBin bin) { // uses a bin, does not own it
3         bin.empty();
4     }
5 }
  
```

### 2. Aggregation — “has-a” (weak whole-part)

A whole contains parts, but a **part can exist without the whole** and may move to or be shared with another whole. UML: a **hollow diamond** at the whole. Real world: an airliner has a crew; crew members still exist when the plane is scrapped. DUMS: a collection zone has bins (a bin can be moved to another zone); a collection schedule has trucks.



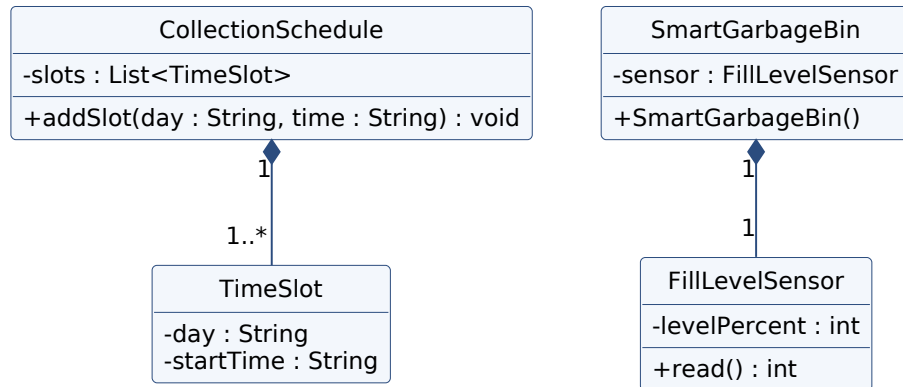
Aggregation in DUMS

```

1 class CollectionZone {
2     private List<GarbageBin> bins = new ArrayList<>();
3     public void addBin(GarbageBin b) { bins.add(b); } // created outside, passed in
4     public void removeBin(GarbageBin b) { bins.remove(b); } // part survives removal
5 }
  
```

### 3. Composition — “part-of / owns-a” (strong whole–part)

The whole **owns** its parts: a part **cannot exist without the whole**, is not shared, and is destroyed with the whole. UML: a **filled diamond** at the whole. Real world: a house is made of rooms. DUMS: a smart bin’s built-in fill-level sensor; the time slots of a collection schedule.



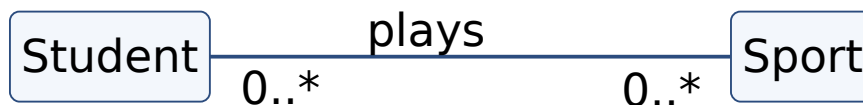
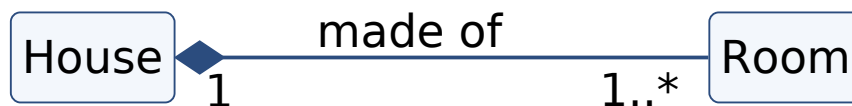
Composition in DUMS

```

1 class SmartGarbageBin {
2     private final FillLevelSensor sensor;
3     public SmartGarbageBin() {
4         sensor = new FillLevelSensor(); // created inside, dies with the bin
5     }
6 }
  
```

## Summary

|                  | Association                   | Aggregation                 | Composition                 |
|------------------|-------------------------------|-----------------------------|-----------------------------|
| <b>Meaning</b>   | uses / knows                  | has (weak)                  | owns / made of (strong)     |
| <b>Lifetime</b>  | independent                   | part outlives whole         | part dies with whole        |
| <b>Sharing</b>   | –                             | part may be shared          | never shared                |
| <b>UML</b>       | plain line                    | hollow diamond at whole     | filled diamond at whole     |
| <b>Java clue</b> | method parameter or reference | part passed in (setter/add) | part created in constructor |
| <b>DUMS</b>      | Truck – Bin                   | Zone ◊– Bin                 | SmartBin ◆– Sensor          |



Generic real-world examples from the slides

#### Q14. DUMS waste component: full class diagram and Java skeleton

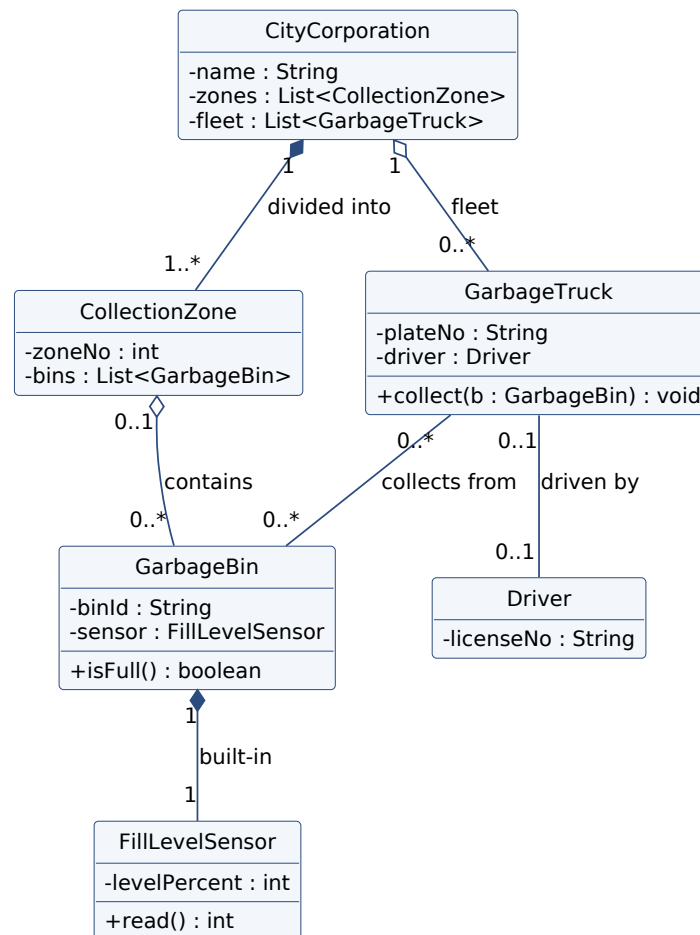
**Asked in:** Set-1 Q2(a) · New

**Marks:** 4.75

In DUMS, a city corporation is divided into collection zones. Each zone contains many smart garbage bins, and a bin can be moved to another zone. Every bin has a built-in fill-level sensor that cannot exist without the bin. Garbage trucks, each driven by a driver, collect waste from bins. Identify the association, aggregation and composition relationships, draw a UML class diagram with multiplicities, and write the Java skeleton showing each relationship.

#### Relationships identified

| Pair                              | Type               | Reason                                                     |
|-----------------------------------|--------------------|------------------------------------------------------------|
| CityCorporation – Collection-Zone | <b>Composition</b> | zones are created by and exist only within the corporation |
| CollectionZone – GarbageBin       | <b>Aggregation</b> | a bin can be moved to another zone                         |
| GarbageBin – FillLevelSensor      | <b>Composition</b> | built-in; cannot exist without the bin                     |
| CityCorporation – GarbageTruck    | <b>Aggregation</b> | trucks can be sold or transferred and still exist          |
| GarbageTruck – GarbageBin         | <b>Association</b> | a truck only uses bins while collecting                    |
| GarbageTruck – Driver             | <b>Association</b> | drivers change shifts; independent lives                   |



DUMS waste-collection component

## Java: Q\_Decomp.java

```

1 import java.util.ArrayList;
2 import java.util.List;
3
4 // ---- Composition: the sensor is created and owned by the bin ----
5 class FillLevelSensor {
6     private int levelPercent;
7     int read() { return levelPercent; }
8     void simulate(int p) { levelPercent = p; }
9 }
10
11 class GarbageBin {
12     private final String binId;
13     private final FillLevelSensor sensor;           // part
14
15     GarbageBin(String binId) {
16         this.binId = binId;
17         this.sensor = new FillLevelSensor();       // created INSIDE the whole
18     }
19     String getBinId() { return binId; }
20     boolean isFull() { return sensor.read() >= 90; }
21     void updateLevel(int p) { sensor.simulate(p); } // sensor never handed out
22 }
23
24 // ---- Aggregation: bins come from outside and may move to another zone ----

```

```

25 class CollectionZone {
26     private final String name;
27     private final List<GarbageBin> bins = new ArrayList<>();
28     CollectionZone(String name) { this.name = name; }
29     void addBin(GarbageBin b) { bins.add(b); }
30     void removeBin(GarbageBin b) { bins.remove(b); }
31     String getName() { return name; }
32 }
33
34 // ---- Association: objects only use each other ----
35 class Driver {
36     private final String name;
37     Driver(String name) { this.name = name; }
38     String getName() { return name; }
39 }
40
41 class GarbageTruck {
42     private final String plateNo;
43     private Driver driver; // association 0..1
44     GarbageTruck(String plateNo) { this.plateNo = plateNo; }
45     void setDriver(Driver d) { this.driver = d; }
46     void collect(GarbageBin bin) { // uses a bin, does not own it
47         System.out.println(plateNo + " (driver " + driver.getName() + ") collected " + bin
48             .getBinId());
49         bin.updateLevel(0);
50     }
51 }
52
53 class CityCorporation {
54     private final List<CollectionZone> zones = new ArrayList<>(); // composition
55     private final List<GarbageTruck> fleet = new ArrayList<>(); // aggregation
56     CityCorporation(String... zoneNames) {
57         for (String z : zoneNames) zones.add(new CollectionZone(z));
58     }
59     CollectionZone zone(int i) { return zones.get(i); }
60     void addTruck(GarbageTruck t) { fleet.add(t); }
61 }
62
63 public class Q_Decomposition {
64     public static void main(String[] args) {
65         CityCorporation city = new CityCorporation("Zone-1", "Zone-2");
66         GarbageBin bin = new GarbageBin("BIN-101");
67         city.zone(0).addBin(bin); // aggregation
68         bin.updateLevel(95);
69
70         GarbageTruck truck = new GarbageTruck("TRUCK-11");
71         truck.setDriver(new Driver("Karim"));
72         city.addTruck(truck);
73         if (bin.isFull()) truck.collect(bin); // association
74
75         city.zone(0).removeBin(bin); // the bin survives a zone change
76         city.zone(1).addBin(bin);
77         System.out.println(bin.getBinId() + " moved to " + city.zone(1).getName());
78     }
79 }

```

**Program output (compiled and run)**

```

1 TRUCK-11 (driver Karim) collected BIN-101
2 BIN-101 moved to Zone-2

```

The output shows the aggregation clearly: BIN-101 is removed from Zone-1 and still exists to be added to Zone-2, whereas its sensor is never visible outside the bin.

**Q15. Aggregation vs composition****Asked in:** Set-2 Q2(c)**Marks:** 2.25

Compare aggregation and composition with suitable DUMS examples.

|                           | Aggregation                                                    | Composition                                   |
|---------------------------|----------------------------------------------------------------|-----------------------------------------------|
| <b>Relationship</b>       | weak “has-a”                                                   | strong “part-of”                              |
| <b>Lifetime of part</b>   | independent of the whole                                       | same as the whole                             |
| <b>Sharing</b>            | a part may belong to several wholes over time                  | a part belongs to exactly one whole           |
| <b>Creation</b>           | part usually created outside and passed in                     | part created inside the whole                 |
| <b>Deleting the whole</b> | parts survive                                                  | parts are destroyed                           |
| <b>UML</b>                | hollow diamond ◇ at whole                                      | filled diamond ◆ at whole                     |
| <b>DUMS example</b>       | CollectionZone ◇– GarbageBin (bin re-assigned to another zone) | SmartGarbageBin ◆– FillLevelSensor (built-in) |
| <b>Other example</b>      | Ward ◇– Citizen (citizen can move)                             | Complaint ◆– StatusHistory                    |

**Two quick tests**

Ask “If the whole is deleted, does the part still make sense?” Yes → aggregation. Ask “Can the part be moved to another whole?” Yes → aggregation. Otherwise composition.

**Q16. FarmingMachine hierarchy: generalization for reuse****Asked in:** 2024 Q2(c) · CT-1 Q3**Marks:** 3 (CT: 6)

In an Agricultural Automation System, design a parent class FarmingMachine and child classes IrrigationSystem and Harvester (add PesticideSprayer) so that the children inherit attributes and behaviours from the parent. Explain which design principle is used. (CT-1:) the three machines were first written as independent classes with repeated attributes and methods; identify the limitations and show how the redesign improves reusability, maintainability and extensibility.

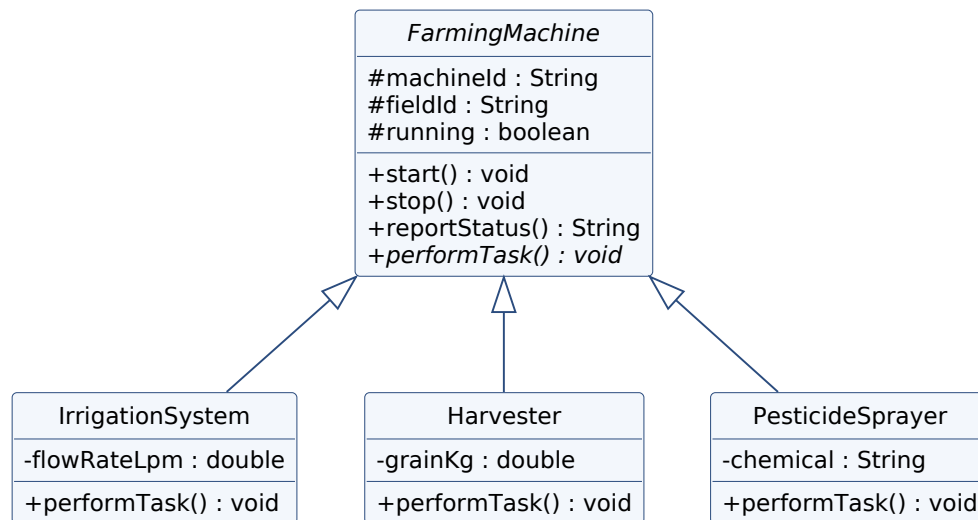
**Design principle used**

**Generalization** (implemented by **inheritance**): common attributes and behaviours are moved up into a general superclass and specialised subclasses inherit them. It models an “is-a” relationship — an irrigation system *is a* farming machine — and applies the DRY (Don’t Repeat Yourself) rule.

**Limitations of the independent classes**

- **Duplicated code:** machineId, fieldId, start(), stop(), reportStatus() written three times.
- **Inconsistent behaviour:** a bug fixed in Harvester.start() is still present in the other two.
- **No common type:** a dashboard cannot hold “all machines” in one list or start them in one loop.

- **Costly extension:** a new machine (e.g. seed planter) copies the same code again.



Generalization: common features in the abstract parent

#### Java: Q\_FarmingMachine.java

```

1  abstract class FarmingMachine {
2      protected final String machineId;
3      protected final String fieldId;
4      protected boolean running;
5
6      protected FarmingMachine(String machineId, String fieldId) {
7          this.machineId = machineId;
8          this.fieldId = fieldId;
9      }
10     public void start() { running = true; System.out.println(machineId + " started in
11     field " + fieldId); }
12     public void stop() { running = false; System.out.println(machineId + " stopped"); }
13     public String reportStatus() { return machineId + (running ? " is RUNNING" : " is IDLE
14     "); }
15     public abstract void performTask(); // every machine does its own job
16 }
17
18 class IrrigationSystem extends FarmingMachine {
19     private final double flowRateLpm;
20     IrrigationSystem(String id, String field, double flowRateLpm) {
21         super(id, field); // reuse parent initialisation
22         this.flowRateLpm = flowRateLpm;
23     }
24     @Override public void performTask() {
25         System.out.println(machineId + ": irrigating at " + flowRateLpm + " L/min");
26     }
27 }
28
29 class Harvester extends FarmingMachine {
30     private double grainKg;
31     Harvester(String id, String field) { super(id, field); }
32     @Override public void performTask() {
33         grainKg += 500;
34         System.out.println(machineId + ": harvesting, grain tank = " + grainKg + " kg");
35     }
36 }
  
```

```

34 }
35
36 class PesticideSprayer extends FarmingMachine {
37     private final String chemical;
38     PesticideSprayer(String id, String field, String chemical) {
39         super(id, field);
40         this.chemical = chemical;
41     }
42     @Override public void performTask() {
43         System.out.println(machineId + ": spraying " + chemical);
44     }
45 }
46
47 public class Q_FarmingMachine {
48     public static void main(String[] args) {
49         FarmingMachine[] machines = {
50             new IrrigationSystem("IRR-1", "F-07", 120.0),
51             new Harvester("HRV-2", "F-07"),
52             new PesticideSprayer("SPR-3", "F-09", "neem oil")
53         };
54         for (FarmingMachine m : machines) {           // one loop works for every machine
55             type
56             m.start();
57             m.performTask();
58             System.out.println(m.reportStatus());
59             m.stop();
60         }
61     }

```

#### Program output (compiled and run)

```

1 IRR-1 started in field F-07
2 IRR-1: irrigating at 120.0 L/min
3 IRR-1 is RUNNING
4 IRR-1 stopped
5 HRV-2 started in field F-07
6 HRV-2: harvesting, grain tank = 500.0 kg
7 HRV-2 is RUNNING
8 HRV-2 stopped
9 SPR-3 started in field F-09
10 SPR-3: spraying neem oil
11 SPR-3 is RUNNING
12 SPR-3 stopped

```

### How the redesign helps

- **Reusability:** common fields and methods are written once in `FarmingMachine`; subclasses reuse them via `super(...)`.
- **Maintainability:** a change to `start()` (e.g. logging) is made in one place.
- **Extensibility:** a `SeedPlanter` just extends the parent and implements `performTask()`; the loop in `main` works for it without change (polymorphism).

#### Common mistake

Use inheritance only for a true “is-a” relationship where the subclass can do *everything* the parent promises (Liskov Substitution Principle). Otherwise use interfaces or composition — see [Q17](#) and [Q18](#).

**Q17. The Whale extends Animal problem****Asked in:** 2023 Q2(b)**Marks:** 2.75

Consider the following Java implementation of a Whale class inherited from Animal. Can you identify any problem with this implementation? Explain.

```
public class Animal {
    private int numberOfLegs;
    private boolean hasTail;
    public Animal(int legs, boolean tail) {
        this.numberOfLegs = legs;
        this.hasTail = tail;
    }
    public void walk() { ... }
    public void run() { ... }
    public void eat() { ... }
}

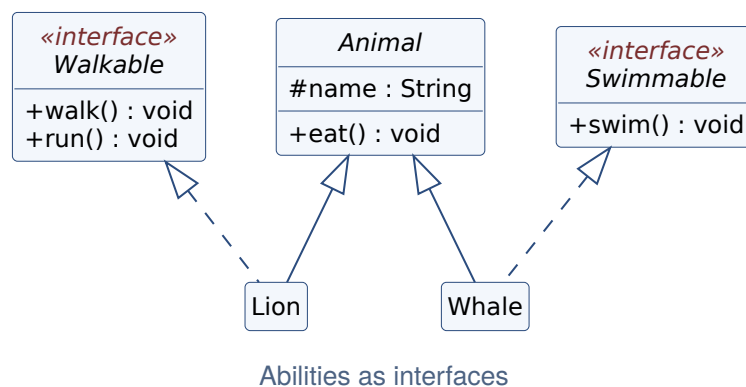
public class Whale extends Animal {
    public Whale() { super(0, true); }
    private void swim() { ... }
    public void run() { this.swim(); }
    public void walk() { this.swim(); }
}
```

**Problems**

1. **Wrong generalization.** Animal contains walk() and run(), which are *not* common to all animals. A whale is forced to inherit behaviour it cannot perform.
2. **Violation of the Liskov Substitution Principle.** Code written for Animal expects walk() to walk. When a Whale is substituted, walk() and run() secretly swim — the behaviour is misleading and the subclass cannot truly replace the parent.
3. **Refused bequest (code smell).** The subclass overrides inherited methods only to cancel or change their meaning.
4. **Hacks in the constructor.** super(0, true) sets legs to 0 just to fit a model built for land animals.
5. **Real behaviour is hidden.** swim() is private, so no client can ask a whale to swim directly or treat swimmers polymorphically.

**Correct design**

Keep only truly common behaviour in Animal, and express optional abilities as interfaces.



```

1  abstract class Animal {
2      protected String name;
3      public void eat() { System.out.println(name + " eats"); }
4  }
5  interface Walkable { void walk(); void run(); }
6  interface Swimmable { void swim(); }
7
8  class Lion extends Animal implements Walkable {
9      public void walk() { System.out.println("Lion walks"); }
10     public void run() { System.out.println("Lion runs"); }
11 }
12 class Whale extends Animal implements Swimmable {
13     public void swim() { System.out.println("Whale swims"); }
14 }

```

### Q18. SolarIrrigationPump cannot refuel

Asked in: Set-1 Q2(c) · New

Marks: 3

Consider the following AAS code. Identify the design problem and redesign the hierarchy so that it does not happen.

```

public class FarmingMachine {
    protected String machineId;
    protected double fuelLevel;
    public void start() { /* ... */ }
    public void stop() { /* ... */ }
    public void refuel(double litres) { fuelLevel += litres; }
}

public class SolarIrrigationPump extends FarmingMachine {
    @Override
    public void refuel(double litres) {
        throw new UnsupportedOperationException("Solar pump has no fuel tank");
    }
}

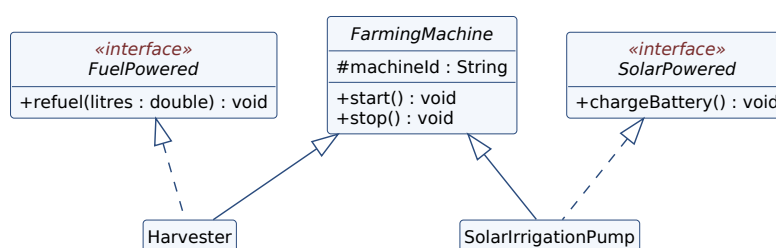
```

### Problem

FarmingMachine assumes every machine burns fuel, but a solar pump does not. The subclass inherits `refuel()` and throws `UnsupportedOperationException`. This violates the **Liskov Substitution Principle**: any code such as `for (FarmingMachine m : fleet) m.refuel(50);` compiles, but crashes at *run time* when it meets a solar pump. It is also a **refused bequest** smell caused by putting non-general behaviour into the parent.

### Redesign

Keep FarmingMachine truly general (start/stop) and move power-specific behaviour into interfaces. Now a solar pump *cannot* be passed to `refuelAll()` — the mistake is caught at *compile time*.



## Power source as interfaces

## Java: Q\_SolarFix.java

```

1  abstract class FarmingMachine {
2      protected final String machineId;
3      protected FarmingMachine(String machineId) { this.machineId = machineId; }
4      public void start() { System.out.println(machineId + " started"); }
5      public void stop() { System.out.println(machineId + " stopped"); }
6  }
7
8  interface FuelPowered { void refuel(double litres); }
9  interface SolarPowered { void chargeBattery(); }
10
11 class Harvester extends FarmingMachine implements FuelPowered {
12     private double fuelLitres;
13     Harvester(String id) { super(id); }
14     @Override public void refuel(double litres) {
15         fuelLitres += litres;
16         System.out.println(machineId + " fuel = " + fuelLitres + " L");
17     }
18 }
19
20 class SolarIrrigationPump extends FarmingMachine implements SolarPowered {
21     private int batteryPercent = 40;
22     SolarIrrigationPump(String id) { super(id); }
23     @Override public void chargeBattery() {
24         batteryPercent = 100;
25         System.out.println(machineId + " battery = " + batteryPercent + " percent");
26     }
27 }
28
29 public class Q_SolarFix {
30     static void refuelAll(FuelPowered[] fleet) { // accepts ONLY machines that use
        fuel
31         for (FuelPowered f : fleet) f.refuel(50);
32     }
33     public static void main(String[] args) {
34         Harvester h = new Harvester("HRV-1");
35         SolarIrrigationPump p = new SolarIrrigationPump("SIP-1");
36         h.start();
37         p.start();
38         refuelAll(new FuelPowered[] { h }); // a solar pump cannot even be passed
        here
39         p.chargeBattery();
40     }
41 }

```

## Program output (compiled and run)

```

1  HRV-1 started
2  SIP-1 started
3  HRV-1 fuel = 50.0 L
4  SIP-1 battery = 100 percent

```

**Q19. SmartDroneSprayer: interface inheritance****Asked in:** Set-2 Q2(b) · New**Marks:** 3

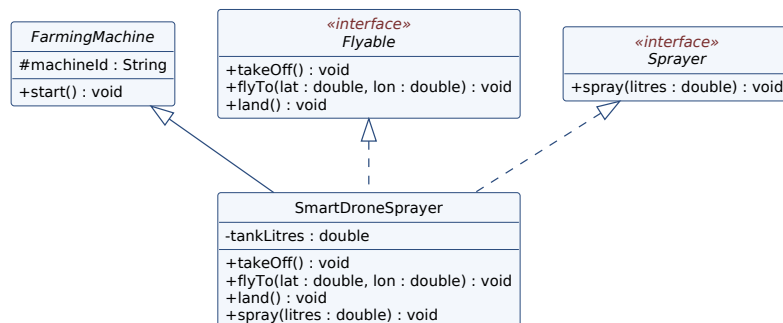
An AAS smart drone is a farming machine that can both fly over fields and spray pesticide. Java does not allow a class to extend more than one class. Show with a UML diagram and Java code how interface inheritance solves this.

**Why Java forbids multiple class inheritance**

If Drone extended both Aircraft and Sprayer and both had a `start()` method with different code, which one should run? This ambiguity is the **diamond problem**. Java avoids it by allowing only one superclass.

**Solution**

A class may **extend one class** and **implement many interfaces**. Interfaces declare abilities (contracts) without inherited state, so there is no conflict: the class itself provides the one implementation.



One superclass + two capability interfaces

**Java: Q\_Drone.java**

```

1  abstract class FarmingMachine {
2      protected final String machineId;
3      protected FarmingMachine(String id) { machineId = id; }
4      public void start() { System.out.println(machineId + " powered on"); }
5  }
6
7  interface Flyable {
8      void takeOff();
9      void flyTo(double lat, double lon);
10     void land();
11 }
12
13 interface Sprayer {
14     void spray(double litres);
15 }
16
17 class SmartDroneSprayer extends FarmingMachine implements Flyable, Sprayer {
18     private double tankLitres = 10;
19     SmartDroneSprayer(String id) { super(id); }
20
21     @Override public void takeOff() { System.out.println(machineId + " taking off"); }
22     @Override public void flyTo(double lat, double lon) {
23         System.out.println(machineId + " flying to (" + lat + ", " + lon + ")");
24     }
25     @Override public void land() { System.out.println(machineId + " landed"); }
  
```

```
26     @Override public void spray(double litres) {
27         tankLitres -= litres;
28         System.out.println(machineId + " sprayed " + litres + " L, tank left " +
29         tankLitres + " L");
30     }
31 }
32 public class Q_Drone {
33     public static void main(String[] args) {
34         SmartDroneSprayer drone = new SmartDroneSprayer("DRN-01");
35         FarmingMachine machine = drone;           // IS-A FarmingMachine
36         Flyable flyer = drone;                   // IS-A Flyable
37         Sprayer sprayer = drone;                 // IS-A Sprayer
38
39         machine.start();
40         flyer.takeOff();
41         flyer.flyTo(24.37, 88.60);
42         sprayer.spray(2.5);
43         flyer.land();
44     }
45 }
```

**Program output (compiled and run)**

```
1 DRN-01 powered on
2 DRN-01 taking off
3 DRN-01 flying to (24.37, 88.6)
4 DRN-01 sprayed 2.5 L, tank left 7.5 L
5 DRN-01 landed
```

**Benefits:** the same object can be used wherever a FarmingMachine, a Flyable or a Sprayer is expected; clients depend only on the capability they need (Interface Segregation); new capabilities can be added as new interfaces without touching the class hierarchy.

## CHAPTER 3

# Section A — Design Complexity and Code Quality

### Q20. Coupling and cohesion

**Asked in:** New (Slides M3) · background for 2021 Q3, 2023 Q3

**Marks:** 3–4

Define coupling and cohesion. How can the coupling of a module be evaluated? Why should a design aim for loose coupling and high cohesion? Give DUMS examples.

### Coupling

Coupling is the **degree of interdependence** between modules — how much one module relies on the internals of another. **Tight coupling** means a change in one module forces changes in others. The slides evaluate coupling with three criteria:

- **Degree:** the number of connections between a module and others. Fewer, smaller connections (few parameters) are better.
- **Ease:** how obvious the connections are. A module should be easy to connect to without understanding its implementation; hidden meanings (e.g. “flag 1 means road”) make connections hard.
- **Flexibility:** how interchangeable the other modules are. Loosely coupled modules can be replaced by another implementation without changes.

From loosest to tightest: data coupling (simple parameters) → stamp coupling (whole structures) → **control coupling** (a flag tells the other module what to do) → common coupling (shared global data) → content coupling (one module modifies another’s internals).

### Cohesion

Cohesion is **how clearly a module focuses on a single task**. In a highly cohesive class, all attributes and methods work together for one purpose. A class that does several unrelated things has low cohesion and is hard to understand, reuse and change. From best to worst: functional → sequential → communicational → procedural → temporal → logical (selected by a flag) → coincidental.

### Why loose coupling and high cohesion

Changes stay local, classes are reusable and testable independently, and the design is easy to understand. The two usually go together: splitting a low-cohesion class also removes the flags that caused tight coupling. There is a balance — over-splitting creates many tiny classes with many connections.

### DUMS examples

**Tight coupling:** BillingModule reads and writes the complaints table directly; renaming a column breaks billing. **Low cohesion:** ComplaintManager stores complaints, sends SMS, prints reports and calculates bills.

### Q21. ServiceManager: evaluate and redesign (DUMS)

**Asked in:** Set-1 Q3 · Slides M3 exercise

**Marks:** 4.5 + 4.25

The ServiceManager class of DUMS handles two municipal services depending on a control flag:

```
public class ServiceManager {  
    public void processRequest(int controlFlag) {  
        switch (controlFlag) {  
            case 0: this.handleWasteCollection(); break;  
            case 1: this.handleRoadMaintenance(); break;  
            default: throw new UnknownControlFlagException();  
        }  
    }  
    private void handleWasteCollection() { /* find truck, assign route ... */ }  
    private void handleRoadMaintenance() { /* find crew, order material ... */ }  
}
```

(a) Evaluate the design in terms of coupling and cohesion with justification. (b) Suggest a better design with a UML class diagram, and show how a new *water-logging relief* service can be added.

### (a) Cohesion: LOW

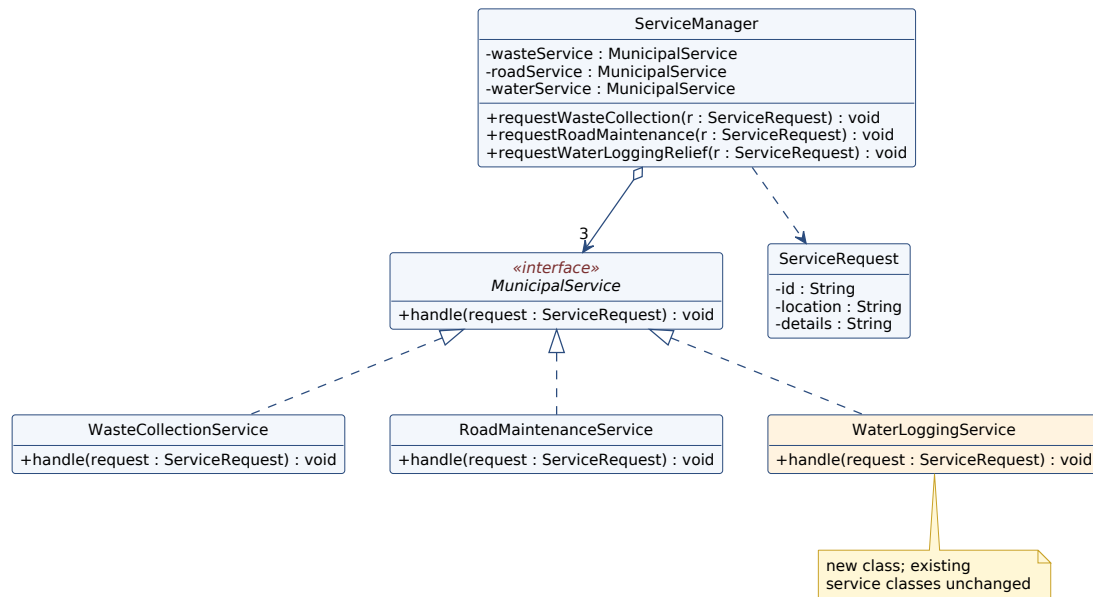
- The class mixes **two unrelated responsibilities**: waste collection (trucks, routes) and road maintenance (repair crews, materials). Their logic has nothing in common except that both are municipal.
- It has **two reasons to change** — a change in truck routing and a change in repair scheduling both edit the same class.
- This is *logical cohesion*: unrelated actions grouped and selected by a flag. Every new service makes the class bigger.

### (a) Coupling: TIGHT (control coupling)

- **Degree**: callers pass a control flag that decides the internal logic of `ServiceManager` — the caller controls the callee.
- **Ease**: the connection is not obvious; a caller must know that 0 means waste and 1 means road. `processRequest(1)` tells a reader nothing.
- **Flexibility**: the road-maintenance implementation cannot be replaced without editing `ServiceManager`; changing flag values or adding a service forces changes in the class *and* in every caller.
- Errors are found late: a wrong flag compiles and fails only at run time with `UnknownControlFlagException`.

### (b) Better design

Give each service its own class (high cohesion) behind a common interface, and keep `ServiceManager` as a thin coordinator with **explicit** method names and no flags (loose coupling). Services are injected through the constructor, so implementations can be swapped.



Redesigned ServiceManager (new service highlighted)

## Java: Q\_ServiceManager.java

```

1 class ServiceRequest {
2     private final String id, location, details;
3     ServiceRequest(String id, String location, String details) {
4         this.id = id; this.location = location; this.details = details;
5     }
6     String getId() { return id; }
7     String getLocation() { return location; }
8     String getDetails() { return details; }
9 }
10
11 interface MunicipalService {
12     void handle(ServiceRequest request);
13 }
14
15 class WasteCollectionService implements MunicipalService {           // ONE job
16     @Override public void handle(ServiceRequest r) {
17         System.out.println("[Waste] " + r.getId() + ": truck sent to " + r.getLocation()
18             + " (" + r.getDetails() + ")");
19     }
20 }
21
22 class RoadMaintenanceService implements MunicipalService {           // ONE job
23     @Override public void handle(ServiceRequest r) {
24         System.out.println("[Road] " + r.getId() + ": repair crew scheduled at " + r.
25             getLocation()
26             + " (" + r.getDetails() + ")");
27     }
28 }
29
30 class WaterLoggingService implements MunicipalService {               // NEW class, nothing
31     // else edited
32     @Override public void handle(ServiceRequest r) {
33         System.out.println("[Water] " + r.getId() + ": pump team sent to " + r.getLocation()
34             + " (" + r.getDetails() + ")");
35     }
36 }

```

```

33     }
34 }
35
36 class ServiceManager {                                // coordinator only
37     private final MunicipalService wasteService;
38     private final MunicipalService roadService;
39     private final MunicipalService waterService;
40
41     ServiceManager(MunicipalService waste, MunicipalService road, MunicipalService water)
42     {
43         this.wasteService = waste;
44         this.roadService = road;
45         this.waterService = water;
46     }
47     public void requestWasteCollection(ServiceRequest r) { wasteService.handle(r); }
48     public void requestRoadMaintenance(ServiceRequest r) { roadService.handle(r); }
49     public void requestWaterLoggingRelief(ServiceRequest r) { waterService.handle(r); }
50 }
51
52 public class Q_ServiceManager {
53     public static void main(String[] args) {
54         ServiceManager manager = new ServiceManager(
55             new WasteCollectionService(), new RoadMaintenanceService(), new
56             WaterLoggingService());
57         manager.requestWasteCollection(new ServiceRequest("REQ-1", "Mirpur-10", "bin
58             overflow"));
59         manager.requestRoadMaintenance(new ServiceRequest("REQ-2", "Farmgate", "pothole"));
60         ;
61         manager.requestWaterLoggingRelief(new ServiceRequest("REQ-3", "Shantinagar", "knee
62             -deep water"));
63     }
64 }

```

#### Program output (compiled and run)

```

1 [Waste] REQ-1: truck sent to Mirpur-10 (bin overflow)
2 [Road] REQ-2: repair crew scheduled at Farmgate (pothole)
3 [Water] REQ-3: pump team sent to Shantinagar (knee-deep water)

```

## Adding water-logging relief

Write one new class `WaterLoggingService` implements `MunicipalService` and add one delegating method to the coordinator. The existing waste and road service classes are **not touched**, no flag value is introduced, and callers of existing methods are unaffected.

## Evaluation of the new design

**Cohesion is high** — each service class does one job. **Coupling is loose** — the coordinator depends only on the `MunicipalService` interface, connections are explicit and few, and any service can be replaced by another implementation.

### Q22. Sensor class with a control flag

**Asked in:** 2023 Q3(a),(b)

**Marks:** 5 + 3.75

A class `Sensor` has two purposes: getting humidity and getting temperature readings. Its `get()` method takes flag 0 to return humidity and 1 to return temperature:

```

public class Sensor {
    private double humidity, temperature;
    public double get(int controlFlag) {
        switch (controlFlag) {
            case 0: return this.humidity;
            case 1: return this.temperature;
            default: throw new UnknownControlFlagException();
        }
    }
}

```

(a) Evaluate the design in terms of coupling and cohesion, with justification in each case. (b) Suggest a better design that meets acceptable coupling and cohesion.

#### Note on the question

The printed paper declares `public void get(int controlFlag)` but returns values — that would not compile. Write the return type as `double` and mention it briefly.

#### (a) Cohesion: LOW

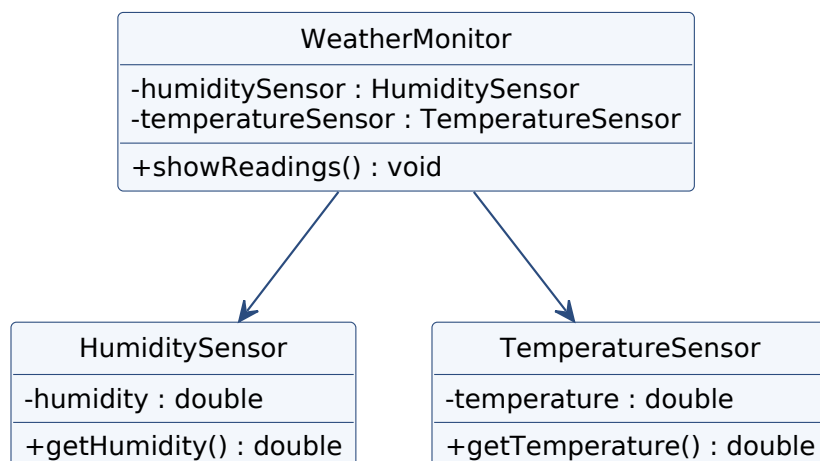
- One class performs **two unrelated tasks**: humidity sensing and temperature sensing. They use different hardware and change for different reasons.
- `get()` does not do one thing; what it does depends on an outside flag (logical cohesion).
- A client that only needs temperature still depends on humidity code; the class cannot be reused for a device that has only one of the sensors.

#### (a) Coupling: TIGHT

- **Control coupling**: the caller passes a flag that selects the internal path.
- **Ease is poor**: `get(1)` is meaningless without reading the implementation; magic numbers cause mistakes.
- **Flexibility is poor**: adding a pressure reading changes `get()`, the flag meanings and possibly every client; a wrong flag fails only at run time.

#### (b) Better design

Split the responsibilities into two cohesive classes with self-explaining methods; a client such as `WeatherMonitor` depends only on what it needs.



## Separated sensors

## Java: Q\_Sensor.java

```
1 class HumiditySensor {
2     private double humidity = 68.5;
3     public double getHumidity() { return humidity; }
4 }
5
6 class TemperatureSensor {
7     private double temperature = 31.2;
8     public double getTemperature() { return temperature; }
9 }
10
11 class WeatherMonitor { // depends only on what it really needs
12     private final HumiditySensor humiditySensor;
13     private final TemperatureSensor temperatureSensor;
14
15     WeatherMonitor(HumiditySensor h, TemperatureSensor t) {
16         humiditySensor = h;
17         temperatureSensor = t;
18     }
19     void showReadings() {
20         System.out.println("Humidity    = " + humiditySensor.getHumidity() + " %");
21         System.out.println("Temperature = " + temperatureSensor.getTemperature() + " C");
22     }
23 }
24
25 public class Q_Sensor {
26     public static void main(String[] args) {
27         new WeatherMonitor(new HumiditySensor(), new TemperatureSensor()).showReadings();
28     }
29 }
```

## Program output (compiled and run)

```
1 Humidity    = 68.5 %
2 Temperature = 31.2 C
```

Now each class has one purpose (high cohesion), calls are explicit with no flags (loose coupling), and a new PressureSensor is a new class that changes nothing. If polymorphism is needed, both can implement a common interface `Sensor { double read(); }`.

## Q23. SmartPhone: coupling, cohesion and separation of concerns

**Asked in:** 2021 Q3(b),(c)**Marks:** 2.75 + 5

Consider the following SmartPhone class.

```
public class SmartPhone {  
    private byte camera;  
    private byte phone;  
    public SmartPhone() { ... }  
    public void takePhoto() { ... }  
    public void savePhoto() { ... }  
    public void cameraFlash() { ... }  
    public void makePhoneCall() { ... }  
    public void encryptOutgoingSound() { ... }  
    public void decipherIncomingSound() { ... }  
}
```

(b) Explain the problems with the class in terms of coupling and cohesion. (c) How can you apply the separation of concerns principle to overcome the problems? Draw a UML diagram of your proposed solution.

### (b) Problems

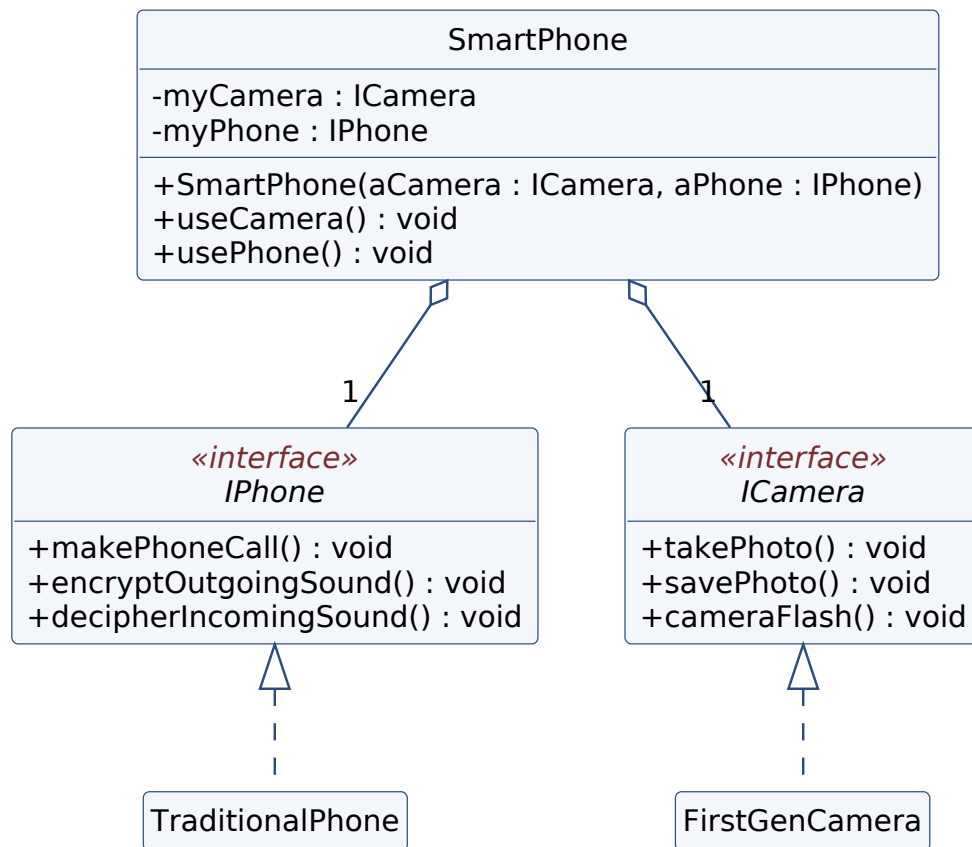
**Low cohesion:** the class has two separate concerns — a *camera* (take/save photo, flash) and a *phone* (call, encrypt, decrypt sound). Its data (camera, phone) and methods form two unrelated groups.

**Tight coupling:** the camera and phone code live in one class and share its internals. A change in camera hardware (e.g. a new sensor needing a different savePhoto) forces changing and retesting the class that also makes phone calls. The camera cannot be reused in a tablet, and the phone part cannot be upgraded independently. Any client that only makes calls still depends on camera code.

### (c) Separation of concerns

**Separation of concerns (SoC)** means dividing a system into parts so that each part addresses one distinct concern. Here:

1. Identify concerns: camera and phone.
2. Put each concern behind its own **interface**: ICamera, IPhone.
3. Implement them in separate classes: FirstGenCamera, TraditionalPhone.
4. Make SmartPhone a coordinator that **has** a camera and a phone (supplied through its constructor) and delegates to them.



SmartPhone after separation of concerns

## Java: Q\_SmartPhone.java

```

1 interface ICamera {
2     void takePhoto();
3     void savePhoto();
4     void cameraFlash();
5 }
6
7 interface IPhone {
8     void makePhoneCall();
9     void encryptOutgoingSound();
10    void decipherIncomingSound();
11 }
12
13 class FirstGenCamera implements ICamera {
14     public void takePhoto() { System.out.println("Photo taken"); }
15     public void savePhoto() { System.out.println("Photo saved"); }
16     public void cameraFlash() { System.out.println("Flash fired"); }
17 }
18
19 class TraditionalPhone implements IPhone {
20     public void makePhoneCall() { System.out.println("Calling..."); }
21     public void encryptOutgoingSound() { System.out.println("Outgoing voice encrypted"); }
22     public void decipherIncomingSound() { System.out.println("Incoming voice decrypted"); }
23 }
24
25 class SmartPhone {
26     // coordinator
  
```

```

26     private final ICamera myCamera;
27     private final IPhone myPhone;
28
29     public SmartPhone(ICamera aCamera, IPhone aPhone) { // parts supplied from outside
30         this.myCamera = aCamera;
31         this.myPhone = aPhone;
32     }
33     public void useCamera() { myCamera.takePhoto(); myCamera.savePhoto(); }
34     public void usePhone() { myPhone.encryptOutgoingSound(); myPhone.makePhoneCall(); }
35 }
36
37 public class Q_SmartPhone {
38     public static void main(String[] args) {
39         SmartPhone phone = new SmartPhone(new FirstGenCamera(), new TraditionalPhone());
40         phone.useCamera();
41         phone.usePhone();
42     }
43 }

```

#### Program output (compiled and run)

```

1 Photo taken
2 Photo saved
3 Outgoing voice encrypted
4 Calling...

```

**Result:** each class has a single concern (high cohesion); SmartPhone depends only on interfaces (loose coupling); a SecondGenCamera can be plugged in without editing SmartPhone; the camera can be reused in a Tablet; each part can be tested alone.

#### Slide code typo

The slide writes `return this.myCamera.takePhoto();` inside a void method — that does not compile. Call the method without return, as above.

### Q24. Conceptual integrity

Asked in: 2021 Q3(a) · 2024 Q5(b)

Marks: 1 / 3.75

What do you understand by conceptual integrity? Explain the ways to achieve conceptual integrity during the design and development of software.

#### Definition

Conceptual integrity is the **consistency and coherence of a software design as a whole**: the system follows one set of design ideas, naming conventions, patterns and style, so that it looks as if it were designed by a single mind even though many people built it. Fred Brooks called it the most important consideration in system design. A system with conceptual integrity is easier to learn, use, extend and maintain.

#### Ways to achieve it

1. **Communication within the team.** Code reviews and agile practices such as daily stand-ups and sprint retrospectives keep everyone aligned on design decisions.
2. **Consistent design principles and programming methods.** Everyone applies the same principles (encapsulation, low coupling, SRP) and the same way of solving recurring problems.
3. **A well-defined architecture.** A clear overall structure (e.g. layered or MVC) that all modules

follow.

4. **A small core group / chief architect** who reviews and accepts design decisions and commits, so that one coherent vision is maintained.
5. **Naming conventions and coding standards** shared by all.
6. **Consistent use of design patterns** for the same kind of problem.
7. **Shared documentation and UML models**, plus regular **refactoring** to remove inconsistencies.

## DUMS illustration

**With integrity:** every module uses the same Complaint status values, the same Observer-based notification mechanism and the same REST naming (/api/v1/complaints, /api/v1/bills). **Without integrity:** billing calls the person customerId, complaints use citizenNid; one module pushes notifications while another polls the database every minute.

### Q25. OrderProcessor code smell; comments as a code smell

**Asked in:** 2024 Q3(a),(c)

**Marks:** 3.5 + 1.75

(a) Consider the following code snippet. Identify the code smell present, describe why it is a problem and how you would refactor the code.

```
public class OrderProcessor {
    ...
    public void process(Order order) {
        order.calculateTotal();
        order.applyDiscount();
        order.generateInvoice();
    }
}
```

(c) How can comments be considered a code smell? Provide examples of when comments indicate poor code quality.

### (a) Code smell: Feature Envy

process() is **more interested in the data and behaviour of Order than in its own class**: it only calls Order's methods one after another. The class also shows *inappropriate intimacy* — it knows the internal checkout steps of an order and their exact order (total → discount → invoice).

### Why it is a problem

- **Knowledge is in the wrong place:** the checkout algorithm belongs to Order but lives outside it.
- **Fragile:** if a tax step is added, OrderProcessor (and every other class that copied these three calls) must change — shotgun surgery.
- **Error-prone:** another client can call the steps in the wrong order (invoice before discount) or skip one.
- **Tight coupling** between OrderProcessor and the internal protocol of Order.

### Refactoring: Move Method (“Tell, don’t ask”)

Move the sequence into Order as one public operation checkout() and make the steps private. OrderProcessor now sends a single message.

Java: Q\_OrderProcessor.java

```
1 class Order {
2     private final double[] itemPrices;
```

```

3     private double total;
4
5     Order(double... itemPrices) { this.itemPrices = itemPrices; }
6
7     public void checkout() {                // the steps live WITH the data they use
8         calculateTotal();
9         applyDiscount();
10        generateInvoice();
11    }
12    private void calculateTotal() { for (double p : itemPrices) total += p; }
13    private void applyDiscount() { if (total > 5000) total *= 0.90; }
14    private void generateInvoice() { System.out.println("Invoice generated. Payable = Tk.
    " + total); }
15 }
16
17 class OrderProcessor {
18     public void process(Order order) {
19         order.checkout();                // one message; no knowledge of internal
    steps
20     }
21 }
22
23 public class Q_OrderProcessor {
24     public static void main(String[] args) {
25         new OrderProcessor().process(new Order(2500, 3000, 700));
26     }
27 }

```

#### Program output (compiled and run)

```
1 Invoice generated. Payable = Tk. 5580.0
```

If OrderProcessor then does nothing else, it has become a *Middle Man* and can be removed.

### (c) Comments as a code smell

Comments themselves are not bad, but when they are needed to *explain unclear code* they act as “deodorant” hiding a smell. The fix is usually to make the code self-explanatory.

| Smelly comment                                             | Better code                              |
|------------------------------------------------------------|------------------------------------------|
| int d; // elapsed days                                     | rename: int elapsedDays;                 |
| // validate // calculate // save blocks in one long method | extract validate(), calculate(), save()  |
| if (flag == 1) // road service                             | use an enum or a separate method         |
| i++; // increment i                                        | delete the redundant comment             |
| Commented-out old code                                     | delete it; version control keeps history |
| Outdated comment that no longer matches the code           | update the code/comment or remove it     |

**Good comments** explain *why* a decision was taken, document public APIs (Javadoc), or warn about consequences.

#### Q26. Single Responsibility Principle

Asked in: 2024 Q3(b)

Marks: 3.5

What is the Single Responsibility Principle (SRP)? Explain with an example showing a violation and a correct implementation.

## Definition

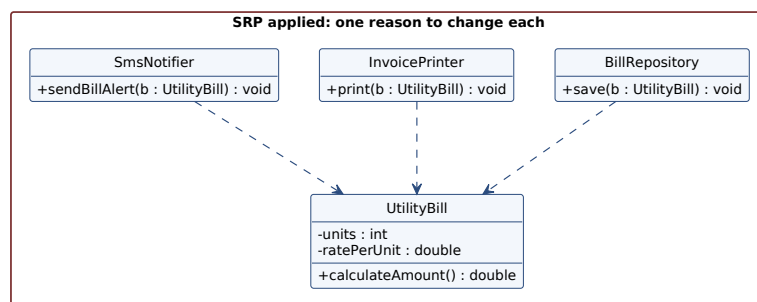
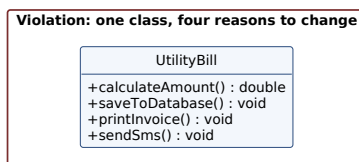
“A class should have only one reason to change.” (Robert C. Martin). Each class should be responsible for one part of the functionality — one job, serving one kind of stakeholder. SRP is cohesion applied at class level.

## Violation

```
public class UtilityBill {
    private int units;
    private double ratePerUnit;
    public double calculateAmount() { return units * ratePerUnit; } // tariff logic
    public void saveToDatabase() { /* JDBC code */ } // persistence
    public void printInvoice() { /* layout, fonts */ } // presentation
    public void sendSms() { /* SMS gateway API */ } // communication
}
```

UtilityBill has **four reasons to change**: the tariff rules (finance department), the database (DBA), the invoice layout (designers) and the SMS provider (IT). A change in the SMS API forces recompiling and retesting the class that calculates money — risky and unrelated.

## Correct implementation



Splitting responsibilities

Java: Q\_SRP.java

```
1 class UtilityBill { // changes only if tariff rules
2     change
3     private final String houseId;
4     private final int units;
5     private final double ratePerUnit;
6     UtilityBill(String houseId, int units, double ratePerUnit) {
7         this.houseId = houseId; this.units = units; this.ratePerUnit = ratePerUnit;
8     }
9     String getHouseId() { return houseId; }
10    double calculateAmount() { return units * ratePerUnit; }
11 }
12
13 class BillRepository { // changes only if storage changes
14     void save(UtilityBill b) { System.out.println("Saved bill of " + b.getHouseId()); }
15 }
16
17 class InvoicePrinter { // changes only if invoice layout
18     changes
19     void print(UtilityBill b) {
20         System.out.println("INVOICE " + b.getHouseId() + " : Tk. " + b.calculateAmount());
21     }
22 }
```

```

20 }
21
22 class SmsNotifier {                                // changes only if SMS provider
    changes
23     void sendBillAlert(UtilityBill b) {
24         System.out.println("SMS to owner of " + b.getHouseId() + ": bill Tk. " + b.
            calculateAmount());
25     }
26 }
27
28 public class Q_SRP {
29     public static void main(String[] args) {
30         UtilityBill bill = new UtilityBill("H-221", 180, 7.5);
31         new BillRepository().save(bill);
32         new InvoicePrinter().print(bill);
33         new SmsNotifier().sendBillAlert(bill);
34     }
35 }

```

#### Program output (compiled and run)

```

1 Saved bill of H-221
2 INVOICE H-221 : Tk. 1350.0
3 SMS to owner of H-221: bill Tk. 1350.0

```

## Benefits

Each class is small, easy to understand and test; changes are isolated (a new SMS provider touches only `SmsNotifier`); classes are reusable (`InvoicePrinter` for water and gas bills); several developers can work without conflicts.

### Q27. Switch-statement smell and the Open/Closed Principle

**Asked in:** Set-2 Q3(a) · New

**Marks:** 3.5

Consider the following code of a DUMS toll plaza. Identify the code smell. Refactor it with polymorphism so that a new vehicle type can be added without modifying existing code, and name the principle applied.

```

public class TollCalculator {
    public double calculateToll(String vehicleType, int axles) {
        switch (vehicleType) {
            case "CAR":         return 750;
            case "BUS":         return 1500;
            case "TRUCK":       return 1000 + axles * 500;
            case "MOTORCYCLE":  return 100;
            default:            throw new IllegalArgumentException("Unknown vehicle");
        }
    }
}

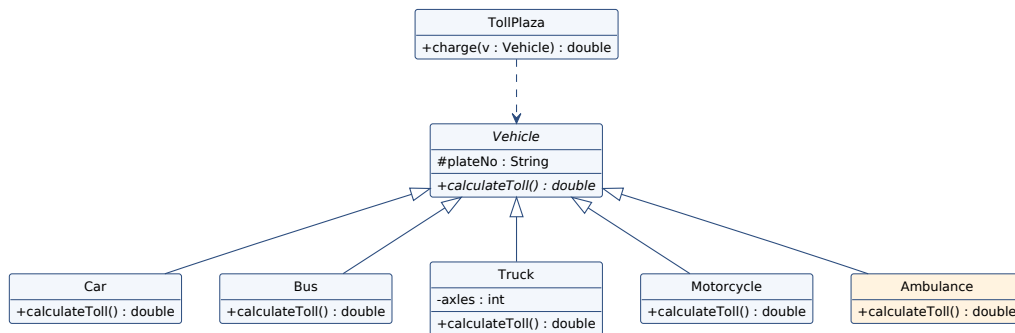
```

## Smell: Switch Statements (type code + conditional)

The behaviour is selected by checking a type string. Problems: every new vehicle type (ambulance, CNG auto-rickshaw) requires editing this method; the same switch tends to be copied elsewhere (speed limits, lane assignment), so one change becomes many (shotgun surgery); a misspelt type string compiles and fails only at run time.

## Refactoring: Replace Conditional with Polymorphism

Create an abstract `Vehicle` with `calculateToll()`; each subclass implements its own rule. TollPlaza simply calls `v.calculateToll()`.



Polymorphic toll calculation (new type highlighted)

### Java: Q\_Toll.java

```

1  abstract class Vehicle {
2      protected final String plateNo;
3      protected Vehicle(String plateNo) { this.plateNo = plateNo; }
4      public abstract double calculateToll();           // each type knows its own toll
5  }
6
7  class Car extends Vehicle {
8      Car(String p) { super(p); }
9      @Override public double calculateToll() { return 750; }
10 }
11
12 class Bus extends Vehicle {
13     Bus(String p) { super(p); }
14     @Override public double calculateToll() { return 1500; }
15 }
16
17 class Truck extends Vehicle {
18     private final int axles;
19     Truck(String p, int axles) { super(p); this.axles = axles; }
20     @Override public double calculateToll() { return 1000 + axles * 500; }
21 }
22
23 class Motorcycle extends Vehicle {
24     Motorcycle(String p) { super(p); }
25     @Override public double calculateToll() { return 100; }
26 }
27
28 class Ambulance extends Vehicle {                     // NEW type: TollPlaza is not edited
29     Ambulance(String p) { super(p); }
30     @Override public double calculateToll() { return 0; }
31 }
32
33 class TollPlaza {                                     // closed for modification
34     public double charge(Vehicle v) {
35         double toll = v.calculateToll();               // polymorphic call replaces the
36         switch
37             System.out.println(v.plateNo + " pays Tk. " + toll);
38         return toll;
39     }

```

```

39 }
40
41 public class Q_Toll {
42     public static void main(String[] args) {
43         TollPlaza plaza = new TollPlaza();
44         Vehicle[] queue = { new Car("CAR-1234"), new Truck("TRUCK-55", 3), new Ambulance("
AMB-09") };
45         for (Vehicle v : queue) plaza.charge(v);
46     }
47 }

```

#### Program output (compiled and run)

```

1 CAR-1234 pays Tk. 750.0
2 TRUCK-55 pays Tk. 2500.0
3 AMB-09 pays Tk. 0.0

```

### Principle: Open/Closed Principle (OCP)

“Software entities should be **open for extension** but **closed for modification**.” Ambulance was added as a new class; TollPlaza and the existing vehicle classes were not edited or retested.

#### Q28. Message chains and the Law of Demeter

Asked in: Set-2 Q3(b) · New

Marks: 3.5

Identify the design problem in the following code and refactor it. Which principle is violated?

```

public class ComplaintService {
    public void escalate(Complaint c) {
        c.getCitizen().getAddress().getZone().getOfficer().assign(c);
    }
}

```

### Smell: Message Chain (“train wreck”)

The service walks through four objects: complaint → citizen → address → zone → officer. This violates the **Law of Demeter** (principle of least knowledge): a method should call methods only on (1) its own object, (2) its parameters, (3) objects it creates, and (4) its direct fields — “talk only to your immediate friends”.

### Why it is a problem

ComplaintService is coupled to the internal structure of four classes. If DUMS starts finding the zone from GPS instead of the address, this line breaks. It is also hard to test (four objects must be mocked) and the same chain is usually repeated in other places.

### Refactoring: Hide Delegate

Each object offers the operation its caller needs and delegates internally. The service now talks only to its parameter.

#### Java: Q\_Demeter.java

```

1 class Officer {
2     private final String title;
3     Officer(String title) { this.title = title; }
4     void assign(Complaint c) { System.out.println(title + " assigned to " + c.getId()); }
5 }

```

```

6
7 class Zone {
8     private final Officer officer;
9     Zone(Officer officer) { this.officer = officer; }
10    void assignToOfficer(Complaint c) { officer.assign(c); }    // hides the officer
11 }
12
13 class Address {
14     private final Zone zone;
15     Address(Zone zone) { this.zone = zone; }
16     void assignToZoneOfficer(Complaint c) { zone.assignToOfficer(c); }
17 }
18
19 class Citizen {
20     private final Address address;
21     Citizen(Address address) { this.address = address; }
22     void routeComplaint(Complaint c) { address.assignToZoneOfficer(c); }
23 }
24
25 class Complaint {
26     private final String id;
27     private final Citizen citizen;
28     Complaint(String id, Citizen citizen) { this.id = id; this.citizen = citizen; }
29     String getId() { return id; }
30     void routeToResponsibleOfficer() { citizen.routeComplaint(this); }
31 }
32
33 class ComplaintService {
34     void escalate(Complaint c) {
35         c.routeToResponsibleOfficer();    // talks only to its direct friend (parameter
36         c)
37     }
38
39 public class Q_Demeter {
40     public static void main(String[] args) {
41         Zone zone5 = new Zone(new Officer("Zone-5 Executive Officer"));
42         Citizen rahim = new Citizen(new Address(zone5));
43         new ComplaintService().escalate(new Complaint("CMP-808", rahim));
44     }
45 }

```

#### Program output (compiled and run)

```
1 Zone-5 Executive Officer assigned to CMP-808
```

#### Balance

Too many pure delegating methods create the opposite smell, *Middle Man*. A good alternative is to inject a dedicated collaborator, e.g. `zoneDirectory.assignResponsibleOfficer(c)`, which owns the lookup logic.

## Section A — Behavioural Modelling with UML

### Q29. Sequence diagram: changing a TV channel with a remote

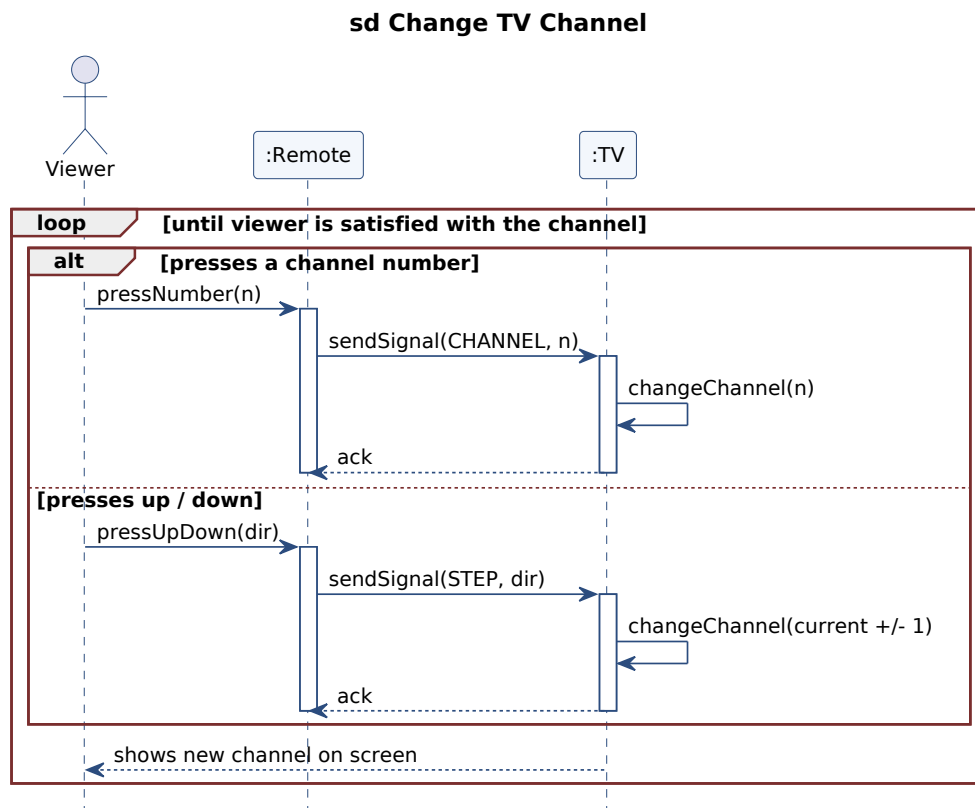
Asked in: 2021 Q2(b) · 2024 Q5(a)

Marks: 3.75 / 5

Consider changing TV channels using a remote. A viewer can press a channel number on the remote to change the channel shown on the TV screen. The viewer can also press the up/down button to change the channel. Draw a UML sequence diagram to model the interaction between the user and the TV through the remote.

#### Elements used

**Lifelines** (actor Viewer, objects :Remote, :TV) run downwards as dashed lines; **activation bars** show when an object is busy; **synchronous messages** are solid arrows with filled heads; **return messages** are dashed; a **self-message** loops back to the same lifeline; **combined fragments** (loop, alt) with **guards** in square brackets model repetition and alternatives. The whole diagram is framed with sd.



Sequence diagram for changing TV channels

#### Reading the diagram

1. The loop fragment repeats until the viewer is satisfied with the channel.
2. Inside it, the alt fragment has two operands. If the viewer presses a number, :Remote receives pressNumber(n) and sends sendSignal(CHANNEL, n) to :TV, which calls changeChannel(n) on itself.
3. Otherwise the viewer presses up/down; the remote sends sendSignal(STEP, dir) and the TV changes to the next or previous channel.

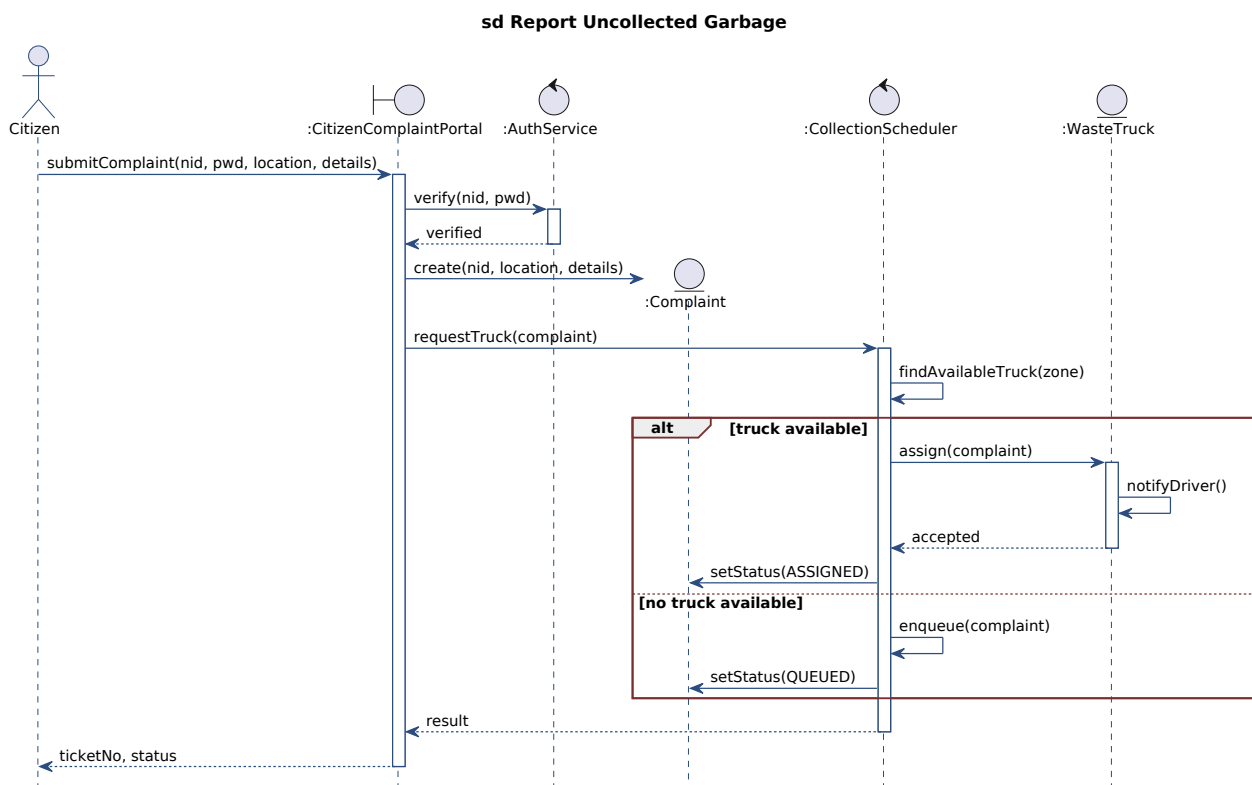
4. After either case, the TV shows the new channel to the viewer.

### Q30. Sequence diagram: citizen reports uncollected garbage

Asked in: Set-1 Q5(a) · New

Marks: 5

A citizen logs into the DUMS complaint portal with NID and password and submits a complaint about uncollected garbage. The portal verifies the citizen through an authentication service, creates a complaint and asks the collection scheduler for a truck. If a truck is available, the scheduler assigns the complaint to the truck (the driver is notified) and marks it ASSIGNED; otherwise it queues the complaint and marks it QUEUED. The citizen receives a ticket number and status. Draw a UML sequence diagram using an alt fragment.



Sequence diagram for reporting uncollected garbage

### Explanation

1. The actor *Citizen* sends `submitComplaint(...)` to the *boundary* object `:CitizenComplaintPortal`.
2. The portal calls the *control* object `:AuthService` to `verify(nid, pwd)`; a dashed return carries "verified".
3. The portal **creates** the *entity* `:Complaint` — its lifeline starts at the creation message.
4. The portal calls `requestTruck(complaint)` on `:CollectionScheduler`, which runs `findAvailableTruck(zone)` on itself.
5. The `alt` fragment: `[truck available]` — assign to `:WasteTruck`, which notifies its driver and replies "accepted"; the complaint becomes ASSIGNED. `[no truck available]` — the scheduler enqueues the complaint and sets it to QUEUED.
6. Results return to the portal, which gives the citizen a ticket number and status.

**Exam tip**

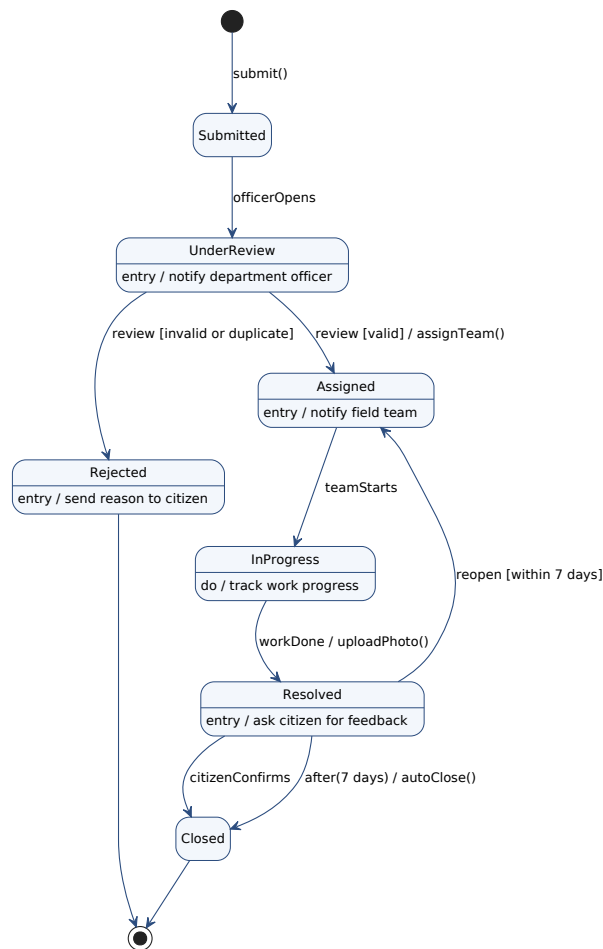
An authentication failure could be shown with another alt (“invalid credentials” → error message). Mentioning it in one sentence earns credit without complicating the drawing.

**Q31. State diagram for a DUMS Complaint**

**Asked in:** Set-1 Q5(b) · New

**Marks:** 3.75

A DUMS complaint is submitted and then reviewed by an officer. An invalid or duplicate complaint is rejected; a valid one is assigned to a field team, goes in progress and becomes resolved. The citizen can confirm it (closed) or reopen it within 7 days; otherwise it is auto-closed after 7 days. Draw a UML state diagram for a Complaint object.



State machine of a Complaint

**Notation and explanation**

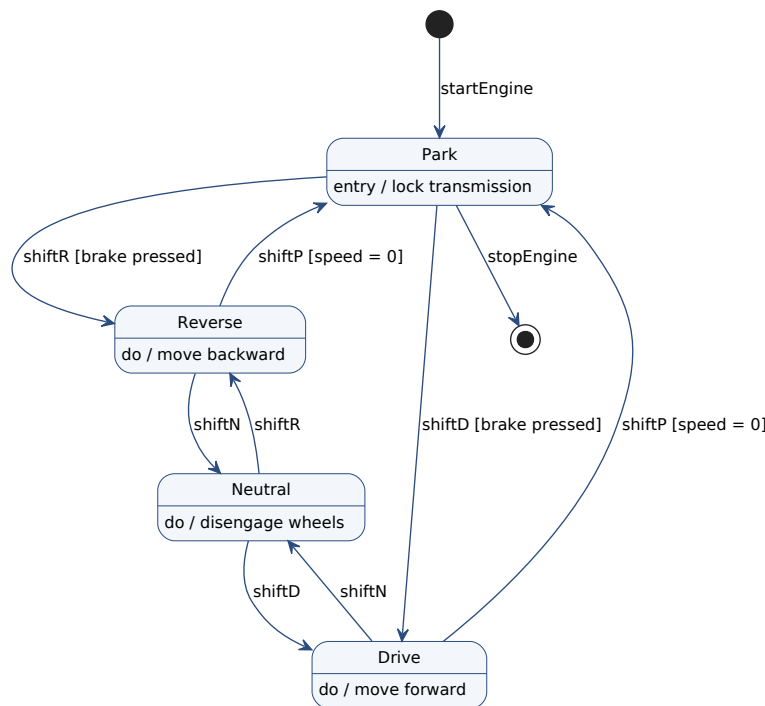
- **Initial pseudo-state** (filled circle) leads to Submitted; **final state** (bull's eye) is reached from Rejected or Closed.
- Each **transition** is labelled event [guard] / action, e.g. review [valid] / assignTeam() — the transition fires on the event only if the guard is true, and then the action runs.
- **Internal activities:** entry / runs on entering a state (e.g. notify the field team when Assigned); do / runs while in the state (track work progress).
- From Resolved, three transitions compete: citizenConfirms → Closed; reopen [within 7 days] → Assigned; the time event after(7 days) / autoClose() → Closed.

**Q32. UML state diagrams and model checking****Asked in:** New (Slides M3)**Marks:** 3–4

Explain the elements of a UML state diagram with an example of a car's transmission. What is model checking and how does it help in design?

**State diagram elements**

A state diagram shows how **one object changes state** in response to events over its lifetime. Elements: **states** (rounded rectangles, a condition of the object), **initial** and **final** pseudo-states, **transitions** (arrows labelled event [guard] / action), and internal activities entry, do and exit.



Car transmission states

Here the car starts in Park. It can shift to Drive or Reverse only if the brake is pressed, and back to Park only when speed is zero; Neutral lies between Drive and Reverse.

**Model checking**

Model checking is a technique for **verifying a design model automatically** against required properties by **systematically exploring every reachable state** of the model. It has three phases:

1. **Modelling phase:** describe the system as a state model (e.g. from the state diagram) and write the properties to be checked, such as “the car can never go from Drive to Reverse directly” (safety) or “every complaint is eventually closed or rejected” (liveness).
2. **Running phase:** the model checker explores all states and transitions.
3. **Analysis phase:** if a property fails, the tool gives a **counterexample** — the exact sequence of events that leads to the error — so the design can be corrected.

**Benefits:** finds deadlocks, unreachable states and unsafe transitions *before* coding, when fixes are cheap; is exhaustive, unlike testing which checks only some paths.

## Section B — Design Pattern Foundations

### Q33. What are design patterns and why learn them?

**Asked in:** 2021 Q4(a) · 2023 Q4(a), Q5(a) · 2024 Q4(a) · Set-1 Q4(a)

**Marks:** 2.25 / 2.75 / 2.5

What are software design patterns? How do they help solve common design issues in object-oriented software, and why is learning and applying them beneficial for developers? Explain two principles of reusable object-oriented design proposed by the Gang of Four.

### Definition

A **software design pattern** is a **general, reusable solution to a commonly occurring problem in software design**, described in a form that can be applied in many different situations. It is not finished code or a library; it is a template that describes *which classes and objects take part, their roles, and how they collaborate*. The idea came from the architect Christopher Alexander; in 1994 Gamma, Helm, Johnson and Vlissides — the **Gang of Four (GoF)** — catalogued 23 object-oriented patterns.

Every pattern has four essential elements: a **name** (vocabulary), the **problem** (when to apply it), the **solution** (classes, responsibilities, collaborations) and the **consequences** (benefits and trade-offs).

### How patterns solve common design issues

| Recurring design issue                                          | Pattern that solves it    |
|-----------------------------------------------------------------|---------------------------|
| Only one instance of a class must exist (logger, configuration) | Singleton                 |
| Client should not depend on concrete classes it creates         | Factory, Abstract Factory |
| An object has many optional parts                               | Builder                   |
| An existing class has an incompatible interface                 | Adapter                   |
| A subsystem is too complex for clients                          | Facade                    |
| Many objects must react when one object changes                 | Observer                  |
| A request must pass through several possible handlers           | Chain of Responsibility   |
| Requests must be queued, logged or undone                       | Command                   |

### Why learn and apply them

1. **Proven solutions:** they capture the experience of expert designers, so we do not reinvent (and re-debug) solutions.
2. **Common vocabulary:** saying “use an Observer here” communicates a whole design in two words.
3. **Better design quality:** patterns promote loose coupling, high cohesion, reusability and extensibility.
4. **Faster development and easier maintenance:** well-known structures are easy to recognise and change.
5. **Understanding frameworks:** Java’s `InputStream` wrappers are Decorators, Swing listeners are Observers, `Runtime.getRuntime()` is a Singleton.
6. **Learning OO principles by example:** each pattern demonstrates encapsulation, polymorphism and composition in practice.

### Two GoF principles of reusable OO design

1. **Program to an interface, not an implementation.** Declare variables and parameters with abstract types so that any implementation can be substituted.

```

1 Notification n = NotificationFactory.create("sms"); // depends on the interface
2 n.send("01712345678", "Complaint received");      // SmsNotification can be swapped

```

**2. Favour object composition over class inheritance.** Build behaviour by *holding* other objects rather than by extending classes. Composition can be changed at run time, keeps classes small and avoids fragile deep hierarchies. Example: `SmartPhone` *has* an `ICamera` (Q23) instead of *extending* a camera class.

#### Common mistake

Patterns are not a goal in themselves. Using a pattern where the problem does not exist adds unnecessary classes and complexity.

### Q34. Categories: creational, structural and behavioural patterns

**Asked in:** 2023 Q4(b), Q4(c), Q5(b) · Set-1 Q4(b)

**Marks:** 2.5–3.75

Classify the GoF design patterns into three categories with examples. What are creational design patterns and why are they important (give an example of each)? How do structural design patterns contribute to the organization of software? Explain with an example.

### The three categories

| Category           | Concern                                                                                                       | GoF patterns (bold = in your notes)                                                                                                                                                                                    |
|--------------------|---------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Creational</b>  | <i>How objects are created</i> — hide the creation logic so the system is independent of how objects are made | <b>Abstract Factory</b> , <b>Builder</b> , <b>Factory Method</b> , <b>Prototype</b> , <b>Singleton</b>                                                                                                                 |
| <b>Structural</b>  | <i>How classes and objects are composed</i> into larger structures while keeping them flexible                | <b>Adapter</b> , <b>Bridge</b> , <b>Composite</b> , <b>Decorator</b> , <b>Facade</b> , <b>Flyweight</b> , <b>Proxy</b> (+ <b>Filter</b> in the notes)                                                                  |
| <b>Behavioural</b> | <i>How objects communicate</i> and share responsibilities                                                     | <b>Chain of Responsibility</b> , <b>Command</b> , <b>Interpreter</b> , <b>Iterator</b> , <b>Mediator</b> , <b>Memento</b> , <b>Observer</b> , <b>State</b> , <b>Strategy</b> , <b>Template Method</b> , <b>Visitor</b> |

### Creational patterns: definition and importance

Creational patterns **abstract the instantiation process**. Instead of scattering new `ConcreteClass()` throughout the code, they put the decision of *which* object to create, *how many*, and *how to assemble it* in one place. Their primary purpose is to make a system independent of how its objects are created, composed and represented.

- **Reduce coupling:** clients depend on interfaces, not on concrete classes.
- **Control instances:** guarantee a single shared object where needed.
- **Manage complex construction:** build objects with many optional parts safely.
- **Ease change:** a new product type is added in one place.

| Pattern                 | Intent                                          | Example                                                           |
|-------------------------|-------------------------------------------------|-------------------------------------------------------------------|
| <b>Singleton</b>        | exactly one instance with a global access point | one <code>Logger</code> / <code>ConfigurationManager</code> (Q35) |
| <b>Factory Method</b>   | a method decides which subclass to instantiate  | <code>NotificationFactory.create("sms")</code> (Q38)              |
| <b>Abstract Factory</b> | create families of related objects              | AgroTech sensor + valve together (Q39)                            |
| <b>Builder</b>          | construct a complex object step by step         | <code>HousingApplication.Builder</code> (Q40)                     |
| <b>Prototype</b>        | create objects by cloning a prototype           | clone a pre-filled complaint template                             |

## How structural patterns organise software

Structural patterns describe how to **combine classes and objects into larger structures** while keeping them flexible and loosely coupled. They:

- **make incompatible parts work together** (Adapter);
- **simplify access to complex subsystems** (Facade);
- **build part-whole hierarchies** that clients treat uniformly (Composite);
- **separate abstraction from implementation** so both vary independently (Bridge);
- **add responsibilities without subclassing** (Decorator).

**Example — Facade in DUMS:** registering a resident needs four subsystems (housing, billing, garbage, SMS). A `ResidentServiceFacade` offers one method `registerNewResident()`, so the portal is coupled to one class instead of four and the subsystems stay independent (full code in Q44).

## Section B — Creational Patterns

### Q35. Convert ConfigurationManager into a Singleton

Asked in: 2024 Q4(b)

Marks: 4

Consider the following ConfigurationManager class. Convert it into a Singleton pattern implementation. Explain why the Singleton pattern is appropriate for a configuration manager, and demonstrate how to use the singleton class with sample client code.

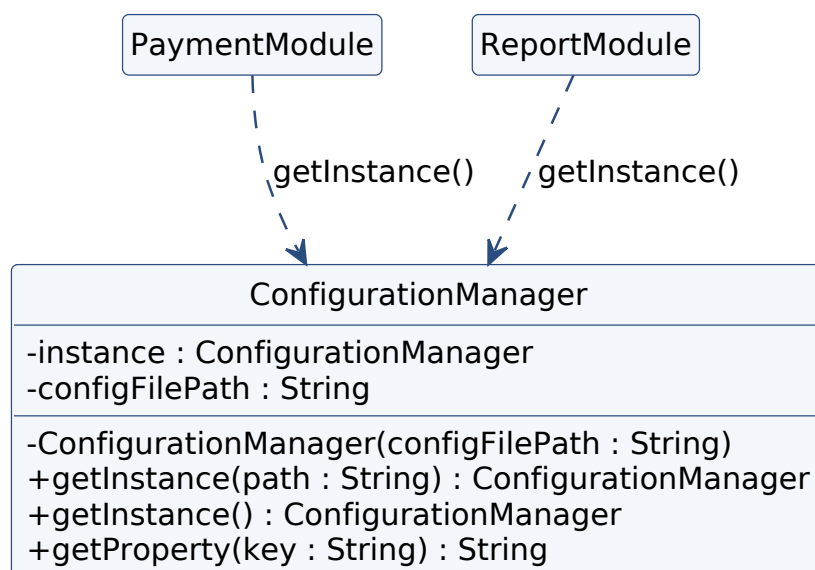
```
class ConfigurationManager {
    private String configFilePath;
    public ConfigurationManager(String configFilePath) {
        this.configFilePath = configFilePath;
    }
    public String getProperty(String key) {
        System.out.println("Retrieved property: " + key + " from " + configFilePath);
        return "value";
    }
}
```

### Singleton pattern

**Intent:** ensure a class has **only one instance** and provide a **global point of access** to it. Category: creational.

### Steps of the conversion

1. Make the constructor private, so no other class can call new.
2. Add a private static ConfigurationManager instance field to hold the single object.
3. Add a public static getInstance(...) method that creates the object on the first call (lazy initialisation) and returns the same object afterwards.
4. Make getInstance synchronized so two threads cannot create two objects.



Singleton ConfigurationManager and its clients

## Java: Q\_ConfigManager.java

```

1 class ConfigurationManager {
2     private static ConfigurationManager instance;           // the one shared object
3     private final String configFilePath;
4
5     private ConfigurationManager(String configFilePath) { // private: 'new' impossible
6         this.configFilePath = configFilePath;
7     }
8
9     public static synchronized ConfigurationManager getInstance(String configFilePath) {
10         if (instance == null) {                             // create only on first call
11             instance = new ConfigurationManager(configFilePath);
12         }
13         return instance;                                     // later calls reuse it
14     }
15
16     public static ConfigurationManager getInstance() {        // convenience for other
17         return getInstance("app.properties");
18     }
19
20     public String getProperty(String key) {
21         System.out.println("Retrieved property: " + key + " from " + configFilePath);
22         return "value";
23     }
24 }
25
26 public class Q_ConfigManager {
27     public static void main(String[] args) {
28         ConfigurationManager c1 = ConfigurationManager.getInstance("config/app.properties"
29     );
30         c1.getProperty("db.url");
31
32         ConfigurationManager c2 = ConfigurationManager.getInstance(); // e.g. another
33         module
34         c2.getProperty("sms.apiKey");
35
36         System.out.println("Same instance? " + (c1 == c2));
37     }
38 }

```

## Program output (compiled and run)

```

1 Retrieved property: db.url from config/app.properties
2 Retrieved property: sms.apiKey from config/app.properties
3 Same instance? true

```

## Why Singleton suits a configuration manager

- **One consistent source of settings:** every module reads the same values; two objects could load different files and disagree.
- **Load once:** reading and parsing a configuration file is costly; it is done only once.
- **Global access:** any module obtains it with `getInstance()` without passing it through many constructors.
- **Controlled memory use:** no duplicate copies of the same data.

**Point worth writing**

With a parameterised `getInstance(path)`, only the *first* call's path is used; later paths are ignored (see the output: the second call still reads `config/app.properties`).

**Q36. Database Singleton; shared logging service****Asked in:** 2021 Q4(b) · 2023 Q5(c)**Marks:** 3.5 / 3–4

(2021) Modify the following Database class to convert it to a singleton class, and show how to call its `executeSql` method from a demo program.

```
class Database {
    private String dbName, user, password;
    public Database(String dbName, String user, String password) {
        this.dbName = dbName;
        this.user = user;
        this.password = password;
    }
    public void executeSql(String sql) {
        System.out.println("Executed query: " + sql);
    }
}
```

(2023) A software system requires a single shared instance of a logging service to record errors across multiple modules. Which creational design pattern should be used? Explain why and outline its structure.

**2021: Database as a Singleton****Java: Q\_Database.java**

```
1 class Database {
2     private static Database instance;
3     private final String dbName, user, password;
4
5     private Database(String dbName, String user, String password) {
6         this.dbName = dbName;
7         this.user = user;
8         this.password = password;
9     }
10
11     public static synchronized Database getInstance(String dbName, String user, String
12     password) {
13         if (instance == null) {
14             instance = new Database(dbName, user, password);
15         }
16         return instance;
17     }
18
19     public void executeSql(String sql) {
20         System.out.println("Executed query on " + dbName + " as " + user + ": " + sql);
21     }
22 }
23
24 public class Q_Database {
25     public static void main(String[] args) {
26         Database db1 = Database.getInstance("dums_db", "admin", "secret");
27         db1.executeSql("SELECT * FROM complaints");
28     }
29 }
```

```

28     Database db2 = Database.getInstance("other_db", "guest", "1234"); // ignored:
    already created
29     db2.executeSql("SELECT COUNT(*) FROM bins");
30
31     System.out.println("db1 == db2 : " + (db1 == db2));
32 }
33 }

```

#### Program output (compiled and run)

```

1 Executed query on dums_db as admin: SELECT * FROM complaints
2 Executed query on dums_db as admin: SELECT COUNT(*) FROM bins
3 db1 == db2 : true

```

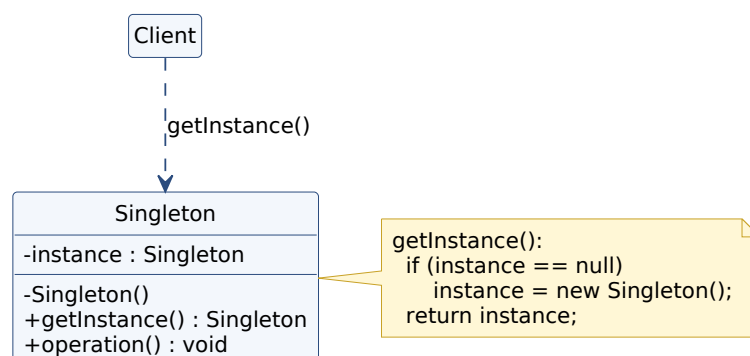
Both references point to the same object, so the second set of arguments is ignored. The constructor is private, the single object is kept in private static instance, and access is only through the synchronized `getInstance(...)`.

#### Error in the handwritten solve

The handwritten solution calls `new Database()` with no arguments, but the class only has a three-argument constructor — it does not compile. Pass the three arguments as above.

## 2023: Logging service

**Pattern: Singleton. Why:** (1) all modules must write to the same log file/stream in order; (2) several logger objects would open the file several times and may overwrite each other's lines; (3) a logger is needed everywhere, so a global access point is convenient; (4) it saves resources. **Structure:** a private constructor, a private static field holding the only instance, and a public static `getInstance()`; clients never use `new`.



General structure of Singleton

#### Java: Q\_Logger.java

```

1 class Logger {
2     private static final Logger INSTANCE = new Logger(); // eager: created when class
    loads
3     private int count;
4
5     private Logger() { }
6     public static Logger getInstance() { return INSTANCE; }
7
8     public synchronized void error(String module, String msg) {
9         count++;
10        System.out.println("[ERROR #" + count + "][" + module + "] " + msg);

```

```

11     }
12 }
13
14 class PaymentModule {
15     void pay() { Logger.getInstance().error("Payment", "gateway timeout"); }
16 }
17
18 class InventoryModule {
19     void update() { Logger.getInstance().error("Inventory", "stock became negative"); }
20 }
21
22 public class Q_Logger {
23     public static void main(String[] args) {
24         new PaymentModule().pay();
25         new InventoryModule().update();
26         System.out.println("Same logger object: " + (Logger.getInstance() == Logger.
            getInstance()));
27     }
28 }

```

#### Program output (compiled and run)

```

1 [ERROR #1][Payment] gateway timeout
2 [ERROR #2][Inventory] stock became negative
3 Same logger object: true

```

Here the instance is created **eagerly** when the class is loaded (static final), which is automatically thread-safe; error() is synchronized so log lines from different threads do not mix.

### Q37. Thread-safe SmsGateway Singleton

Asked in: Set-1 Q4(c) · New

Marks: 3.75

Several DUMS modules (housing, garbage, traffic) create their own object of the following class, and every object opens a new costly connection. Convert it into a thread-safe, lazily initialised Singleton and show how the modules use it.

```

class SmsGateway {
    private String connectionId;
    public SmsGateway() {
        connectionId = openConnection("sms.dums.gov.bd", 8443); // slow, costly
    }
    public void send(String phone, String text) {
        System.out.println "[" + connectionId + "] to " + phone + ": " + text);
    }
}

```

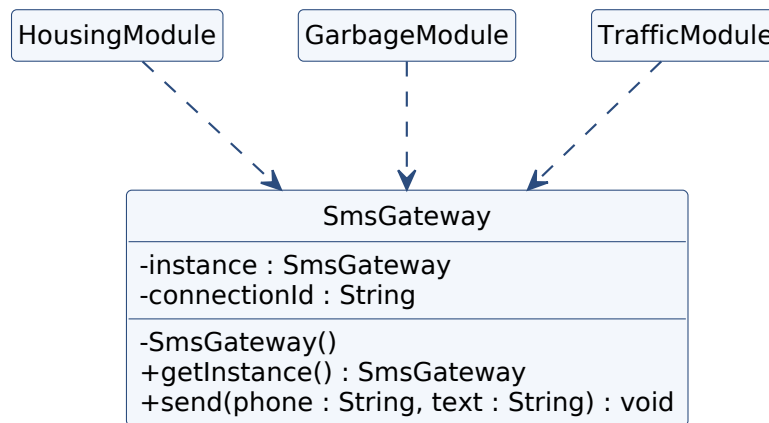
### Problem with a naive lazy Singleton

```

1 public static SmsGateway getInstance() {
2     if (instance == null) // thread A and thread B both see null ...
3         instance = new SmsGateway(); // ... and both create an object: 2 connections
4     return instance;
5 }

```

This **race condition** breaks the single-instance guarantee in a multi-threaded server.

**Solution: double-checked locking with volatile**

One SmsGateway shared by all modules

**Java: Q\_SmsGateway.java**

```

1 class SmsGateway {
2     private static volatile SmsGateway instance;    // volatile: every thread sees the
3     private static int connectionsOpened = 0;      final object
4     private final String connectionId;
5
6     private SmsGateway() {                          // costly work happens only once
7         connectionId = "CONN-" + (++connectionsOpened);
8         System.out.println("Opening SMS connection " + connectionId);
9     }
10
11     public static SmsGateway getInstance() {
12         if (instance == null) {                      // 1st check: no locking on the fast
13             synchronized (SmsGateway.class) {        path
14                 if (instance == null) {                // 2nd check: inside the lock
15                     instance = new SmsGateway();
16                 }
17             }
18         }
19         return instance;
20     }
21
22     public void send(String phone, String text) {
23         System.out.println "[" + connectionId + "] to " + phone + ": " + text);
24     }
25 }
26
27 public class Q_SmsGateway {
28     public static void main(String[] args) throws InterruptedException {
29         Thread housing = new Thread(() -> SmsGateway.getInstance().send("01711000001", "
30             Housing permit approved"));
31         Thread garbage = new Thread(() -> SmsGateway.getInstance().send("01811000002", "
32             Bin collected in Ward 21"));
33         Thread traffic = new Thread(() -> SmsGateway.getInstance().send("01911000003", "
34             Road closed at Mirpur-10"));
35         housing.start(); garbage.start(); traffic.start();
36         housing.join(); garbage.join(); traffic.join();
37     }
38 }
  
```

```

34     }
35 }

```

#### Program output (compiled and run)

```

1 Opening SMS connection CONN-1
2 [CONN-1] to 01911000003: Road closed at Mirpur-10
3 [CONN-1] to 01811000002: Bin collected in Ward 21
4 [CONN-1] to 01711000001: Housing permit approved

```

- **First check** (without lock) makes later calls fast.
- **synchronized block** lets only one thread create the object.
- **Second check** inside the lock stops a thread that was waiting from creating a second object.
- **volatile** guarantees that other threads never see a half-constructed object.

The output shows “Opening SMS connection” **only once** although three threads call `getInstance()` (message order may vary between runs).

### Other thread-safe variants

| Variant                         | Note                                                                                               |
|---------------------------------|----------------------------------------------------------------------------------------------------|
| <b>Eager initialisation</b>     | <code>private static final X INSTANCE = new X();</code> — simplest, but created even if never used |
| <b>Synchronized method</b>      | <code>public static synchronized X getInstance()</code> — simple but locks on every call           |
| <b>Double-checked locking</b>   | lazy and fast (above)                                                                              |
| <b>Holder class (Bill Pugh)</b> | a static nested class holds the instance; the JVM loads it lazily and safely                       |
| <b>Enum Singleton</b>           | <code>enum X { INSTANCE; }</code> — also safe against serialization and reflection                 |

### Drawbacks of Singleton

It introduces global state, hides dependencies, makes unit testing harder (hard to replace with a mock) and can become a bottleneck; use it only when exactly one instance is truly required.

#### Q38. Factory pattern for DUMS notifications

**Asked in:** Set-2 Q4(a) · New (Notes)

**Marks:** 4.5

DUMS notifies citizens by SMS, e-mail and push notification. Currently every module creates the concrete notification objects with `new`. Apply the Factory pattern with a UML diagram and Java code, and show how a WhatsApp channel can be added later.

### Problem with `new` everywhere

```

1 if (channel.equals("sms")) n = new SmsNotification();           // repeated in complaint,
2 else if (channel.equals("email")) n = new EmailNotification(); // billing, housing ...

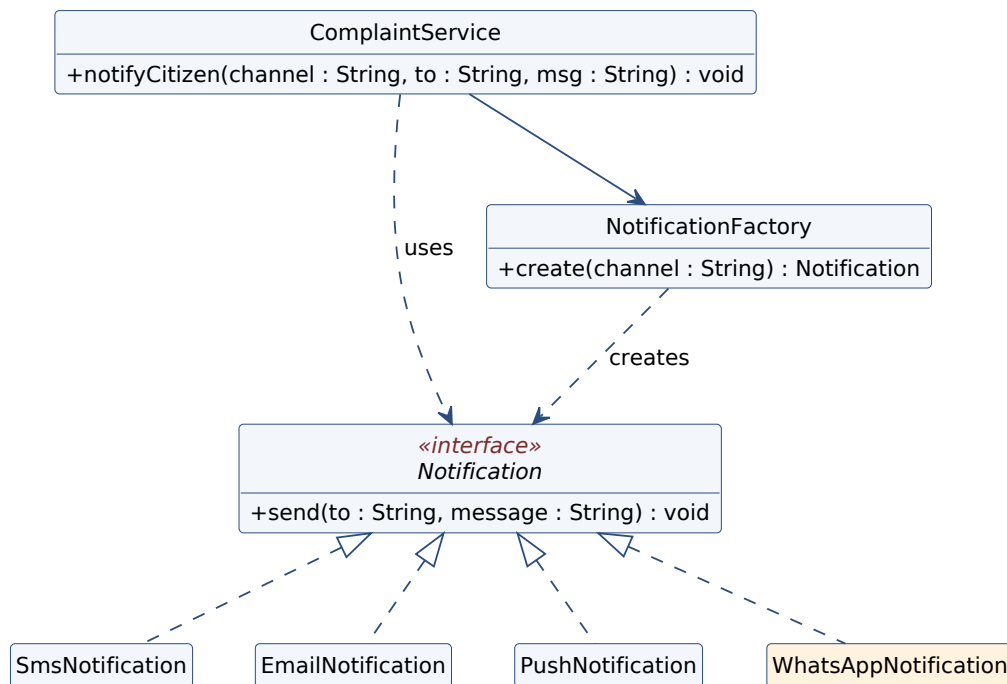
```

Every module depends on every concrete class, the selection logic is duplicated, and adding WhatsApp means editing all modules.

### Factory pattern

**Intent:** create objects **without exposing the creation logic** to the client, and refer to the newly created

object through a **common interface**. Roles: **Product** interface (Notification), **Concrete products** (SmsNotification, ...), **Factory** (NotificationFactory), **Client** (ComplaintService).



Factory for notifications (new channel highlighted)

#### Java: Q\_NotificationFactory.java

```

1 interface Notification {
2     void send(String to, String message);
3 }
4
5 class SmsNotification implements Notification {
6     public void send(String to, String message) { System.out.println("SMS    -> " + to +
7         ": " + message); }
8 }
9
10 class EmailNotification implements Notification {
11     public void send(String to, String message) { System.out.println("EMAIL    -> " + to +
12         ": " + message); }
13 }
14
15 class PushNotification implements Notification {
16     public void send(String to, String message) { System.out.println("PUSH    -> " + to +
17         ": " + message); }
18 }
19
20 class WhatsAppNotification implements Notification {           // new channel
21     public void send(String to, String message) { System.out.println("WHATSAPP -> " + to +
22         ": " + message); }
23 }
24
25 class NotificationFactory {                                   // the ONLY place that uses '
26     new'
27     public static Notification create(String channel) {
28         if (channel == null) throw new IllegalArgumentException("channel is null");
29         switch (channel.toUpperCase()) {

```

```

25         case "SMS":         return new SmsNotification();
26         case "EMAIL":       return new EmailNotification();
27         case "PUSH":        return new PushNotification();
28         case "WHATSAPP":     return new WhatsAppNotification(); // the only line added
29         default: throw new IllegalArgumentException("Unknown channel: " + channel);
30     }
31 }
32 }
33
34 class ComplaintService {                                     // client knows only the
35     interface
36     public void notifyCitizen(String channel, String to, String msg) {
37         Notification n = NotificationFactory.create(channel);
38         n.send(to, msg);
39     }
40 }
41 public class Q_NotificationFactory {
42     public static void main(String[] args) {
43         ComplaintService service = new ComplaintService();
44         service.notifyCitizen("sms", "01712345678", "Complaint CMP-12 received");
45         service.notifyCitizen("email", "rahim@mail.com", "Complaint CMP-12 assigned");
46         service.notifyCitizen("whatsapp", "01712345678", "Complaint CMP-12 resolved");
47     }
48 }

```

#### Program output (compiled and run)

```

1 SMS      -> 01712345678: Complaint CMP-12 received
2 EMAIL    -> rahim@mail.com: Complaint CMP-12 assigned
3 WHATSAPP -> 01712345678: Complaint CMP-12 resolved

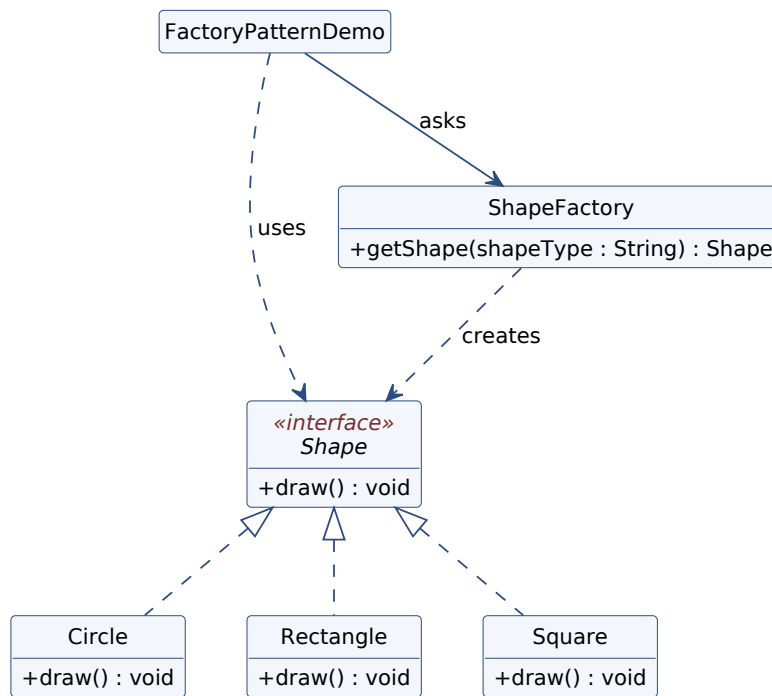
```

## Adding WhatsApp

(1) Write `WhatsAppNotification` implements `Notification`; (2) add one case in `NotificationFactory`. `ComplaintService` and all other clients are unchanged because they only know `Notification` and the factory.

## Benefits

Loose coupling (clients depend on the interface), creation logic in one place, easy extension, and consistent object configuration. The notes' `ShapeFactory.getShape("CIRCLE")` is the same idea.



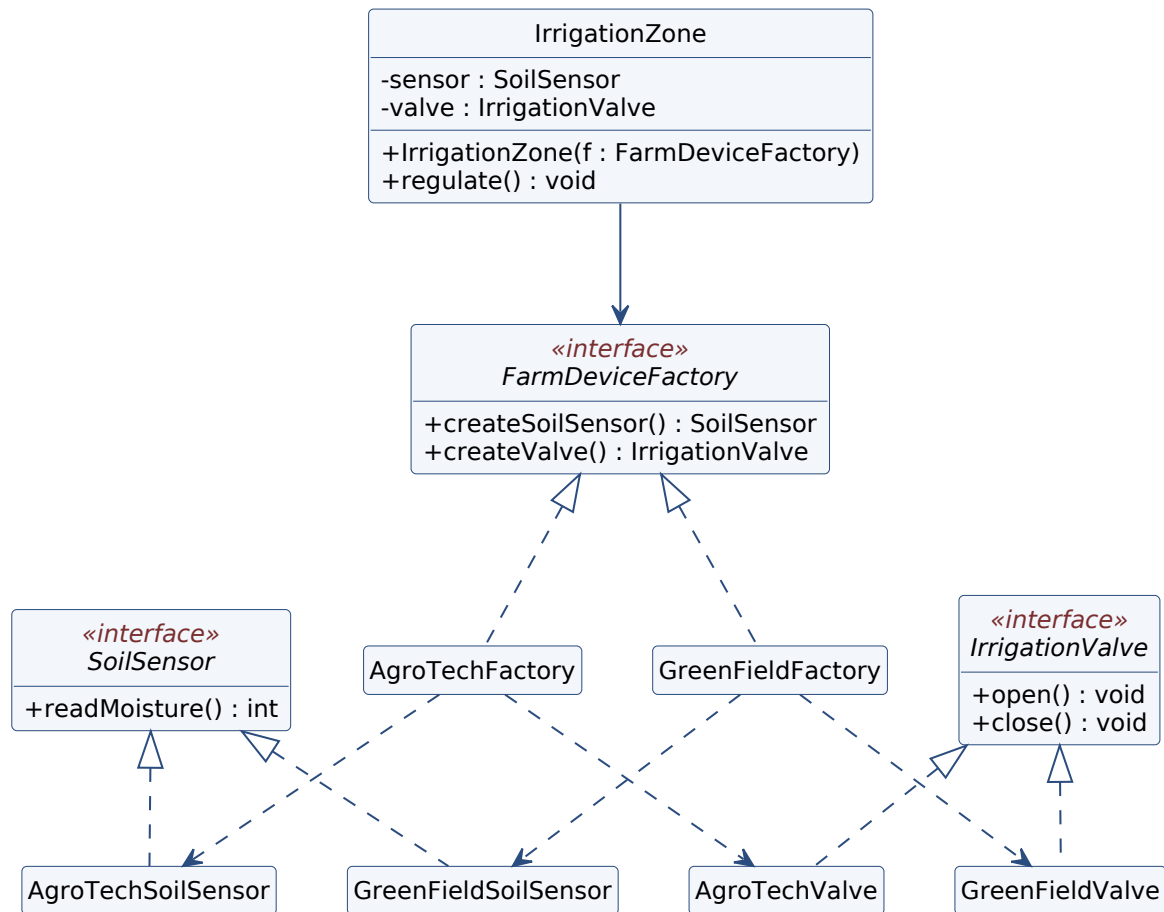
ShapeFactory example from the notes

**Q39. Abstract Factory for AAS device families****Asked in:** New (Notes)**Marks:** 4.5

AAS buys soil sensors and irrigation valves from two vendors, AgroTech (Modbus protocol) and GreenField (LoRa protocol). A sensor from one vendor must never be combined with a valve from the other. Apply the Abstract Factory pattern and compare it with the Factory pattern.

**Abstract Factory**

**Intent:** provide an interface for creating **families of related or dependent objects** without specifying their concrete classes. It is a “factory of factories”. Roles: **AbstractFactory** (FarmDeviceFactory), **ConcreteFactory** (AgroTechFactory, GreenFieldFactory), **AbstractProduct** (SoilSensor, IrrigationValve), **ConcreteProduct**, **Client** (IrrigationZone).



Abstract Factory for vendor device families

## Java: Q\_AbstractFactoryAAS.java

```

1 interface SoilSensor { int readMoisture(); }
2 interface IrrigationValve { void open(); void close(); }
3
4 class AgroTechSoilSensor implements SoilSensor {
5     public int readMoisture() { return 28; }
6 }
7 class AgroTechValve implements IrrigationValve {
8     public void open() { System.out.println("AgroTech valve OPEN (Modbus protocol)"); }
9     public void close() { System.out.println("AgroTech valve CLOSED (Modbus protocol)"); }
10 }
11 class GreenFieldSoilSensor implements SoilSensor {
12     public int readMoisture() { return 55; }
13 }
14 class GreenFieldValve implements IrrigationValve {
15     public void open() { System.out.println("GreenField valve OPEN (LoRa protocol)"); }
16     public void close() { System.out.println("GreenField valve CLOSED (LoRa protocol)"); }
17 }
18
19 interface FarmDeviceFactory { // abstract factory: one family per
20     // vendor
21     SoilSensor createSoilSensor();
22     IrrigationValve createValve();
23 }
24 class AgroTechFactory implements FarmDeviceFactory {
25     public SoilSensor createSoilSensor() { return new AgroTechSoilSensor(); }

```

```

25     public IrrigationValve createValve() { return new AgroTechValve(); }
26 }
27 class GreenFieldFactory implements FarmDeviceFactory {
28     public SoilSensor createSoilSensor() { return new GreenFieldSoilSensor(); }
29     public IrrigationValve createValve() { return new GreenFieldValve(); }
30 }
31
32 class IrrigationZone {                                // client: sees only interfaces
33     private final SoilSensor sensor;
34     private final IrrigationValve valve;
35     IrrigationZone(FarmDeviceFactory factory) {
36         sensor = factory.createSoilSensor();           // both devices guaranteed compatible
37         valve = factory.createValve();
38     }
39     void regulate() {
40         int m = sensor.readMoisture();
41         System.out.print("moisture " + m + " percent -> ");
42         if (m < 40) valve.open(); else valve.close();
43     }
44 }
45
46 public class Q_AbstractFactoryAAS {
47     public static void main(String[] args) {
48         new IrrigationZone(new AgroTechFactory()).regulate();
49         new IrrigationZone(new GreenFieldFactory()).regulate();
50     }
51 }

```

#### Program output (compiled and run)

```

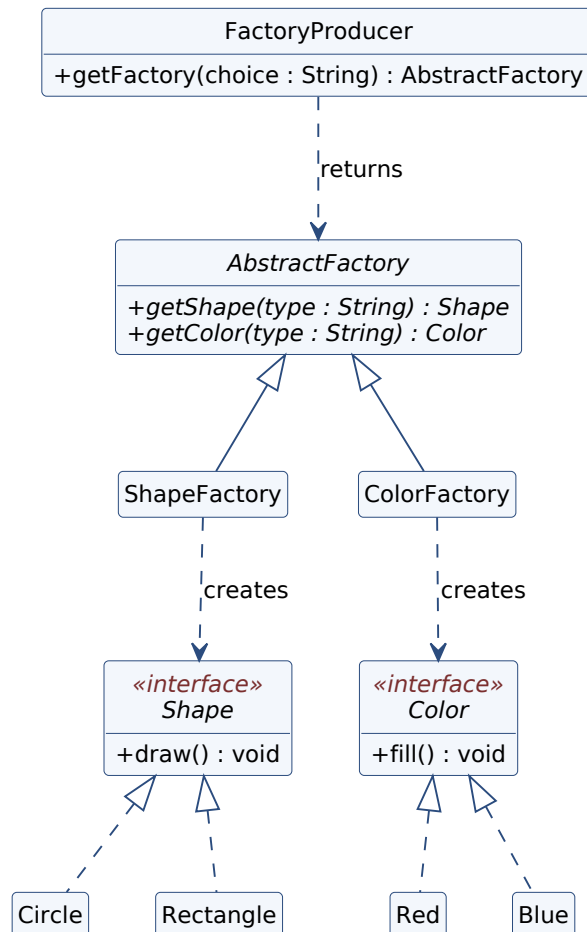
1 moisture 28 percent -> AgroTech valve OPEN (Modbus protocol)
2 moisture 55 percent -> GreenField valve CLOSED (LoRa protocol)

```

Because `IrrigationZone` obtains both devices from one factory, the family is always consistent; switching vendor means passing a different factory.

### Factory vs Abstract Factory

|                      | Factory                                                | Abstract Factory                                                                        |
|----------------------|--------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>Creates</b>       | one product type (one of its subclasses)               | a family of several related product types                                               |
| <b>Structure</b>     | one factory class/method                               | an interface of factories with one concrete factory per family                          |
| <b>Change</b>        | choose a different product                             | switch a whole family by switching the factory                                          |
| <b>Notes example</b> | <code>ShapeFactory</code> returns a <code>Shape</code> | <code>FactoryProducer</code> returns a <code>Shape</code> or <code>Color</code> factory |



The notes' Shape/Color abstract factory

**Q40. Builder for a DUMS HousingApplication****Asked in:** Set-2 Q4(b) · New (Notes)**Marks:** 4.25

A HousingApplication in DUMS has two mandatory fields (applicant name, NID) and five optional fields (family size, monthly income, preferred area, freedom-fighter quota, disability quota), which has produced many overloaded constructors. Apply the Builder pattern with Java code.

**Problem: telescoping constructors**

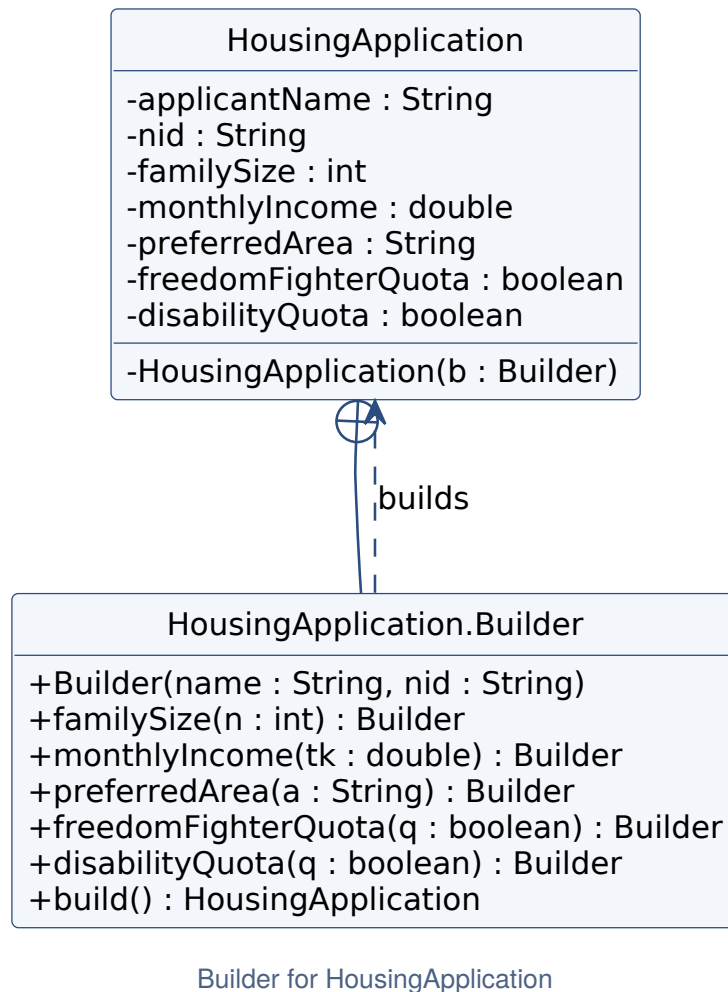
```

1 new HousingApplication("Rahima", "1990123456");
2 new HousingApplication("Rahima", "1990123456", 5);
3 new HousingApplication("Rahima", "1990123456", 5, 25000, "Mirpur", false, true);
4 // which boolean is which quota? easy to swap by mistake
  
```

Many constructors are hard to read and maintain; using setters instead leaves the object half-built and mutable.

**Builder pattern**

**Intent:** separate the **construction of a complex object** from its representation, building it **step by step** so the same process can create different representations. Here a static nested Builder takes the mandatory fields in its constructor, offers one named, chainable method per optional field, validates everything in build(), and calls the private constructor of HousingApplication.



## Java: Q\_HousingBuilder.java

```

1  class HousingApplication {
2      private final String applicantName;    // mandatory
3      private final String nid;              // mandatory
4      private final int familySize;          // optional fields below
5      private final double monthlyIncome;
6      private final String preferredArea;
7      private final boolean freedomFighterQuota;
8      private final boolean disabilityQuota;
9
10     private HousingApplication(Builder b) { // only the Builder can call this
11         this.applicantName = b.applicantName;
12         this.nid = b.nid;
13         this.familySize = b.familySize;
14         this.monthlyIncome = b.monthlyIncome;
15         this.preferredArea = b.preferredArea;
16         this.freedomFighterQuota = b.freedomFighterQuota;
17         this.disabilityQuota = b.disabilityQuota;
18     }
19
20     @Override public String toString() {
21         return applicantName + " (NID " + nid + "): family=" + familySize + ", income=Tk."
22             + monthlyIncome
23             + ", area=" + preferredArea + ", FF quota=" + freedomFighterQuota
24             + ", disability quota=" + disabilityQuota;
25     }
26 }
  
```

```

24     }
25
26     public static class Builder {
27         private final String applicantName;
28         private final String nid;
29         private int familySize = 1;           // sensible defaults
30         private double monthlyIncome = 0;
31         private String preferredArea = "Any";
32         private boolean freedomFighterQuota = false;
33         private boolean disabilityQuota = false;
34
35         public Builder(String applicantName, String nid) { // mandatory data first
36             this.applicantName = applicantName;
37             this.nid = nid;
38         }
39         public Builder familySize(int n)           { familySize = n;           return
this; }
40         public Builder monthlyIncome(double tk)    { monthlyIncome = tk;       return
this; }
41         public Builder preferredArea(String area)  { preferredArea = area;      return
this; }
42         public Builder freedomFighterQuota(boolean q) { freedomFighterQuota = q; return
this; }
43         public Builder disabilityQuota(boolean q)  { disabilityQuota = q;       return
this; }
44
45         public HousingApplication build() {           // validate once, then create
46             if (nid == null || nid.isBlank())
47                 throw new IllegalStateException("NID is required");
48             if (familySize < 1)
49                 throw new IllegalStateException("family size must be at least 1");
50             return new HousingApplication(this);
51         }
52     }
53 }
54
55 public class Q_HousingBuilder {
56     public static void main(String[] args) {
57         HousingApplication a1 = new HousingApplication.Builder("Rahima Begum", "1990123456
")
58             .familySize(5)
59             .monthlyIncome(25000)
60             .preferredArea("Mirpur")
61             .build();
62
63         HousingApplication a2 = new HousingApplication.Builder("Abdul Karim", "5512349876"
)
64             .freedomFighterQuota(true)
65             .build();
66
67         System.out.println(a1);
68         System.out.println(a2);
69     }
70 }

```

**Program output (compiled and run)**

```

1 Rahima Begum (NID 1990123456): family=5, income=Tk.25000.0, area=Mirpur, FF quota=false, disability
  quota=false
2 Abdul Karim (NID 5512349876): family=1, income=Tk.0.0, area=Any, FF quota=true, disability quota=
  false

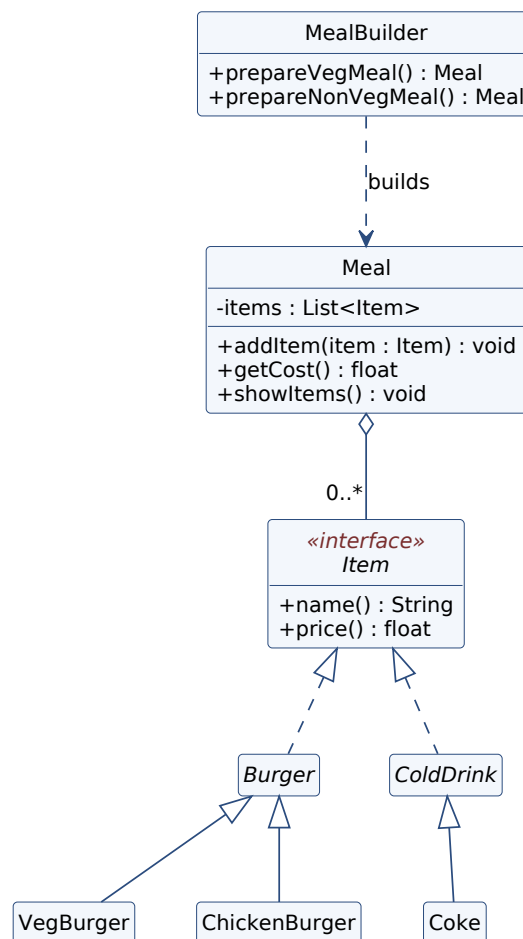
```

**Benefits**

Readable code (each value is named), optional fields have defaults, validation happens once before creation, the product can be immutable (all `final`), and adding a new optional field does not multiply constructors.

**The notes' version**

The notes use a `MealBuilder` that assembles a `Meal` from `Items` (`VegBurger`, `Coke`): `prepareVegMeal()` and `prepareNonVegMeal()` build different representations with the same steps. Either example is acceptable.



MealBuilder from the notes

## Section B — Structural Patterns

### Q41. Adapter pattern: problem, real-world example, UML, media player

**Asked in:** 2021 Q4(c) · 2023 Q6(a) · 2024 Q4(c), Q6(b)

**Marks:** 3 / 4.5 / 2 / 4.25

What problem does the Adapter design pattern solve? Explain it with a real-world example and draw its class diagram. (2024 Q6(b)) You are building a music player that can play only MP3 files. Later you want it to support MP4 and VLC, but the library for those formats works differently and does not match your player's methods. You cannot modify the library and do not want to rewrite your player. Which pattern helps? Explain with example code.

#### Problem solved

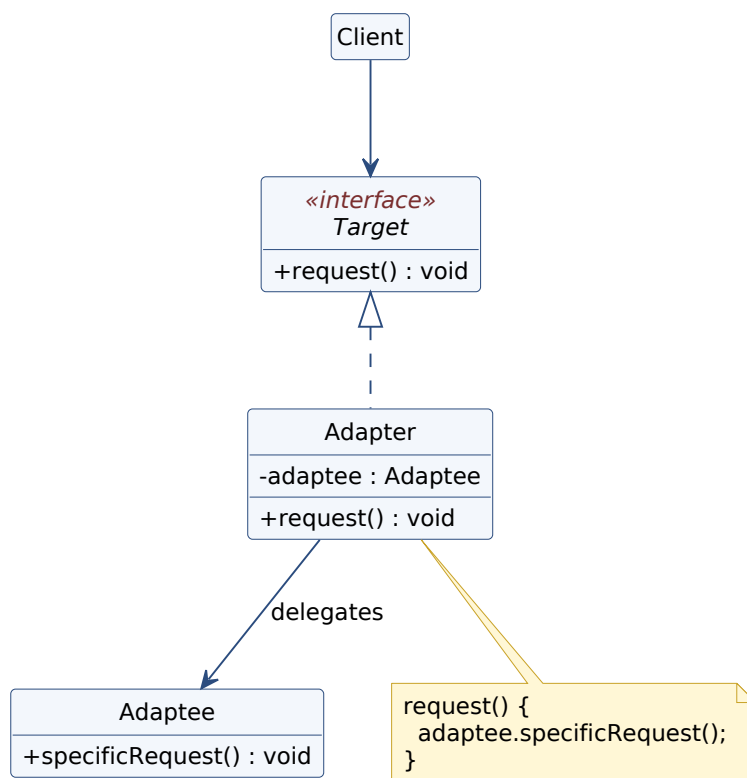
Adapter lets classes with **incompatible interfaces work together**. When a client expects interface A but an existing (often third-party) class offers interface B, and neither can be changed, an adapter class implements A and translates each call into calls on B. Category: structural. Also known as *Wrapper*.

#### Real-world examples

A **travel plug adapter** lets a laptop with a three-pin plug work in a two-pin socket without changing the laptop or the wall. A **memory-card reader** lets an SD card be used through a USB port.

#### Participants and UML

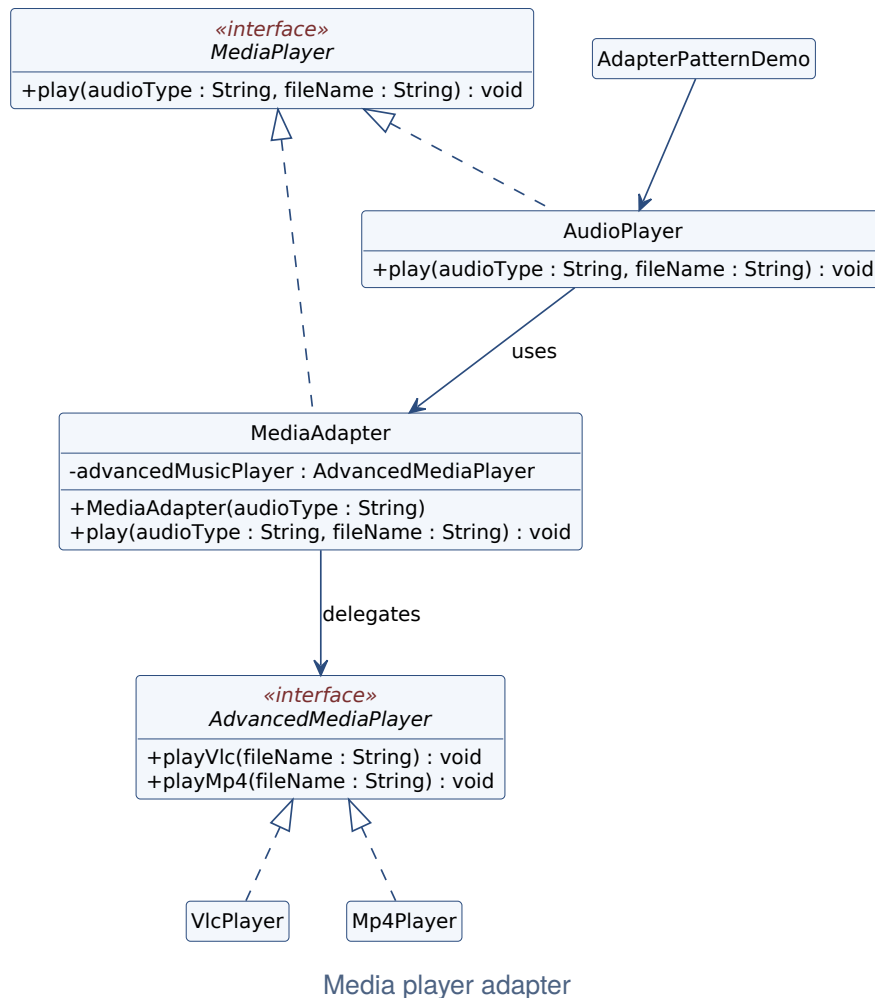
**Target** — the interface the client expects; **Adaptee** — the existing class with an incompatible interface; **Adapter** — implements Target and holds an Adaptee, converting calls; **Client** — works only with Target.



General structure of the (object) Adapter

## Music player solution

**Pattern: Adapter.** Target = MediaPlayer.play(type, file); Adaptee = the advanced library AdvancedMediaPlayer with playVlc() and playMp4(); Adapter = MediaAdapter; Client = AudioPlayer, which keeps playing MP3 itself and delegates other formats to the adapter.



### Java: Q\_MediaAdapter.java

```

1  interface MediaPlayer {                                // Target (what the client expects)
2      void play(String audioType, String fileName);
3  }
4
5  interface AdvancedMediaPlayer {                        // Adaptee interface (external
6      void playVlc(String fileName);                    library)
7      void playMp4(String fileName);
8  }
9
10 class VlcPlayer implements AdvancedMediaPlayer {
11     public void playVlc(String fileName) { System.out.println("Playing vlc file. Name: " +
12         fileName); }
13     public void playMp4(String fileName) { /* not supported */ }
14 }
15
16 class Mp4Player implements AdvancedMediaPlayer {
17     public void playVlc(String fileName) { /* not supported */ }
18 }
  
```

```

17     public void playMp4(String fileName) { System.out.println("Playing mp4 file. Name: " +
18         fileName); }
19 }
20 class MediaAdapter implements MediaPlayer {           // Adapter
21     private final AdvancedMediaPlayer advancedMusicPlayer;
22
23     MediaAdapter(String audioType) {
24         if (audioType.equalsIgnoreCase("vlc"))         advancedMusicPlayer = new VlcPlayer();
25         else if (audioType.equalsIgnoreCase("mp4")) advancedMusicPlayer = new Mp4Player();
26         else throw new IllegalArgumentException(audioType);
27     }
28
29     @Override public void play(String audioType, String fileName) { // translate the
30         call
31         if (audioType.equalsIgnoreCase("vlc")) advancedMusicPlayer.playVlc(fileName);
32         else
33             advancedMusicPlayer.playMp4(fileName);
34     }
35
36 class AudioPlayer implements MediaPlayer {           // existing player, still plays mp3
37     itself
38     @Override public void play(String audioType, String fileName) {
39         if (audioType.equalsIgnoreCase("mp3")) {
40             System.out.println("Playing mp3 file. Name: " + fileName);
41         } else if (audioType.equalsIgnoreCase("vlc") || audioType.equalsIgnoreCase("mp4"))
42         {
43             new MediaAdapter(audioType).play(audioType, fileName);
44         } else {
45             System.out.println("Invalid media. " + audioType + " format not supported");
46         }
47     }
48
49 public class Q_MediaAdapter {
50     public static void main(String[] args) {
51         AudioPlayer player = new AudioPlayer();
52         player.play("mp3", "song.mp3");
53         player.play("mp4", "video.mp4");
54         player.play("vlc", "movie.vlc");
55         player.play("avi", "clip.avi");
56     }
57 }

```

#### Program output (compiled and run)

```

1 Playing mp3 file. Name: song.mp3
2 Playing mp4 file. Name: video.mp4
3 Playing vlc file. Name: movie.vlc
4 Invalid media. avi format not supported

```

## Benefits

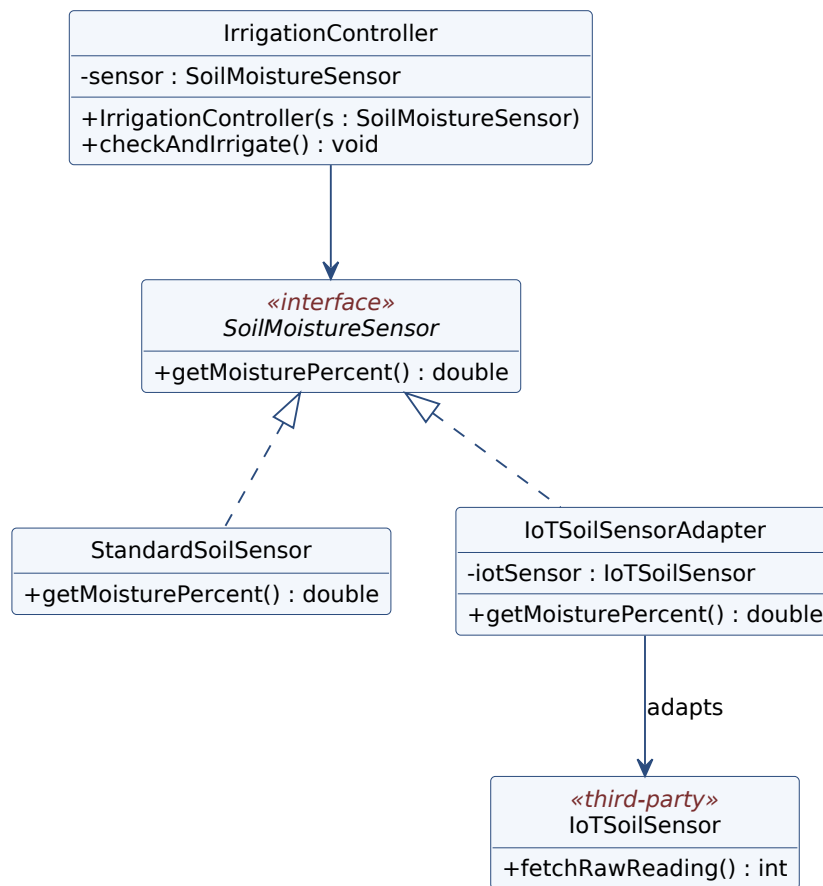
- Existing library and player code are **reused without modification** (Open/Closed).
- The conversion logic is isolated in one class (Single Responsibility).
- The client depends only on the Target interface, so the adaptee can be replaced later.

**Q42. Adapter for an IoT soil-moisture sensor (AAS)****Asked in:** CT-2 Q3**Marks:** 5

The AAS irrigation controller works with sensors through `getMoisturePercent()`. A new third-party IoT soil sensor provides only `fetchRawReading()`, returning a raw ADC value (0 = wet, 1023 = dry), and its code cannot be changed. Use the Adapter pattern so the controller can use the IoT sensor without modification. (*paraphrased from CT-2*)

**Mapping roles**

Client = IrrigationController; Target = SoilMoistureSensor; Adaptee = IoTSoilSensor; Adapter = IoTSoilSensorAdapter, which calls `fetchRawReading()` and converts the value:  $\text{percent} = \frac{1023 - \text{raw}}{1023} \times 100$ .



IoT soil sensor adapter

**Java: Q\_SoilAdapter.java**

```

1 interface SoilMoistureSensor {                                // Target: what the controller
2     expects
3     double getMoisturePercent();
4 }
5 class StandardSoilSensor implements SoilMoistureSensor {
6     public double getMoisturePercent() { return 45.0; }
7 }
8
9 class IoTSoilSensor {                                         // Adaptee: third-party, cannot be
    changed

```

```

10     public int fetchRawReading() { return 818; }           // ADC value: 0 = wet ... 1023 = dry
11 }
12
13 class IoTSoilSensorAdapter implements SoilMoistureSensor { // Adapter
14     private final IoTSoilSensor iotSensor;
15     IoTSoilSensorAdapter(IoTSoilSensor iotSensor) { this.iotSensor = iotSensor; }
16
17     @Override public double getMoisturePercent() {
18         int raw = iotSensor.fetchRawReading();           // call the incompatible
19         API                                               // convert raw ->
20         return Math.round((1023 - raw) * 1000.0 / 1023) / 10.0; percent
21     }
22 }
23
24 class IrrigationController {                             // existing client: NOT modified
25     private static final double THRESHOLD = 30.0;
26     private final SoilMoistureSensor sensor;
27     IrrigationController(SoilMoistureSensor sensor) { this.sensor = sensor; }
28
29     void checkAndIrrigate() {
30         double m = sensor.getMoisturePercent();
31         System.out.println("Moisture " + m + " percent -> " + (m < THRESHOLD ? "START pump
32         " : "pump stays OFF"));
33     }
34 }
35
36 public class Q_SoilAdapter {
37     public static void main(String[] args) {
38         new IrrigationController(new StandardSoilSensor()).checkAndIrrigate();
39         new IrrigationController(new IoTSoilSensorAdapter(new IoTSoilSensor())).
40         checkAndIrrigate();
41     }
42 }

```

#### Program output (compiled and run)

```

1 Moisture 45.0 percent -> pump stays OFF
2 Moisture 20.0 percent -> START pump

```

### Why this is a good design

The controller and the existing `StandardSoilSensor` are untouched; the IoT sensor is plugged in through the same interface; data-format conversion lives in exactly one class; if another vendor appears, only a new adapter is written.

#### Q43. Mobile-wallet payment adapter; object vs class adapter

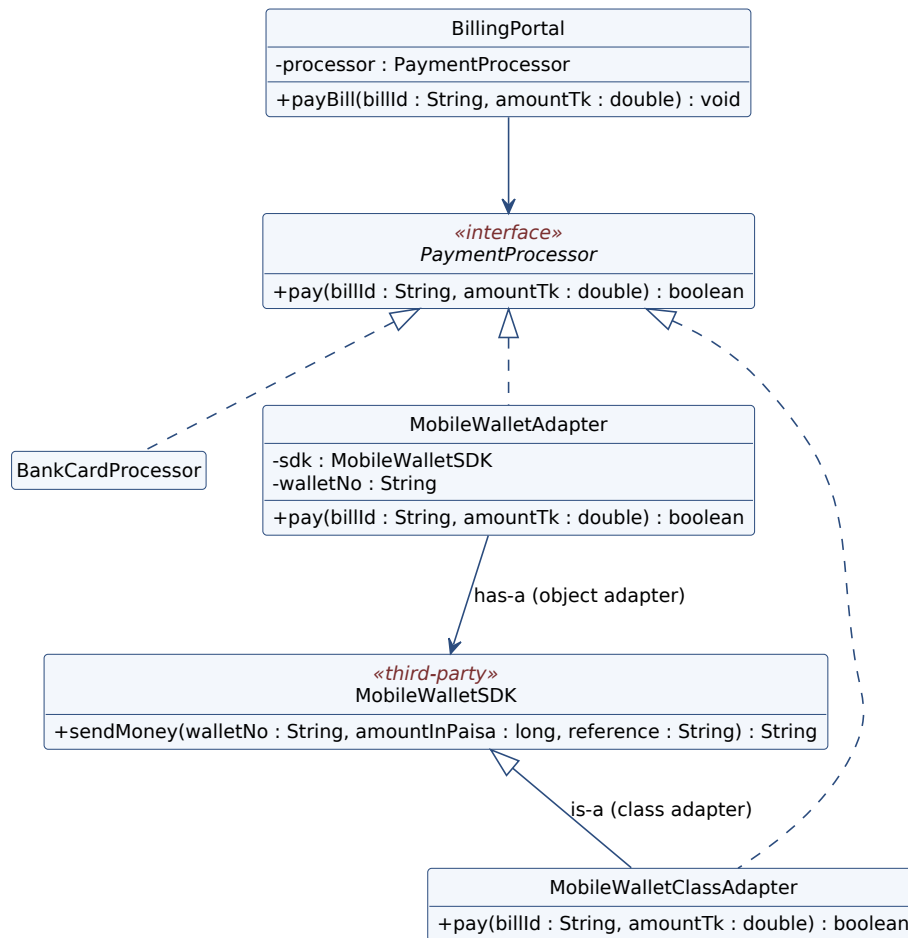
**Asked in:** Set-2 Q5(a) · New

**Marks:** 4.5

The DUMS billing portal pays bills through `PaymentProcessor.pay(String billId, double amountTk): boolean`. A third-party `MobileWalletSDK` offers only `sendMoney(String walletNo, long amountInPaisa, String reference): String` and cannot be modified. Apply the Adapter pattern with UML and code, and distinguish between an object adapter and a class adapter.

### What the adapter must translate

Method name (`pay` → `sendMoney`); parameters (Taka as `double` → paisa as `long`, bill id → reference, plus wallet number); return type (transaction `String` → `boolean`).



Both adapter forms

#### Java: Q\_PaymentAdapter.java

```

1  interface PaymentProcessor {                                // Target
2      boolean pay(String billId, double amountTk);
3  }
4
5  class BankCardProcessor implements PaymentProcessor {
6      public boolean pay(String billId, double amountTk) {
7          System.out.println("Card payment Tk. " + amountTk + " for " + billId);
8          return true;
9      }
10 }
11
12 class MobileWalletSDK {                                    // Adaptee (third-party
13     library)
14     public String sendMoney(String walletNo, long amountInPaisa, String reference) {
15         System.out.println("Wallet " + walletNo + " sent " + amountInPaisa + " paisa, ref
16         " + reference);
17         return amountInPaisa > 0 ? "TXN-7781" : "FAILED";
18     }
19 }
20 // Object adapter: HAS-A adaptee (composition)
21 class MobileWalletAdapter implements PaymentProcessor {
22     private final MobileWalletSDK sdk;
23     private final String walletNo;

```

```

23     MobileWalletAdapter(MobileWalletSDK sdk, String walletNo) { this.sdk = sdk; this.
        walletNo = walletNo; }
24
25     @Override public boolean pay(String billId, double amountTk) {
26         long paisa = Math.round(amountTk * 100);           // Tk -> paisa
27         String txn = sdk.sendMoney(walletNo, paisa, billId); // re-map parameters
28         return !txn.equals("FAILED");                     // String -> boolean
29     }
30 }
31
32 // Class adapter: IS-A adaptee (inheritance) - works only if the SDK class is not final
33 class MobileWalletClassAdapter extends MobileWalletSDK implements PaymentProcessor {
34     private final String walletNo;
35     MobileWalletClassAdapter(String walletNo) { this.walletNo = walletNo; }
36
37     @Override public boolean pay(String billId, double amountTk) {
38         return !sendMoney(walletNo, Math.round(amountTk * 100), billId).equals("FAILED");
39     }
40 }
41
42 class BillingPortal {                                     // Client: unchanged
43     private final PaymentProcessor processor;
44     BillingPortal(PaymentProcessor processor) { this.processor = processor; }
45     void payBill(String billId, double amountTk) {
46         System.out.println(billId + (processor.pay(billId, amountTk) ? " PAID" : " FAILED"
47     ));
48     }
49 }
50 public class Q_PaymentAdapter {
51     public static void main(String[] args) {
52         new BillingPortal(new BankCardProcessor()).payBill("WATER-2291", 850.50);
53         new BillingPortal(new MobileWalletAdapter(new MobileWalletSDK(), "01712345678")).
54         payBill("ELEC-7710", 1240.75);
55         new BillingPortal(new MobileWalletClassAdapter("01812345678")).payBill("GAS-3301",
56         975.00);
57     }
58 }

```

#### Program output (compiled and run)

```

1 Card payment Tk. 850.5 for WATER-2291
2 WATER-2291 PAID
3 Wallet 01712345678 sent 124075 paisa, ref ELEC-7710
4 ELEC-7710 PAID
5 Wallet 01812345678 sent 97500 paisa, ref GAS-3301
6 GAS-3301 PAID

```

## Object adapter vs class adapter

|                                         | Object adapter                                    | Class adapter                                                                   |
|-----------------------------------------|---------------------------------------------------|---------------------------------------------------------------------------------|
| <b>Mechanism</b>                        | <b>composition:</b> adapter <i>has</i> an adaptee | <b>inheritance:</b> adapter <i>extends</i> the adaptee                          |
| <b>UML</b>                              | association from adapter to adaptee               | generalization to adaptee + realization of target                               |
| <b>Works with subclasses of adaptee</b> | yes — any adaptee object can be passed            | no — fixed to one adaptee class                                                 |
| <b>Final adaptee class</b>              | fine                                              | impossible if the class is <code>final</code> ; Java allows only one superclass |
| <b>Override adaptee behaviour</b>       | harder                                            | easy (inherits it)                                                              |
| <b>Preferred</b>                        | usually (“favour composition”)                    | rarely                                                                          |

#### Q44. Facade pattern: scenario, steps and facilities

**Asked in:** 2021 Q6(a),(b),(c) · Set-2 Q5(b)

**Marks:** 1.75 + 4 + 3 / 4.25

Present a scenario where you may need the Facade design pattern. Explain the steps to implement it. What facilities does the Facade pattern provide? (Set-2) Registering a new DUMS resident requires calls to `HousingService`, `UtilityBillingService`, `GarbageService` and `NotificationService` in the correct order; show how Facade simplifies this with UML and code.

### Facade pattern

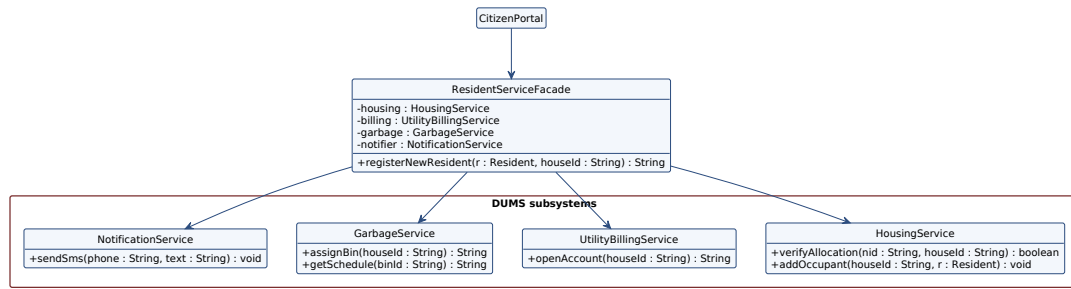
**Intent:** provide a **unified, simplified interface to a set of interfaces in a subsystem**. The facade does not add new functionality; it coordinates existing subsystem classes so clients need not know them. Category: structural.

#### (a) Scenario

In DUMS, when a new resident moves into an allotted flat, the citizen portal must: verify the allocation and add the occupant (housing), open a utility account (billing), assign a garbage bin and fetch its pickup schedule (garbage), and send a welcome SMS (notification). Without a facade, the portal team must learn four APIs and call about seven methods in the right order — and repeat this in the mobile app and the officer panel. Other classic scenarios: a home-theatre “watch movie” button, or a compiler’s `single compile()` over scanner, parser and code generator.

#### (b) Steps to implement

1. **Identify the subsystem** classes and the common high-level tasks clients need (register resident).
2. **Create a facade class** (`ResidentServiceFacade`) that holds references to the subsystem objects.
3. **Add simple methods** to the facade, one per high-level task, which call the subsystem objects in the correct order and handle intermediate results.
4. **Make clients use the facade** instead of the subsystem classes.
5. **Keep the subsystem accessible** for advanced clients that need fine control (the facade is optional, not a wall).



Facade over DUMS subsystems

## Java: Q\_Facade.java

```

1 class Resident {
2     final String name, nid, phone;
3     Resident(String name, String nid, String phone) { this.name = name; this.nid = nid;
4         this.phone = phone; }
5 }
6 class HousingService {
7     boolean verifyAllocation(String nid, String houseId) {
8         System.out.println(" Housing : allocation of " + houseId + " verified for NID " +
9             nid);
10        return true;
11    }
12    void addOccupant(String houseId, Resident r) { System.out.println(" Housing : " + r.
13        name + " added to " + houseId); }
14 }
15 class UtilityBillingService {
16     String openAccount(String houseId) {
17         System.out.println(" Billing : utility account opened for " + houseId);
18         return "UA-" + houseId;
19     }
20 }
21 class GarbageService {
22     String assignBin(String houseId) {
23         System.out.println(" Garbage : bin BIN-77 assigned to " + houseId);
24         return "BIN-77";
25     }
26     String getSchedule(String binId) { return "Sun/Wed 7:00 AM"; }
27 }
28 class NotificationService {
29     void sendSms(String phone, String text) { System.out.println(" SMS -> " + phone + ":
30         " + text); }
31 }
32 class ResidentServiceFacade { // one simple entry point
33     private final HousingService housing = new HousingService();
34     private final UtilityBillingService billing = new UtilityBillingService();
35     private final GarbageService garbage = new GarbageService();
36     private final NotificationService notifier = new NotificationService();
37
38     public String registerNewResident(Resident r, String houseId) {
39         if (!housing.verifyAllocation(r.nid, houseId)) return "REJECTED";
40         housing.addOccupant(houseId, r);
41     }

```

```

42     String account = billing.openAccount(houseId);
43     String bin = garbage.assignBin(houseId);
44     String schedule = garbage.getSchedule(bin);
45     notifier.sendSms(r.phone, "Welcome! Account " + account + ", garbage pickup " +
    schedule);
46     return account;
47 }
48 }
49
50 public class Q_Facade {                                // plays the role of CitizenPortal
51     public static void main(String[] args) {
52         ResidentServiceFacade facade = new ResidentServiceFacade();
53         System.out.println("Registering new resident...");
54         String acc = facade.registerNewResident(new Resident("Nusrat Jahan", "5512349876",
    "01911223344"), "H-45B");
55         System.out.println("Done. Account = " + acc);
56     }
57 }

```

#### Program output (compiled and run)

```

1 Registering new resident...
2   Housing : allocation of H-45B verified for NID 5512349876
3   Housing : Nusrat Jahan added to H-45B
4   Billing : utility account opened for H-45B
5   Garbage : bin BIN-77 assigned to H-45B
6   SMS -> 01911223344: Welcome! Account UA-H-45B, garbage pickup Sun/Wed 7:00 AM
7   Done. Account = UA-H-45B

```

### (c) Facilities provided

- **Simplicity:** one method instead of many; clients are easier to write and read.
- **Loose coupling:** clients depend on one facade, not on four subsystems; subsystems can change internally without affecting clients.
- **Layering:** a facade defines the entry point to each subsystem layer.
- **Consistency and less duplication:** the correct call order is written once.
- **Flexibility retained:** subsystem classes remain available when needed.

#### Q45. Composite pattern for City, Zone, Ward and Bin

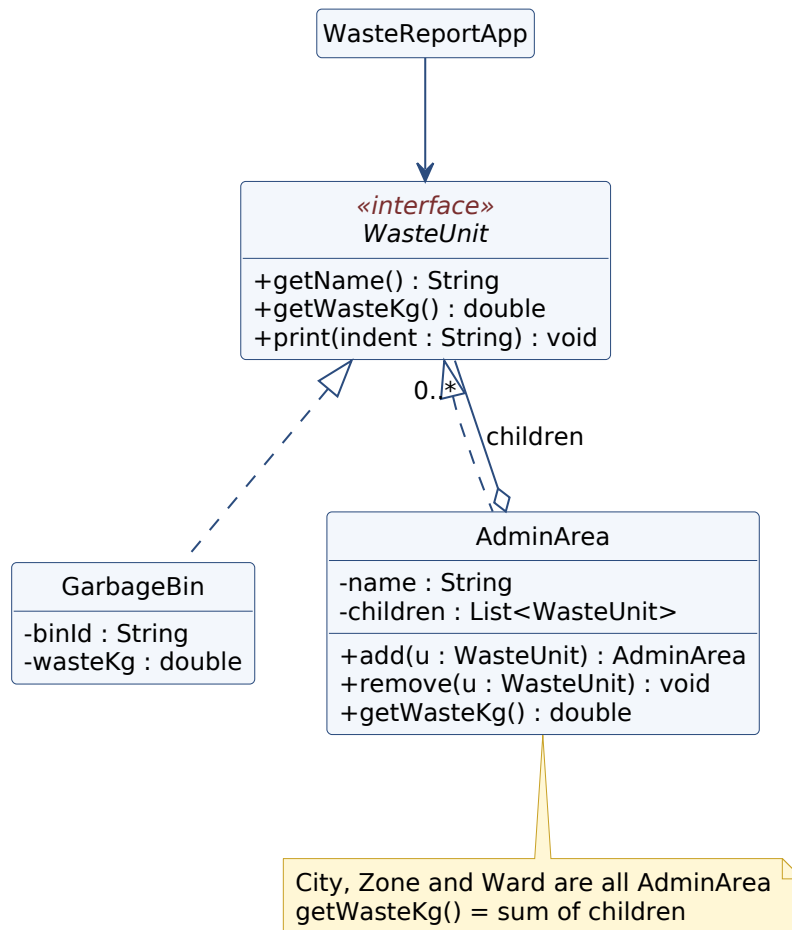
**Asked in:** Set-2 Q6(a) · New (Notes)

**Marks:** 4.5

A DUMS city is divided into zones, zones into wards, and wards contain garbage bins. The authority needs the total waste at any level, and the report code must treat a single bin and a whole zone in the same way. Which design pattern is suitable? Explain with a UML diagram and Java code.

### Composite pattern

**Intent:** compose objects into **tree structures** to represent part-whole hierarchies, and let clients treat **individual objects and compositions uniformly**. Roles: **Component** (WasteUnit), **Leaf** (GarbageBin), **Composite** (AdminArea for city, zone and ward, holding a list of Components), **Client** (report app).



Composite waste hierarchy

## Java: Q\_CompositeCity.java

```

1  import java.util.ArrayList;
2  import java.util.List;
3
4  interface WasteUnit {                                // Component
5      String getName();
6      double getWasteKg();
7      void print(String indent);
8  }
9
10 class GarbageBin implements WasteUnit {               // Leaf
11     private final String binId;
12     private final double wasteKg;
13     GarbageBin(String binId, double wasteKg) { this.binId = binId; this.wasteKg = wasteKg; }
14     public String getName() { return binId; }
15     public double getWasteKg() { return wasteKg; }
16     public void print(String indent) { System.out.println(indent + "- " + binId + " : " +
17         wasteKg + " kg"); }
18 }
19
20 class AdminArea implements WasteUnit {                 // Composite: City, Zone or Ward
21     private final String name;
22     private final List<WasteUnit> children = new ArrayList<>();
23     AdminArea(String name) { this.name = name; }

```

```

23 public AdminArea add(WasteUnit u) { children.add(u); return this; }
24 public void remove(WasteUnit u) { children.remove(u); }
25 public String getName() { return name; }
26 public double getWasteKg() {
27     double total = 0;
28     for (WasteUnit u : children) total += u.getWasteKg(); // recursive sum
29     return total;
30 }
31 public void print(String indent) {
32     System.out.println(indent + "+ " + name + " : " + getWasteKg() + " kg");
33     for (WasteUnit u : children) u.print(indent + "  ");
34 }
35 }
36
37 public class Q_CompositeCity {
38     public static void main(String[] args) {
39         AdminArea ward21 = new AdminArea("Ward-21").add(new GarbageBin("BIN-1", 120)).add(
40 new GarbageBin("BIN-2", 80));
41         AdminArea ward22 = new AdminArea("Ward-22").add(new GarbageBin("BIN-3", 150));
42         AdminArea zone5 = new AdminArea("Zone-5").add(ward21).add(ward22);
43         AdminArea zone6 = new AdminArea("Zone-6").add(new AdminArea("Ward-30").add(new
44 GarbageBin("BIN-4", 60)));
45         AdminArea city = new AdminArea("City Corporation").add(zone5).add(zone6);
46
47         city.print("");
48         WasteUnit unit = ward21; // leaf or composite: same
49         interface
50         System.out.println("Ward-21 total = " + unit.getWasteKg() + " kg");
51     }
52 }

```

#### Program output (compiled and run)

```

1 + City Corporation : 410.0 kg
2   + Zone-5 : 350.0 kg
3     + Ward-21 : 200.0 kg
4       - BIN-1 : 120.0 kg
5       - BIN-2 : 80.0 kg
6     + Ward-22 : 150.0 kg
7       - BIN-3 : 150.0 kg
8   + Zone-6 : 60.0 kg
9     + Ward-30 : 60.0 kg
10      - BIN-4 : 60.0 kg
11 Ward-21 total = 200.0 kg

```

## How it works

`GarbageBin.getWasteKg()` returns its own value; `AdminArea.getWasteKg()` asks each child and adds the results — the call recurses down the tree. The client calls the same method on the city, a ward or a bin.

## Benefits

Uniform client code, easy to add new levels (e.g. Mahalla) or new leaves (e.g. CommunityContainer), and naturally recursive operations (totals, printing, searching).

#### The notes' version

The notes use one `Employee` class that holds a list of subordinate `Employees` (CEO → heads → clerks). It is a simplified composite where leaf and composite are the same class.

**Q46. Bridge pattern for DUMS alerts****Asked in:** New (Notes)**Marks:** 4

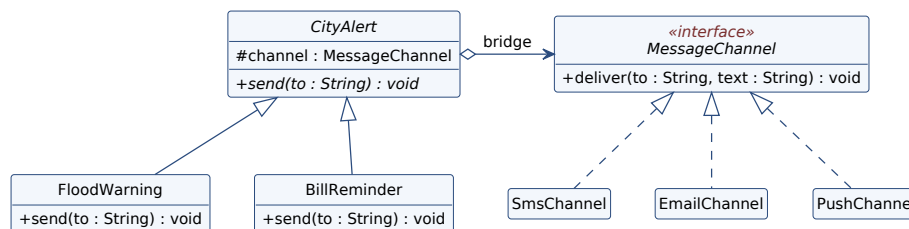
DUMS sends different kinds of alerts (flood warning, bill reminder) through different channels (SMS, e-mail, push). Creating a class for every combination leads to a class explosion. Apply the Bridge pattern.

**Problem**

With inheritance alone we need FloodWarningSms, FloodWarningEmail, FloodWarningPush, BillReminderSms, ...:  $m$  alert types  $\times$   $n$  channels classes. Adding one channel adds  $m$  classes.

**Bridge**

**Intent:** decouple an abstraction from its implementation so that the two can vary independently. The abstraction (CityAlert) holds a reference to an implementor interface (MessageChannel) — that reference is the bridge. Now we need only  $m + n$  classes.



Bridge between alert types and channels

**Java: Q\_BridgeAlert.java**

```

1  interface MessageChannel {                                // Implementor
2      void deliver(String to, String text);
3  }
4  class SmsChannel implements MessageChannel {
5      public void deliver(String to, String text) { System.out.println("SMS to " + to + ": "
6          + text); }
7  }
8  class EmailChannel implements MessageChannel {
9      public void deliver(String to, String text) { System.out.println("Email to " + to + ": "
10         + text); }
11 }
12 class PushChannel implements MessageChannel {
13     public void deliver(String to, String text) { System.out.println("Push to " + to + ": "
14         + text); }
15 }
16 abstract class CityAlert {                                // Abstraction
17     protected final MessageChannel channel;                // the bridge
18     protected CityAlert(MessageChannel channel) { this.channel = channel; }
19     public abstract void send(String to);
20 }
21 class FloodWarning extends CityAlert {                    // Refined abstraction
22     private final String area;
23     FloodWarning(MessageChannel c, String area) { super(c); this.area = area; }
24     public void send(String to) { channel.deliver(to, "FLOOD WARNING for " + area + ".
25         Move to higher ground."); }
26 }
27 class BillReminder extends CityAlert {                    // Refined abstraction
28 
```

```

25     private final double dueTk;
26     BillReminder(MessageChannel c, double dueTk) { super(c); this.dueTk = dueTk; }
27     public void send(String to) { channel.deliver(to, "Your utility bill of Tk. " + dueTk
    + " is due."); }
28 }
29
30 public class Q_BridgeAlert {
31     public static void main(String[] args) {
32         new FloodWarning(new SmsChannel(), "Mirpur").send("01710000000");
33         new FloodWarning(new PushChannel(), "Mirpur").send("app-user-42");
34         new BillReminder(new EmailChannel(), 1350).send("owner@mail.com");
35     }
36 }

```

#### Program output (compiled and run)

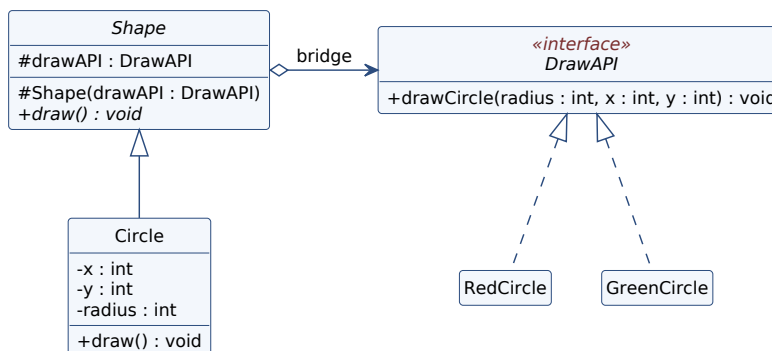
```

1 SMS to 01710000000: FLOOD WARNING for Mirpur. Move to higher ground.
2 Push to app-user-42: FLOOD WARNING for Mirpur. Move to higher ground.
3 Email to owner@mail.com: Your utility bill of Tk. 1350.0 is due.

```

## Bridge vs Adapter

Adapter is applied *after* design to make existing incompatible classes work together; Bridge is designed *up front* so that abstraction and implementation can evolve separately. The notes' example uses Shape (abstraction) with DrawAPI (implementor: RedCircle, GreenCircle).



Bridge example from the notes

### Q47. Filter (Criteria) pattern for DUMS complaints

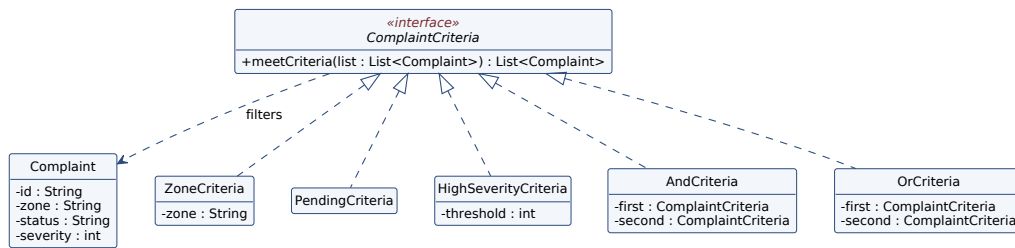
**Asked in:** New (Notes)

**Marks:** 4

The DUMS dashboard must list complaints by different, combinable conditions: by zone, by pending status, by high severity, and combinations such as "Zone-5 AND pending" or "(Zone-5 AND pending) OR severity at least 9". Apply the Filter pattern.

## Filter (Criteria) pattern

**Intent:** filter a set of objects using **different criteria** and **chain them in a decoupled way** through logical operations. Each criterion is a class implementing a common interface with `meetCriteria(List)`; `AndCriteria` and `OrCriteria` combine two criteria.



Complaint criteria

## Java: Q\_FilterComplaint.java

```

1  import java.util.ArrayList;
2  import java.util.List;
3
4  class Complaint {
5      private final String id, zone, status;
6      private final int severity;
7      Complaint(String id, String zone, String status, int severity) {
8          this.id = id; this.zone = zone; this.status = status; this.severity = severity;
9      }
10     String getZone() { return zone; }
11     String getStatus() { return status; }
12     int getSeverity() { return severity; }
13     @Override public String toString() { return id; }
14 }
15
16 interface ComplaintCriteria {
17     List<Complaint> meetCriteria(List<Complaint> list);
18 }
19
20 class ZoneCriteria implements ComplaintCriteria {
21     private final String zone;
22     ZoneCriteria(String zone) { this.zone = zone; }
23     public List<Complaint> meetCriteria(List<Complaint> list) {
24         List<Complaint> r = new ArrayList<>();
25         for (Complaint c : list) if (c.getZone().equalsIgnoreCase(zone)) r.add(c);
26         return r;
27     }
28 }
29
30 class PendingCriteria implements ComplaintCriteria {
31     public List<Complaint> meetCriteria(List<Complaint> list) {
32         List<Complaint> r = new ArrayList<>();
33         for (Complaint c : list) if (c.getStatus().equals("PENDING")) r.add(c);
34         return r;
35     }
36 }
37
38 class HighSeverityCriteria implements ComplaintCriteria {
39     private final int threshold;
40     HighSeverityCriteria(int threshold) { this.threshold = threshold; }
41     public List<Complaint> meetCriteria(List<Complaint> list) {
42         List<Complaint> r = new ArrayList<>();
43         for (Complaint c : list) if (c.getSeverity() >= threshold) r.add(c);
44         return r;
45     }
46 }

```

```

47
48 class AndCriteria implements ComplaintCriteria {           // filter the result of the first
    by the second
49     private final ComplaintCriteria first, second;
50     AndCriteria(ComplaintCriteria first, ComplaintCriteria second) { this.first = first;
    this.second = second; }
51     public List<Complaint> meetCriteria(List<Complaint> list) {
52         return second.meetCriteria(first.meetCriteria(list));
53     }
54 }
55
56 class OrCriteria implements ComplaintCriteria {             // union of both results
57     private final ComplaintCriteria first, second;
58     OrCriteria(ComplaintCriteria first, ComplaintCriteria second) { this.first = first;
    this.second = second; }
59     public List<Complaint> meetCriteria(List<Complaint> list) {
60         List<Complaint> r = new ArrayList<>(first.meetCriteria(list));
61         for (Complaint c : second.meetCriteria(list)) if (!r.contains(c)) r.add(c);
62         return r;
63     }
64 }
65
66 public class Q_FilterComplaint {
67     public static void main(String[] args) {
68         List<Complaint> all = List.of(
69             new Complaint("C1", "Zone-5", "PENDING", 8),
70             new Complaint("C2", "Zone-5", "RESOLVED", 9),
71             new Complaint("C3", "Zone-6", "PENDING", 3),
72             new Complaint("C4", "Zone-5", "PENDING", 2),
73             new Complaint("C5", "Zone-7", "RESOLVED", 10));
74
75         ComplaintCriteria zone5Pending = new AndCriteria(new ZoneCriteria("Zone-5"), new
    PendingCriteria());
76         ComplaintCriteria urgent = new OrCriteria(zone5Pending, new HighSeverityCriteria
    (9));
77
78         System.out.println("Zone-5 AND pending                : " + zone5Pending.
    meetCriteria(all));
79         System.out.println("(Zone-5 AND pending) OR severity >= 9 : " + urgent.
    meetCriteria(all));
80     }
81 }

```

#### Program output (compiled and run)

```

1 Zone-5 AND pending                : [C1, C4]
2 (Zone-5 AND pending) OR severity >= 9 : [C1, C4, C2, C5]

```

## Benefits

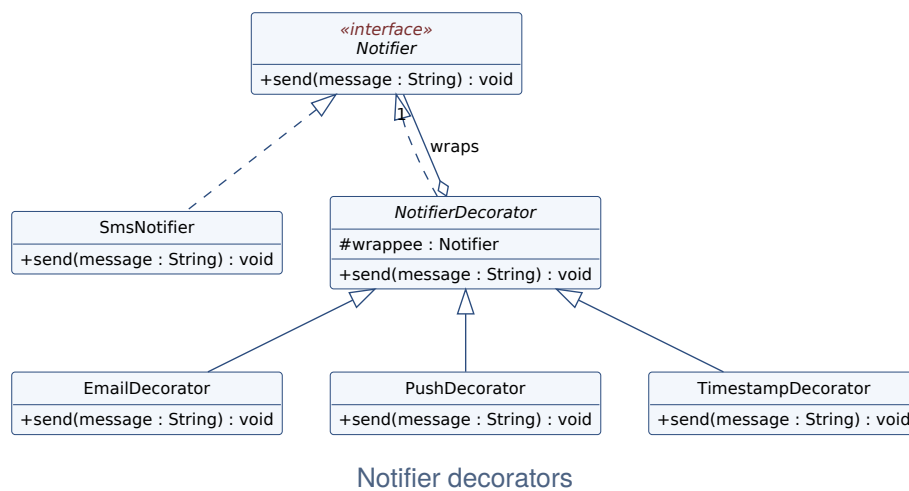
Each rule is a small, reusable, testable class; new rules are added without touching others; any combination can be built at run time without writing new filtering loops. The notes filter Person objects with CriteriaMale, CriteriaSingle, AndCriteria, OrCriteria in exactly the same way.

**Q48. Decorator pattern for layered notifications****Asked in:** Extra (not in notes; low priority)**Marks:** 4

DUMS sends a basic SMS for every event. For emergencies, the same message must also go by e-mail and push notification, with a timestamp, in any combination. Explain how the Decorator pattern adds these responsibilities without creating a subclass for every combination.

**Decorator**

**Intent:** attach additional responsibilities to an object dynamically. A decorator implements the same interface as the object it wraps, forwards calls to it, and adds behaviour before or after. Decorators can be stacked in any order.

**Java: Q\_Decorator.java**

```

1  interface Notifier {                                // Component
2      void send(String message);
3  }
4
5  class SmsNotifier implements Notifier {              // Concrete component
6      public void send(String message) { System.out.println("SMS : " + message); }
7  }
8
9  abstract class NotifierDecorator implements Notifier { // Base decorator: IS-A and HAS-A
10     Notifier
11     protected final Notifier wrappee;
12     NotifierDecorator(Notifier wrappee) { this.wrappee = wrappee; }
13     public void send(String message) { wrappee.send(message); } // forward by default
14 }
15
16 class EmailDecorator extends NotifierDecorator {
17     EmailDecorator(Notifier n) { super(n); }
18     @Override public void send(String message) {
19         super.send(message); // existing behaviour
20         System.out.println("EMAIL : " + message); // + extra behaviour
21     }
22 }
23
24 class PushDecorator extends NotifierDecorator {
25     PushDecorator(Notifier n) { super(n); }
26     @Override public void send(String message) {

```

```

26         super.send(message);
27         System.out.println("PUSH : " + message);
28     }
29 }
30
31 class TimestampDecorator extends NotifierDecorator {
32     TimestampDecorator(Notifier n) { super(n); }
33     @Override public void send(String message) {
34         super.send("[10:30 AM] " + message);           // changes the message, then
35         forwards
36     }
37 }
38
39 public class Q_Decorator {
40     public static void main(String[] args) {
41         Notifier basic = new SmsNotifier();
42         basic.send("Garbage collected in Ward-21");
43
44         System.out.println("--- emergency: SMS + Email + Push, with timestamp ---");
45         Notifier emergency = new TimestampDecorator(new PushDecorator(new EmailDecorator(
46             new SmsNotifier())));
47         emergency.send("Fire reported at Ward-14 market");
48     }
49 }

```

#### Program output (compiled and run)

```

1 SMS : Garbage collected in Ward-21
2 --- emergency: SMS + Email + Push, with timestamp ---
3 SMS : [10:30 AM] Fire reported at Ward-14 market
4 EMAIL : [10:30 AM] Fire reported at Ward-14 market
5 PUSH : [10:30 AM] Fire reported at Ward-14 market

```

## Decorator vs inheritance

Inheritance fixes behaviour at compile time and needs a subclass per combination (SmsEmail, SmsEmailPush, ...). Decorators combine behaviour at run time with one small class per feature. Java I/O uses this idea: `new BufferedReader(new InputStreamReader(System.in))`.

## Section B — Behavioural Patterns and MVC

### Q49. Observer: underlying principle; Facebook followers

Asked in: 2024 Q6(a) · 2021 Q5(b)

Marks: 4.5 / 4.5

(2021) What is the underlying principle of the Observer design pattern? Explain with an example.  
 (2024) In Facebook, a user can follow another user. Whenever the followed user posts a new status update, all followers receive a notification in their feeds. Which design pattern is most suitable for implementing this notification system? Explain it with a UML class diagram.

### Observer pattern

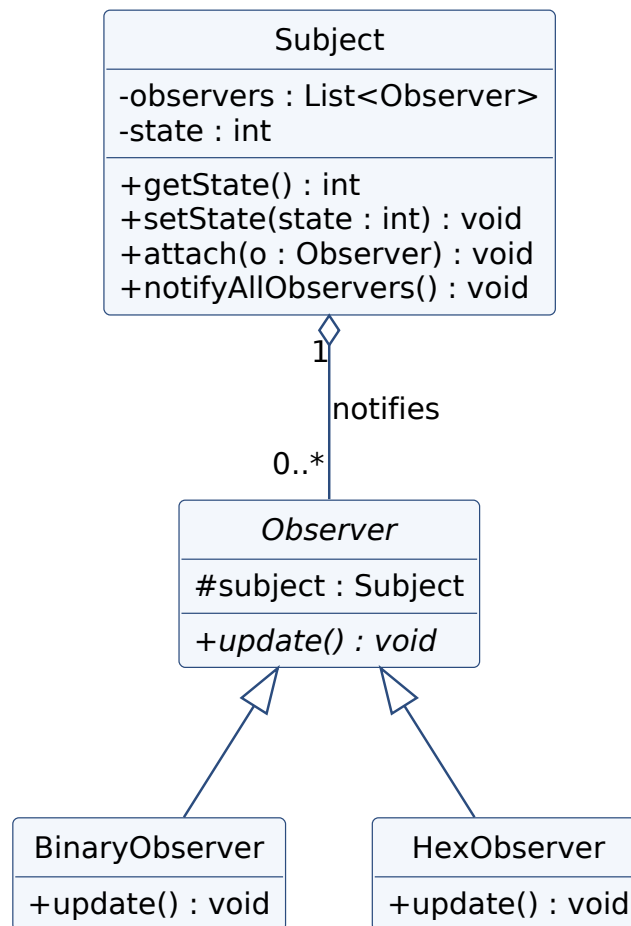
**Intent:** define a **one-to-many dependency** between objects so that when one object (the **subject**) changes state, all its dependents (**observers**) are **notified and updated automatically**. Category: behavioural. Also called Publish–Subscribe.

### Underlying principle

- **Loose coupling between subject and observers.** The subject knows only that its observers implement an Observer interface with `update()`; it does not know their concrete classes or what they do with the news.
- **Push instead of poll** (the “Hollywood principle”: *don’t call us, we’ll call you*). Observers do not keep asking “has anything changed?”; the subject calls them when it changes.
- **Dynamic subscription.** Observers can register and unregister at run time; new observer types are added without modifying the subject (Open/Closed).

### Participants

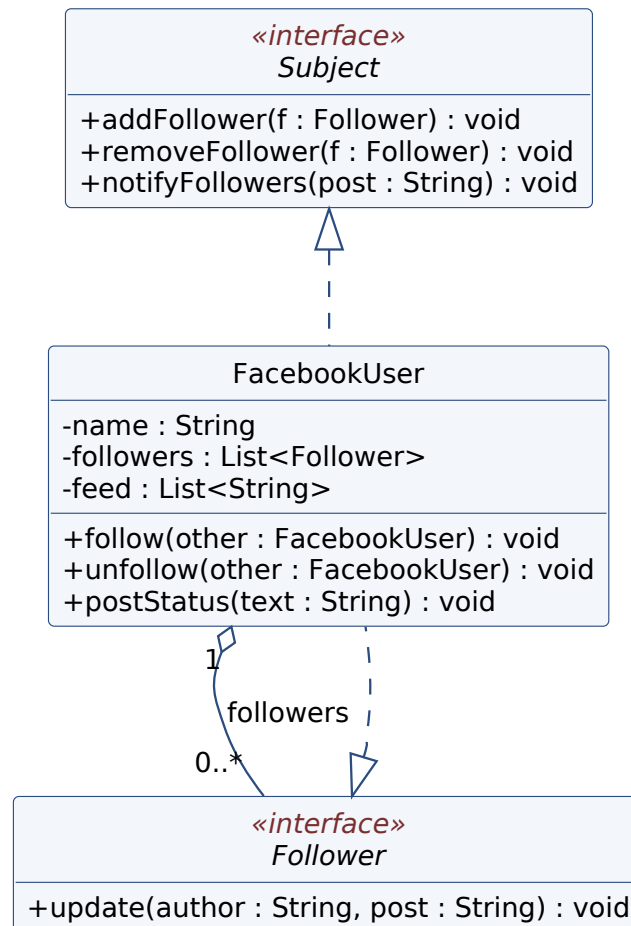
**Subject** — keeps a list of observers; offers `attach/detach/notify`. **ConcreteSubject** — holds state and calls `notify` when it changes. **Observer** — interface with `update()`. **ConcreteObserver** — reacts to the update.



Observer structure in the notes (state shown in binary / hex)

## Facebook followers

**Most suitable pattern: Observer.** The followed user is the subject; followers are observers; posting a status is the state change; the notification is `update()`. Because every user can both be followed and follow others, `FacebookUser` implements *both* roles.



Observer for Facebook followers

## Java: Q\_ObserverFacebook.java

```

1  import java.util.ArrayList;
2  import java.util.List;
3
4  interface Follower {                                // Observer
5      void update(String author, String post);
6  }
7
8  interface Subject {                                // Subject
9      void addFollower(Follower f);
10     void removeFollower(Follower f);
11     void notifyFollowers(String post);
12 }
13
14 class FacebookUser implements Subject, Follower {    // a user can be followed AND can
15     follow
16     private final String name;
17     private final List<Follower> followers = new ArrayList<>();
18     private final List<String> feed = new ArrayList<>();
19
20     FacebookUser(String name) { this.name = name; }
21
22     public void follow(FacebookUser other) { other.addFollower(this); }
23     public void unfollow(FacebookUser other) { other.removeFollower(this); }

```

```

24     public void postStatus(String text) {
25         System.out.println(name + " posted: \"" + text + "\"");
26         notifyFollowers(text);                // state changed -> notify everyone
27     }
28
29     @Override public void addFollower(Follower f) { followers.add(f); }
30     @Override public void removeFollower(Follower f) { followers.remove(f); }
31     @Override public void notifyFollowers(String post) {
32         for (Follower f : followers) f.update(name, post);
33     }
34
35     @Override public void update(String author, String post) {
36         feed.add(author + ": " + post);
37         System.out.println("    feed of " + name + " <- " + author + ": " + post + " (feed
38         size " + feed.size() + ")");
39     }
40
41     public class Q_ObserverFacebook {
42         public static void main(String[] args) {
43             FacebookUser arif = new FacebookUser("Arif");
44             FacebookUser nadia = new FacebookUser("Nadia");
45             FacebookUser tanvir = new FacebookUser("Tanvir");
46
47             nadia.follow(arif);
48             tanvir.follow(arif);
49             arif.postStatus("Final exam routine published!");
50
51             tanvir.unfollow(arif);                // dynamic unsubscription
52             arif.postStatus("Study group at 8 PM");
53         }
54     }

```

#### Program output (compiled and run)

```

1 Arif posted: "Final exam routine published!"
2   feed of Nadia <- Arif: Final exam routine published! (feed size 1)
3   feed of Tanvir <- Arif: Final exam routine published! (feed size 1)
4 Arif posted: "Study group at 8 PM"
5   feed of Nadia <- Arif: Study group at 8 PM (feed size 2)

```

## Why Observer fits

The poster does not need to know who follows or how each feed is displayed; followers are added or removed at any time (Tanvir stops receiving posts after unfollowing); new kinds of followers (a page, an analytics service) can subscribe without changing FacebookUser.

### Q50. Observer for DUMS complaint status notifications

**Asked in:** CT-2 Q1

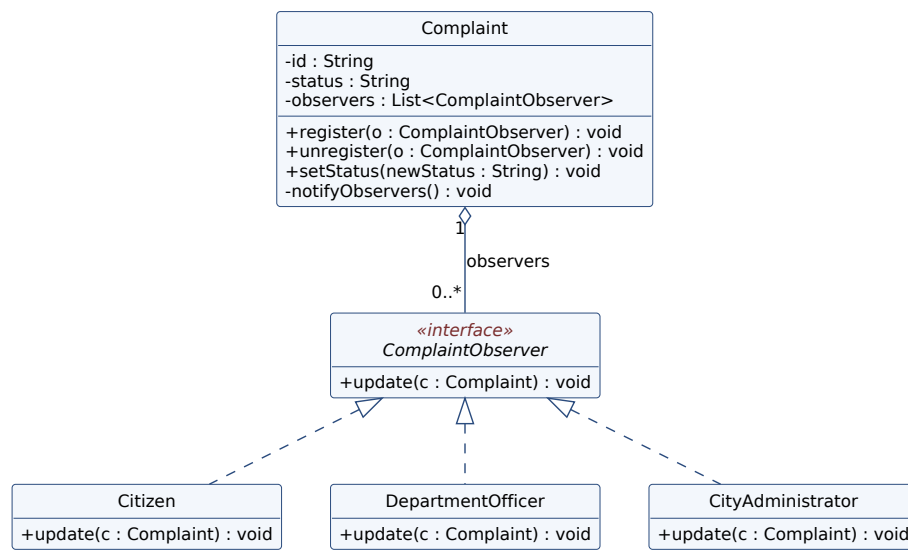
**Marks:** 5

When the status of a DUMS complaint changes (e.g. SUBMITTED → IN\_PROGRESS → RESOLVED), the citizen who filed it, the responsible department officer and the city administrator's monitoring panel must be notified automatically. Identify and apply a suitable design pattern with a UML diagram and code. (*paraphrased from CT-2*)

## Pattern and role mapping

**Observer.** Subject = Complaint (its status is the observed state). Observer interface = ComplaintObserver.update

Concrete observers = Citizen, DepartmentOfficer, CityAdministrator.



Observer for complaint status

#### Java: Q\_ObserverComplaint.java

```

1  import java.util.ArrayList;
2  import java.util.List;
3
4  interface ComplaintObserver {
5      void update(Complaint c);
6  }
7
8  class Complaint {                                // Subject
9      private final String id, category;
10     private String status = "SUBMITTED";
11     private final List<ComplaintObserver> observers = new ArrayList<>();
12
13     Complaint(String id, String category) { this.id = id; this.category = category; }
14
15     public void register(ComplaintObserver o) { observers.add(o); }
16     public void unregister(ComplaintObserver o) { observers.remove(o); }
17
18     public void setStatus(String newStatus) {
19         this.status = newStatus;
20         notifyObservers();                        // automatic, nobody has to poll
21     }
22     private void notifyObservers() {
23         for (ComplaintObserver o : observers) o.update(this);
24     }
25     public String getId() { return id; }
26     public String getCategory() { return category; }
27     public String getStatus() { return status; }
28 }
29
30 class Citizen implements ComplaintObserver {
31     private final String phone;
32     Citizen(String phone) { this.phone = phone; }
33     public void update(Complaint c) {
34         System.out.println(" [Citizen " + phone + "] complaint " + c.getId() + " is now "
35             + c.getStatus());
36     }
37 }
  
```

```

35     }
36 }
37
38 class DepartmentOfficer implements ComplaintObserver {
39     private final String department;
40     DepartmentOfficer(String department) { this.department = department; }
41     public void update(Complaint c) {
42         System.out.println(" [" + department + " officer] " + c.getCategory() + " case "
43         + c.getId() + ": " + c.getStatus());
44     }
45 }
46
47 class CityAdministrator implements ComplaintObserver {
48     public void update(Complaint c) {
49         System.out.println(" [City admin monitor] alert: " + c.getId() + " -> " + c.
50         getStatus());
51     }
52 }
53
54 public class Q_ObserverComplaint {
55     public static void main(String[] args) {
56         Complaint c = new Complaint("CMP-2041", "Waste collection");
57         c.register(new Citizen("01711-000111"));
58         c.register(new DepartmentOfficer("Waste Management"));
59         c.register(new CityAdministrator());
60
61         System.out.println("Status -> IN_PROGRESS");
62         c.setStatus("IN_PROGRESS");
63         System.out.println("Status -> RESOLVED");
64         c.setStatus("RESOLVED");
65     }
66 }

```

#### Program output (compiled and run)

```

1 Status -> IN_PROGRESS
2 [Citizen 01711-000111] complaint CMP-2041 is now IN_PROGRESS
3 [Waste Management officer] Waste collection case CMP-2041: IN_PROGRESS
4 [City admin monitor] alert: CMP-2041 -> IN_PROGRESS
5 Status -> RESOLVED
6 [Citizen 01711-000111] complaint CMP-2041 is now RESOLVED
7 [Waste Management officer] Waste collection case CMP-2041: RESOLVED
8 [City admin monitor] alert: CMP-2041 -> RESOLVED

```

## Advantages in DUMS

- setStatus() automatically informs everyone — no module has to poll the database.
- Complaint is not coupled to SMS, dashboards or officer apps; it only calls update().
- A new stakeholder (e.g. a WardCouncillor or a public-statistics service) is added by writing one observer class and registering it.

### Q51. Observer for an AAS weather station

Asked in: Set-1 Q6(a) · New

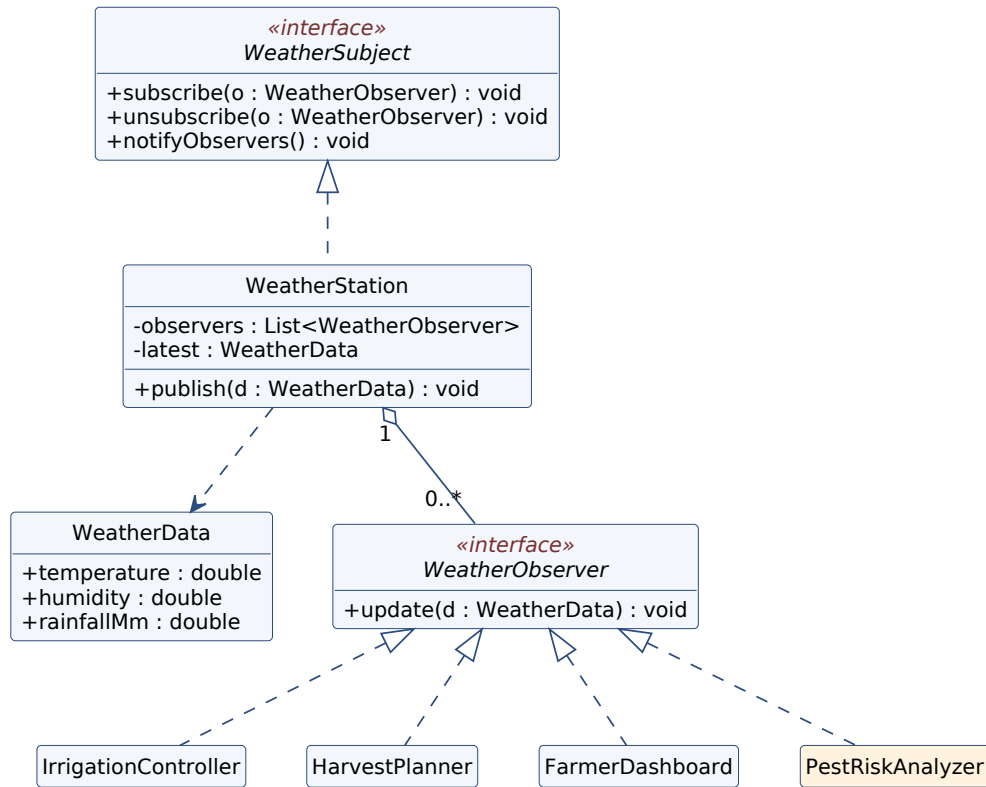
Marks: 4.5

In AAS, a WeatherStation publishes temperature, humidity and rainfall. The irrigation controller, harvest planner and farmer dashboard must react whenever a new reading arrives, and

new modules (e.g. a pest-risk analyser) will be added later without changing the station. Which design pattern is suitable? Explain with a UML class diagram and Java code.

### Pattern: Observer

Subject = WeatherStation; Observer = WeatherObserver; concrete observers = IrrigationController, HarvestPlanner, FarmerDashboard, and later PestRiskAnalyzer.



Observer for weather data (late addition highlighted)

#### Java: Q\_ObserverWeather.java

```

1 import java.util.ArrayList;
2 import java.util.List;
3
4 class WeatherData {
5     final double temperature, humidity, rainfallMm;
6     WeatherData(double t, double h, double r) { temperature = t; humidity = h; rainfallMm
7         = r; }
8 }
9 interface WeatherObserver { void update(WeatherData d); }
10
11 interface WeatherSubject {
12     void subscribe(WeatherObserver o);
13     void unsubscribe(WeatherObserver o);
14     void notifyObservers();
15 }
16
17 class WeatherStation implements WeatherSubject {
18     private final List<WeatherObserver> observers = new ArrayList<>();
19     private WeatherData latest;

```

```

20
21     public void subscribe(WeatherObserver o) { observers.add(o); }
22     public void unsubscribe(WeatherObserver o) { observers.remove(o); }
23     public void notifyObservers() { for (WeatherObserver o : observers) o.update(latest);
    }
24
25     public void publish(WeatherData d) {           // a new reading arrives from the sensors
26         latest = d;
27         notifyObservers();
28     }
29 }
30
31 class IrrigationController implements WeatherObserver {
32     public void update(WeatherData d) {
33         System.out.println(" Irrigation : " + (d.rainfallMm > 10 ? "skip today, enough
rain" : "schedule irrigation"));
34     }
35 }
36
37 class HarvestPlanner implements WeatherObserver {
38     public void update(WeatherData d) {
39         System.out.println(" Harvest      : " + (d.rainfallMm > 20 ? "postpone harvesting"
: "harvest window open"));
40     }
41 }
42
43 class FarmerDashboard implements WeatherObserver {
44     public void update(WeatherData d) {
45         System.out.println(" Dashboard : " + d.temperature + " C, humidity " + d.
humidity + ", rain " + d.rainfallMm + " mm");
46     }
47 }
48
49 class PestRiskAnalyzer implements WeatherObserver {    // added later, WeatherStation
unchanged
50     public void update(WeatherData d) {
51         boolean risk = d.humidity > 85 && d.temperature > 28;
52         System.out.println(" Pest risk : " + (risk ? "HIGH" : "normal"));
53     }
54 }
55
56 public class Q_ObserverWeather {
57     public static void main(String[] args) {
58         WeatherStation station = new WeatherStation();
59         station.subscribe(new IrrigationController());
60         station.subscribe(new HarvestPlanner());
61         station.subscribe(new FarmerDashboard());
62
63         System.out.println("Reading 1:");
64         station.publish(new WeatherData(33.5, 70, 2));
65
66         station.subscribe(new PestRiskAnalyzer());
67         System.out.println("Reading 2:");
68         station.publish(new WeatherData(30.0, 90, 25));
69     }
70 }

```

**Program output (compiled and run)**

```

1 Reading 1:
2   Irrigation : schedule irrigation
3   Harvest    : harvest window open
4   Dashboard  : 33.5 C, humidity 70.0, rain 2.0 mm
5 Reading 2:
6   Irrigation : skip today, enough rain
7   Harvest    : postpone harvesting
8   Dashboard  : 30.0 C, humidity 90.0, rain 25.0 mm
9   Pest risk   : HIGH

```

**Explanation**

`publish()` stores the reading and calls `notifyObservers()`, which calls `update()` on each subscriber. Each module applies its own rule (skip irrigation after rain, postpone harvest). `PestRiskAnalyzer` subscribes after the first reading and receives only the second one — it was added without editing `WeatherStation`.

**Push vs pull**

Here the subject **pushes** the `WeatherData` object in `update(d)`. In the **pull** model the subject sends only a signal and observers call `station.getLatest()` to fetch what they need. Push is simpler; pull avoids sending data some observers do not use.

**Q52. Chain of Responsibility: bank cheque authorisation****Asked in:** 2023 Q6(b)**Marks:** 4.25

You are designing a system for money withdrawal from a bank account using a cheque with the following authorisation rules: for Tk. < 10,000 only the cashier's authorisation is sufficient; for Tk. 10,000 to 10,00,000 authorisation from both the cashier and a senior officer is needed; for Tk. > 10,00,000 authorisation from the senior officer and the manager is needed. Implement this behaviour using an appropriate software design pattern.

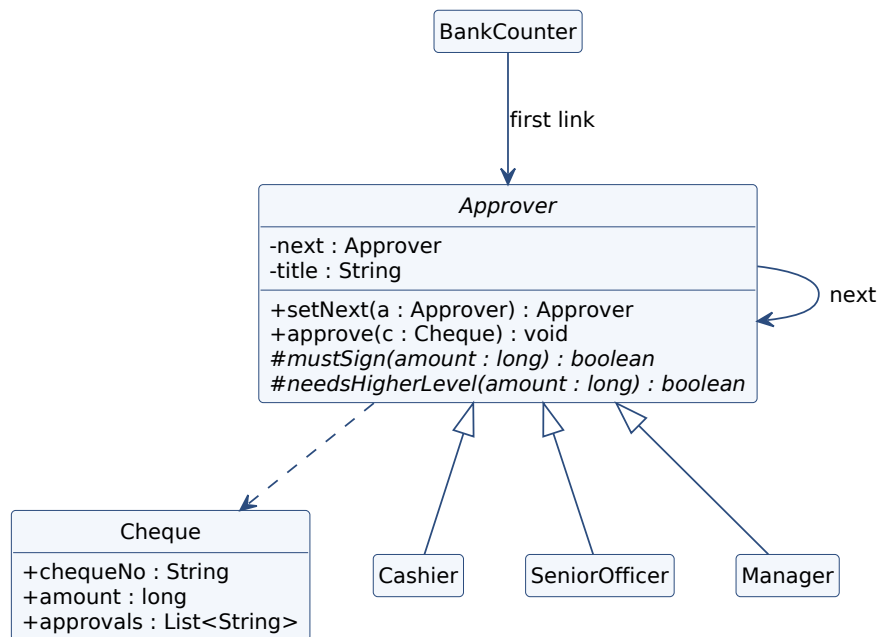
**Pattern: Chain of Responsibility**

**Intent:** avoid coupling the sender of a request to its receiver by giving **more than one object a chance to handle the request**. The handlers are linked in a **chain** and the request is **passed along** until the required handling is complete. Category: behavioural.

**Why it fits**

The cheque must pass through the authority levels Cashier → Senior Officer → Manager. Each level decides for itself (1) whether it must sign and (2) whether a higher level is still needed. The bank counter only hands the cheque to the first link and does not contain any if-else ladder of amounts.

| Amount (Tk.)        | Cashier         | Senior Officer  | Manager |
|---------------------|-----------------|-----------------|---------|
| below 10,000        | signs           | –               | –       |
| 10,000 to 10,00,000 | signs, forwards | signs           | –       |
| above 10,00,000     | forwards only   | signs, forwards | signs   |



Chain of Responsibility for cheque authorisation

## Java: Q\_CoRBank.java

```

1 import java.util.ArrayList;
2 import java.util.List;
3
4 class Cheque {
5     final String chequeNo;
6     final long amount;
7     final List<String> approvals = new ArrayList<>();
8     Cheque(String chequeNo, long amount) { this.chequeNo = chequeNo; this.amount = amount; }
9 }
10
11 abstract class Approver {
12     private Approver next;
13     private final String title;
14     protected Approver(String title) { this.title = title; }
15
16     public Approver setNext(Approver next) { this.next = next; return next; } // allows a
17     .setNext(b).setNext(c)
18
19     public final void approve(Cheque c) {
20         if (mustSign(c.amount)) { // my part of the authorisation
21             c.approvals.add(title);
22             System.out.println(" " + title + " authorised " + c.chequeNo);
23         }
24         if (needsHigherLevel(c.amount) && next != null) {
25             next.approve(c); // pass the request along the chain
26         }
27     }
28     protected abstract boolean mustSign(long amount);
29     protected abstract boolean needsHigherLevel(long amount);
30 }
31
32 class Cashier extends Approver { // signs up to 10 lakh
33     Cashier() { super("Cashier"); }
34 }
  
```

```

33     protected boolean mustSign(long amt)          { return amt <= 10_00_000; }
34     protected boolean needsHigherLevel(long amt) { return amt >= 10_000; }
35 }
36
37 class SeniorOfficer extends Approver {           // signs from 10,000 upward
38     SeniorOfficer() { super("Senior Officer"); }
39     protected boolean mustSign(long amt)          { return amt >= 10_000; }
40     protected boolean needsHigherLevel(long amt) { return amt > 10_00_000; }
41 }
42
43 class Manager extends Approver {                 // signs above 10 lakh
44     Manager() { super("Manager"); }
45     protected boolean mustSign(long amt)          { return amt > 10_00_000; }
46     protected boolean needsHigherLevel(long amt) { return false; }
47 }
48
49 public class Q_CoRBank {
50     public static void main(String[] args) {
51         Approver chain = new Cashier();
52         chain.setNext(new SeniorOfficer()).setNext(new Manager());
53
54         long[] amounts = { 7_500, 2_50_000, 25_00_000 };
55         int n = 1;
56         for (long a : amounts) {
57             Cheque c = new Cheque("CHQ-" + n++, a);
58             System.out.println("Withdraw Tk. " + a + ":");
59             chain.approve(c);
60             System.out.println(" => approved by " + c.approvals + ", cash released");
61         }
62     }
63 }

```

#### Program output (compiled and run)

```

1 Withdraw Tk. 7500:
2   Cashier authorised CHQ-1
3   => approved by [Cashier], cash released
4 Withdraw Tk. 250000:
5   Cashier authorised CHQ-2
6   Senior Officer authorised CHQ-2
7   => approved by [Cashier, Senior Officer], cash released
8 Withdraw Tk. 2500000:
9   Senior Officer authorised CHQ-3
10  Manager authorised CHQ-3
11  => approved by [Senior Officer, Manager], cash released

```

### Code explanation

`Approver.approve()` is the common algorithm of every link: sign if `mustSign()`, then forward if `needsHigherLevel()` and a next link exists. Subclasses only encode their own limits. `setNext()` returns the next handler so the chain is built in one line. Changing a limit or inserting an “Assistant Manager” touches one class and the chain set-up, not the client.

#### Do not skip this pattern

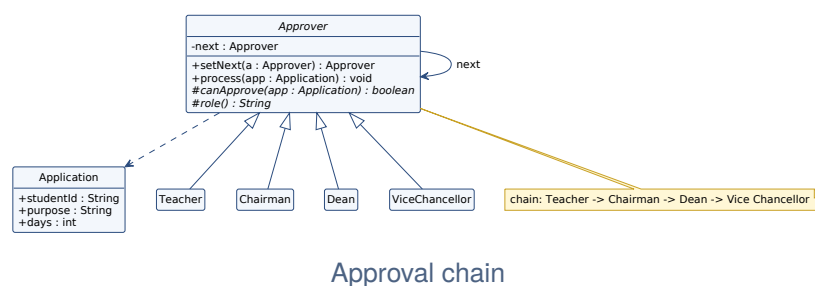
The handwritten solve says Chain of Responsibility is “not in syllabus”, but it was asked in this 2023 final and again in CT-2.

**Q53. Chain of Responsibility: university approval hierarchy****Asked in:** CT-2 Q2**Marks:** 5

A student's leave application at a university may be approved by a Teacher, the Chairman, the Dean or the Vice Chancellor depending on its duration. If a level lacks the authority, the application moves to the next higher level (Teacher → Chairman → Dean → Vice Chancellor). Apply a suitable design pattern. (*paraphrased from CT-2; limits assumed: Teacher up to 3 days, Chairman 7, Dean 15, VC 60*)

**Pattern and roles**

**Chain of Responsibility.** Request = Application; Handler = abstract Approver with a next link; Concrete handlers = Teacher, Chairman, Dean, ViceChancellor; Client = the office that submits to the teacher. Unlike the bank case, the *first* level with enough authority approves and the request stops there.



Approval chain

**Java: Q\_CoRUniversity.java**

```

1 class Application {
2     final String studentId;
3     final String purpose;
4     final int days;
5     Application(String studentId, String purpose, int days) {
6         this.studentId = studentId; this.purpose = purpose; this.days = days;
7     }
8 }
9
10 abstract class Approver {
11     private Approver next;
12     public Approver setNext(Approver next) { this.next = next; return next; }
13
14     public void process(Application app) {
15         if (canApprove(app)) {
16             System.out.println(" APPROVED by " + role());
17         } else if (next != null) {
18             System.out.println(" " + role() + " has no authority -> forwards");
19             next.process(app); // pass it up the chain
20         } else {
21             System.out.println(" REJECTED: beyond every authority");
22         }
23     }
24     protected abstract boolean canApprove(Application app);
25     protected abstract String role();
26 }
27
28 class Teacher extends Approver {
29     protected boolean canApprove(Application a) { return a.days <= 3; }
30     protected String role() { return "Teacher"; }

```

```

31 }
32 class Chairman extends Approver {
33     protected boolean canApprove(Application a) { return a.days <= 7; }
34     protected String role() { return "Chairman"; }
35 }
36 class Dean extends Approver {
37     protected boolean canApprove(Application a) { return a.days <= 15; }
38     protected String role() { return "Dean"; }
39 }
40 class ViceChancellor extends Approver {
41     protected boolean canApprove(Application a) { return a.days <= 60; }
42     protected String role() { return "Vice Chancellor"; }
43 }
44
45 public class Q_CoRUniversity {
46     public static void main(String[] args) {
47         Approver teacher = new Teacher();
48         teacher.setNext(new Chairman()).setNext(new Dean()).setNext(new ViceChancellor());
49
50         Application[] apps = {
51             new Application("2010776101", "fever", 2),
52             new Application("2010776102", "family emergency", 6),
53             new Application("2010776103", "exchange programme", 45),
54             new Application("2010776104", "no valid reason", 120)
55         };
56         for (Application a : apps) {
57             System.out.println(a.studentId + " asks " + a.days + " days (" + a.purpose + "
58                 );");
59             teacher.process(a);
60         }
61     }

```

#### Program output (compiled and run)

```

1 2010776101 asks 2 days (fever):
2   APPROVED by Teacher
3 2010776102 asks 6 days (family emergency):
4   Teacher has no authority -> forwards
5   APPROVED by Chairman
6 2010776103 asks 45 days (exchange programme):
7   Teacher has no authority -> forwards
8   Chairman has no authority -> forwards
9   Dean has no authority -> forwards
10  APPROVED by Vice Chancellor
11 2010776104 asks 120 days (no valid reason):
12  Teacher has no authority -> forwards
13  Chairman has no authority -> forwards
14  Dean has no authority -> forwards
15  REJECTED: beyond every authority

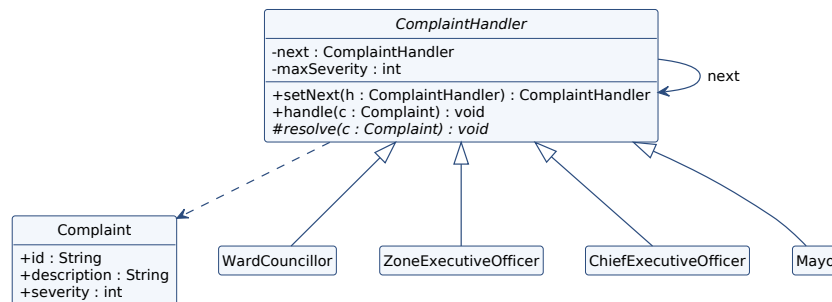
```

## Benefits

The student does not need to know who will approve; each approver knows only its own limit and its successor; the hierarchy can be changed (e.g. add a Pro-VC between Dean and VC) by re-linking the chain, without editing the other classes; a request that nobody can approve is handled at the end of the chain.

**Q54. Chain of Responsibility: DUMS complaint escalation****Asked in:** Set-1 Q6(b) · New**Marks:** 4.25

DUMS escalates complaints by severity: 1–3 are resolved by the Ward Councillor, 4–6 by the Zone Executive Officer, 7–8 by the Chief Executive Officer and 9–10 by the Mayor. The sender should not need to know who finally resolves the complaint. Implement this using an appropriate design pattern.



Complaint escalation chain

**Java: Q\_CoRComplaint.java**

```

1 class Complaint {
2     final String id, description;
3     final int severity;
4     Complaint(String id, String description, int severity) {
5         this.id = id; this.description = description; this.severity = severity;
6     }
7 }
8
9 abstract class ComplaintHandler {
10     private ComplaintHandler next;
11     private final int maxSeverity;           // highest severity this level may
12     resolve
13     protected ComplaintHandler(int maxSeverity) { this.maxSeverity = maxSeverity; }
14
15     public ComplaintHandler setNext(ComplaintHandler next) { this.next = next; return next
16     ; }
17
18     public void handle(Complaint c) {
19         if (c.severity <= maxSeverity) {
20             resolve(c);
21         } else if (next != null) {
22             next.handle(c);           // escalate
23         } else {
24             System.out.println(c.id + ": severity " + c.severity + " is out of range");
25         }
26     }
27     protected abstract void resolve(Complaint c);
28 }
29
30 class WardCouncillor extends ComplaintHandler {
31     WardCouncillor() { super(3); }
32     protected void resolve(Complaint c) { System.out.println(c.id + " (sev " + c.severity
33     + ") -> Ward Councillor: " + c.description); }
34 }
35
36 class ZoneExecutiveOfficer extends ComplaintHandler {

```

```

33     ZoneExecutiveOfficer() { super(6); }
34     protected void resolve(Complaint c) { System.out.println(c.id + " (sev " + c.severity
    + ") -> Zone Executive Officer: " + c.description); }
35 }
36 class ChiefExecutiveOfficer extends ComplaintHandler {
37     ChiefExecutiveOfficer() { super(8); }
38     protected void resolve(Complaint c) { System.out.println(c.id + " (sev " + c.severity
    + ") -> Chief Executive Officer: " + c.description); }
39 }
40 class Mayor extends ComplaintHandler {
41     Mayor() { super(10); }
42     protected void resolve(Complaint c) { System.out.println(c.id + " (sev " + c.severity
    + ") -> Mayor: " + c.description); }
43 }
44
45 public class Q_CoRComplaint {
46     public static void main(String[] args) {
47         ComplaintHandler chain = new WardCouncillor();
48         chain.setNext(new ZoneExecutiveOfficer())
49             .setNext(new ChiefExecutiveOfficer())
50             .setNext(new Mayor());
51
52         chain.handle(new Complaint("CMP-1", "street light broken", 2));
53         chain.handle(new Complaint("CMP-2", "bins not collected for 5 days", 5));
54         chain.handle(new Complaint("CMP-3", "drain collapse on main road", 8));
55         chain.handle(new Complaint("CMP-4", "city-wide water contamination", 10));
56         chain.handle(new Complaint("CMP-5", "bad input", 14));
57     }
58 }

```

#### Program output (compiled and run)

```

1 CMP-1 (sev 2) -> Ward Councillor: street light broken
2 CMP-2 (sev 5) -> Zone Executive Officer: bins not collected for 5 days
3 CMP-3 (sev 8) -> Chief Executive Officer: drain collapse on main road
4 CMP-4 (sev 10) -> Mayor: city-wide water contamination
5 CMP-5: severity 14 is out of range

```

## Explanation

Each handler stores the highest severity it may resolve. `handle()` resolves the complaint if it is within the limit, otherwise escalates to next; the last link reports invalid input (CMP-5). The client only calls `chain.handle(c)`.

## Advantages and disadvantages of Chain of Responsibility

| Advantages                                              | Disadvantages                                                       |
|---------------------------------------------------------|---------------------------------------------------------------------|
| Sender and receiver are decoupled                       | A request may reach the end unhandled if the chain is misconfigured |
| Each handler has a single responsibility                | Long chains can be slow and harder to debug                         |
| Handlers can be added, removed or reordered at run time | Order of links matters and must be documented                       |

**Q55. Command pattern for the AAS farmer remote app****Asked in:** Set-2 Q6(b) · New (Notes)**Marks:** 4.25

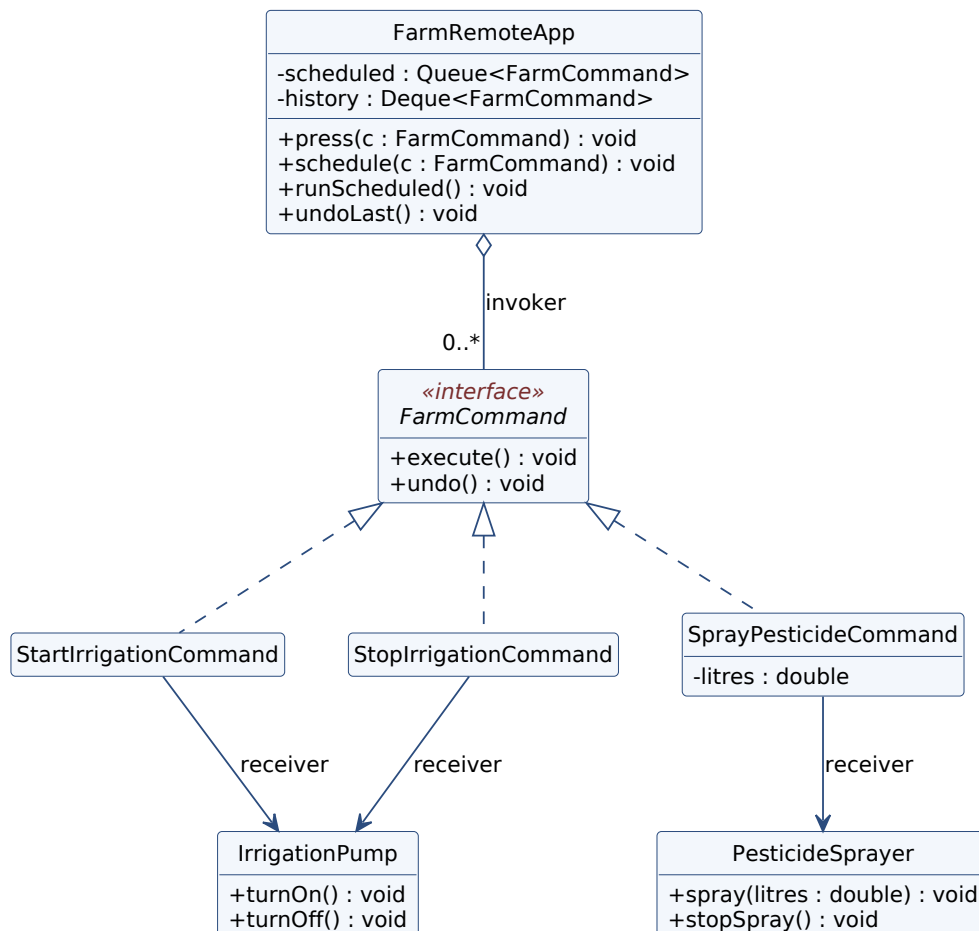
The AAS farmer app has buttons to start irrigation, stop irrigation and spray pesticide. Commands must be queueable for a night schedule, and the last command must be undoable. Apply an appropriate design pattern.

**Pattern: Command**

**Intent:** encapsulate a request as an object, thereby letting you parameterise clients with different requests, **queue or log requests**, and support **undoable operations**. Category: behavioural.

**Roles**

| Role                    | Responsibility                                             | In this design                                                                                                      |
|-------------------------|------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>Command</b>          | declares <code>execute()</code> (and <code>undo()</code> ) | <code>FarmCommand</code>                                                                                            |
| <b>Concrete command</b> | binds a receiver to an action                              | <code>StartIrrigationCommand</code> ,<br><code>StopIrrigationCommand</code> ,<br><code>SprayPesticideCommand</code> |
| <b>Receiver</b>         | knows how to do the real work                              | <code>IrrigationPump</code> , <code>PesticideSprayer</code>                                                         |
| <b>Invoker</b>          | asks commands to execute; keeps queue and history          | <code>FarmRemoteApp</code>                                                                                          |
| <b>Client</b>           | creates commands and gives them to the invoker             | <code>main</code>                                                                                                   |



## Command pattern for farm operations

## Java: Q\_CommandIrrigation.java

```

1  import java.util.ArrayDeque;
2  import java.util.Deque;
3  import java.util.LinkedList;
4  import java.util.Queue;
5
6  class IrrigationPump {                                // Receiver
7      void turnOn() { System.out.println(" Pump ON"); }
8      void turnOff() { System.out.println(" Pump OFF"); }
9  }
10
11 class PesticideSprayer {                               // Receiver
12     void spray(double litres) { System.out.println(" Spraying " + litres + " L"); }
13     void stopSpray() { System.out.println(" Spraying stopped (rolled back)"); }
14 }
15
16 interface FarmCommand {                               // Command
17     void execute();
18     void undo();
19 }
20
21 class StartIrrigationCommand implements FarmCommand {
22     private final IrrigationPump pump;
23     StartIrrigationCommand(IrrigationPump pump) { this.pump = pump; }
24     public void execute() { pump.turnOn(); }
25     public void undo() { pump.turnOff(); }
26 }
27
28 class StopIrrigationCommand implements FarmCommand {
29     private final IrrigationPump pump;
30     StopIrrigationCommand(IrrigationPump pump) { this.pump = pump; }
31     public void execute() { pump.turnOff(); }
32     public void undo() { pump.turnOn(); }
33 }
34
35 class SprayPesticideCommand implements FarmCommand {
36     private final PesticideSprayer sprayer;
37     private final double litres;
38     SprayPesticideCommand(PesticideSprayer sprayer, double litres) { this.sprayer =
39         sprayer; this.litres = litres; }
40     public void execute() { sprayer.spray(litres); }
41     public void undo() { sprayer.stopSpray(); }
42 }
43
44 class FarmRemoteApp {                                  // Invoker: knows only FarmCommand
45     private final Queue<FarmCommand> scheduled = new LinkedList<>();
46     private final Deque<FarmCommand> history = new ArrayDeque<>();
47
48     void press(FarmCommand c) { c.execute(); history.push(c); }
49     void schedule(FarmCommand c) { scheduled.add(c); }
50     void runScheduled() { while (!scheduled.isEmpty()) press(scheduled.poll()); }
51     void undoLast() { if (!history.isEmpty()) history.pop().undo(); }
52 }
53
54 public class Q_CommandIrrigation {                    // Client: creates and wires
55     commands

```

```
54 public static void main(String[] args) {
55     IrrigationPump pump = new IrrigationPump();
56     PesticideSprayer sprayer = new PesticideSprayer();
57     FarmRemoteApp app = new FarmRemoteApp();
58
59     System.out.println("Button: Start irrigation");
60     app.press(new StartIrrigationCommand(pump));
61
62     System.out.println("Night schedule running:");
63     app.schedule(new StopIrrigationCommand(pump));
64     app.schedule(new SprayPesticideCommand(sprayer, 12.5));
65     app.runScheduled();
66
67     System.out.println("Undo last command:");
68     app.undoLast();
69 }
70 }
```

#### Program output (compiled and run)

```
1 Button: Start irrigation
2   Pump ON
3 Night schedule running:
4   Pump OFF
5   Spraying 12.5 L
6 Undo last command:
7   Spraying stopped (rolled back)
```

### How the requirements are met

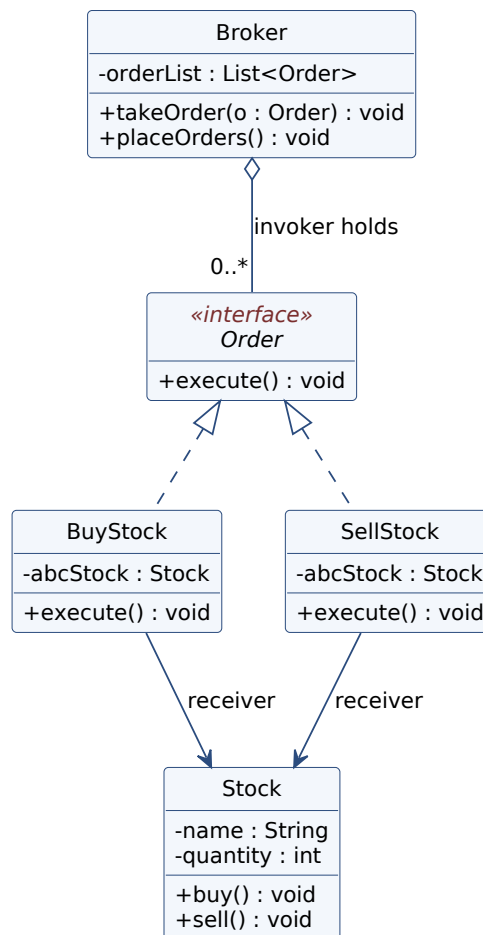
**Buttons:** each button is simply bound to a command object; the app never calls the pump directly.

**Queueing:** commands are objects, so they are stored in a Queue and run later by `runScheduled()`.

**Undo:** executed commands are pushed on a history stack; `undoLast()` pops the last one and calls its `undo()`. New operations (e.g. `FertilizeCommand`) are new classes; the invoker is unchanged.

#### The notes' version

Stock trading: Order (command), BuyStock/SellStock (concrete commands), Stock (receiver), Broker with `takeOrder()` and `placeOrders()` (invoker).



Command example from the notes

**Q56. Mediator pattern for a smart traffic intersection****Asked in:** New (Notes)**Marks:** 4

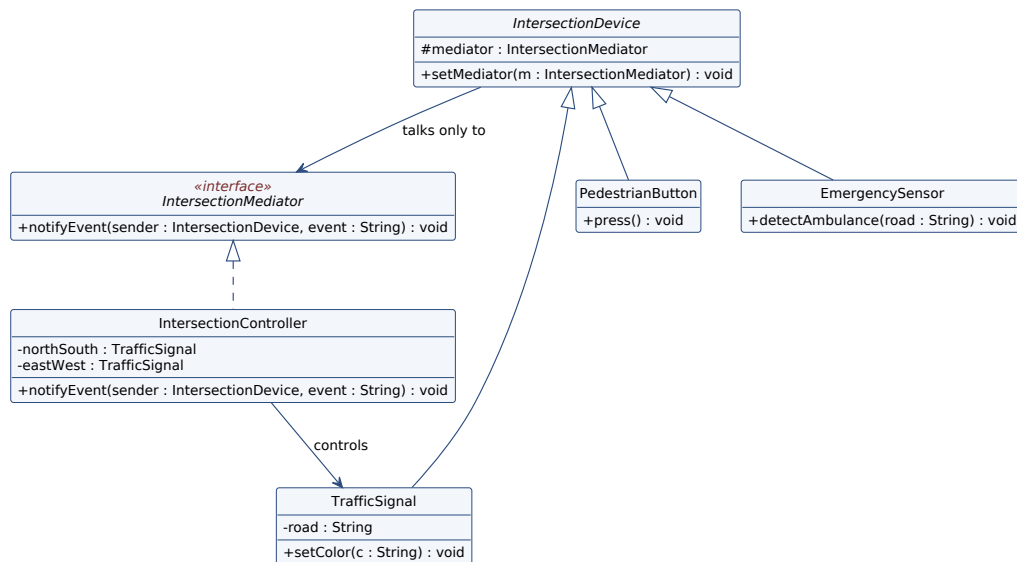
At a DUMS intersection, two traffic signals, a pedestrian button and an ambulance sensor must coordinate: when the button is pressed both signals turn red; when an ambulance approaches on one road, that road gets green and the other red. If every device talks directly to every other device the design becomes tangled. Apply the Mediator pattern.

**Problem**

With  $n$  devices talking to each other directly there are up to  $n(n - 1)$  dependencies; each device must know the others, and adding a camera means changing all of them.

**Mediator**

**Intent:** define an object that **encapsulates how a set of objects interact**. Colleagues communicate only through the mediator, which **reduces many-to-many relationships to one-to-many** and keeps coordination rules in one place.



Mediator for an intersection

## Java: Q\_MediatorTraffic.java

```

1 interface IntersectionMediator {
2     void notifyEvent(IntersectionDevice sender, String event);
3 }
4
5 abstract class IntersectionDevice {                                // Colleague
6     protected IntersectionMediator mediator;
7     void setMediator(IntersectionMediator m) { mediator = m; }
8 }
9
10 class TrafficSignal extends IntersectionDevice {
11     private final String road;
12     TrafficSignal(String road) { this.road = road; }
13     void setColor(String color) { System.out.println(" " + road + " signal -> " + color); }
14 }
15
16 class PedestrianButton extends IntersectionDevice {
17     void press() {
18         System.out.println("Pedestrian button pressed");
19         mediator.notifyEvent(this, "PEDESTRIAN_WAITING"); // knows nothing about signals
20     }
21 }
22
23 class EmergencySensor extends IntersectionDevice {
24     void detectAmbulance(String road) {
25         System.out.println("Ambulance detected on " + road + " road");
26         mediator.notifyEvent(this, "AMBULANCE_" + road);
27     }
28 }
29
30 class IntersectionController implements IntersectionMediator {    // Concrete mediator
31     private final TrafficSignal northSouth, eastWest;
32
33     IntersectionController(TrafficSignal ns, TrafficSignal ew, PedestrianButton btn,
34         EmergencySensor es) {
35         northSouth = ns;

```

```

35     eastWest = ew;
36     ns.setMediator(this); ew.setMediator(this); btn.setMediator(this); es.setMediator(
    this);
37 }
38
39 @Override public void notifyEvent(IntersectionDevice sender, String event) { // all
    rules in ONE place
40     if (event.equals("PEDESTRIAN_WAITING")) {
41         northSouth.setColor("RED");
42         eastWest.setColor("RED");
43     } else if (event.equals("AMBULANCE_NS")) {
44         eastWest.setColor("RED");
45         northSouth.setColor("GREEN");
46     } else if (event.equals("AMBULANCE_EW")) {
47         northSouth.setColor("RED");
48         eastWest.setColor("GREEN");
49     }
50 }
51 }
52
53 public class Q_MediatorTraffic {
54     public static void main(String[] args) {
55         TrafficSignal ns = new TrafficSignal("North-South");
56         TrafficSignal ew = new TrafficSignal("East-West");
57         PedestrianButton button = new PedestrianButton();
58         EmergencySensor sensor = new EmergencySensor();
59         new IntersectionController(ns, ew, button, sensor);
60
61         button.press();
62         sensor.detectAmbulance("EW");
63     }
64 }

```

#### Program output (compiled and run)

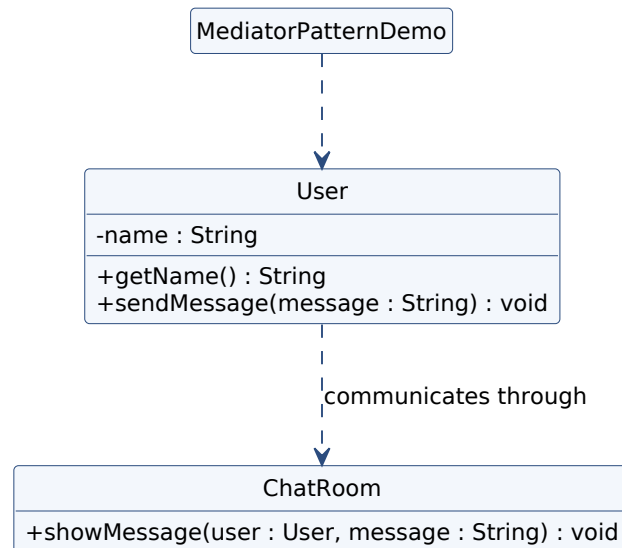
```

1 Pedestrian button pressed
2   North-South signal -> RED
3   East-West signal -> RED
4 Ambulance detected on EW road
5   North-South signal -> RED
6   East-West signal -> GREEN

```

## Mediator vs Observer

Observer broadcasts a change to any number of *unknown* subscribers; Mediator *coordinates* a known group of colleagues and contains the logic of who does what in response. The notes' ChatRoom is a simple mediator: users send messages through ChatRoom.showMessage() instead of to each other.



ChatRoom mediator from the notes

**Q57. MVC for a maintainable web application****Asked in:** 2021 Q5(a)**Marks:** 4.25

Explain how the MVC design pattern can be used to develop a maintainable web application.

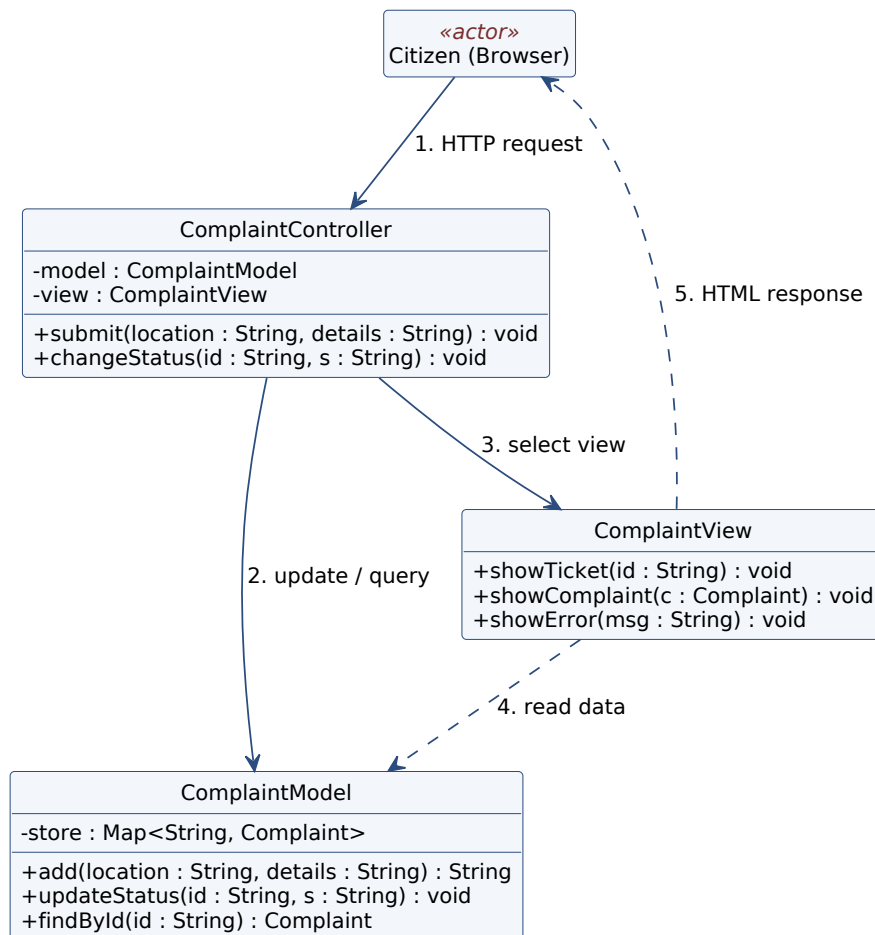
**MVC**

Model–View–Controller separates an interactive application into three kinds of components:

- **Model:** the application data and business rules (e.g. complaints, validation, status changes). It knows nothing about screens.
- **View:** presentation — renders model data as HTML/JSON pages. It contains no business rules.
- **Controller:** receives user requests (HTTP), calls the model to update or query data, and selects the view to render.

**Request flow in a DUMS complaint web app**

(1) The citizen submits a form → (2) ComplaintController validates input and calls ComplaintModel.add() → (3) the controller chooses ComplaintView → (4) the view reads model data → (5) HTML is returned to the browser.



MVC in a DUMS complaint web application

## Java: Q\_MVC.java

```

1  import java.util.LinkedHashMap;
2  import java.util.Map;
3
4  class Complaint {
5      final String id, location, details;
6      String status = "SUBMITTED";
7      Complaint(String id, String location, String details) { this.id = id; this.location =
        location; this.details = details; }
8  }
9
10 class ComplaintModel {                                // MODEL: data + business rules
11     private final Map<String, Complaint> store = new LinkedHashMap<>();
12     private int counter = 100;
13
14     String add(String location, String details) {
15         if (details.isBlank()) throw new IllegalArgumentException("Complaint details are
        required");
16         String id = "CMP-" + (++counter);
17         store.put(id, new Complaint(id, location, details));
18         return id;
19     }
20     void updateStatus(String id, String status) { store.get(id).status = status; }
21     Complaint findById(String id) { return store.get(id); }
22 }

```

```

23
24 class ComplaintView {                                // VIEW: presentation only
25     void showTicket(String id) { System.out.println("<h2>Thank you! Ticket " + id + "</h2>
    "); }
26     void showComplaint(Complaint c) {
27         System.out.println("<p>" + c.id + " | " + c.location + " | " + c.details + " | " +
    c.status + "</p>");
28     }
29     void showError(String msg) { System.out.println("<p class='error'>" + msg + "</p>"); }
30 }
31
32 class ComplaintController {                            // CONTROLLER: handles user
    requests
33     private final ComplaintModel model;
34     private final ComplaintView view;
35     ComplaintController(ComplaintModel model, ComplaintView view) { this.model = model;
    this.view = view; }
36
37     void submit(String location, String details) {      // e.g. POST /complaints
38         try {
39             view.showTicket(model.add(location, details));
40         } catch (IllegalArgumentException e) {
41             view.showError(e.getMessage());
42         }
43     }
44     void changeStatus(String id, String status) {      // e.g. PUT /complaints/{id}
45         model.updateStatus(id, status);
46         view.showComplaint(model.findById(id));
47     }
48 }
49
50 public class Q_MVC {
51     public static void main(String[] args) {
52         ComplaintController controller = new ComplaintController(new ComplaintModel(), new
    ComplaintView());
53         controller.submit("Mohammadpur", "Drain overflowing");
54         controller.submit("Uttara", " ");
55         controller.changeStatus("CMP-101", "IN_PROGRESS");
56     }
57 }

```

#### Program output (compiled and run)

```

1 <h2>Thank you! Ticket CMP-101</h2>
2 <p class='error'>Complaint details are required</p>
3 <p>CMP-101 | Mohammadpur | Drain overflowing | IN_PROGRESS</p>

```

## Why MVC makes the application maintainable

1. **Separation of concerns:** UI designers change views, backend developers change models, without stepping on each other.
2. **Multiple views of one model:** the same `ComplaintModel` serves the web page, a mobile JSON API and the admin dashboard.
3. **Independent change:** replacing HTML with a React front end or MySQL with PostgreSQL affects only one layer.
4. **Testability:** models and controllers can be unit-tested without a browser.

5. **Parallel development and reuse**, supported by frameworks such as Spring MVC, Django and Laravel.
6. It combines patterns: the view observes the model (Observer), and controllers are strategies for handling input.

**Q58. Pattern identification drill****Asked in:** New · practice for scenario questions**Marks:** –

For each scenario, name the most suitable design pattern and the clue that reveals it.

| Scenario                                                                     | Pattern                        | Clue                                |
|------------------------------------------------------------------------------|--------------------------------|-------------------------------------|
| Only one database connection pool / logger / configuration for the whole app | <b>Singleton</b>               | “single shared instance”            |
| Create SMS, email or push object based on a string                           | <b>Factory</b>                 | “which subclass to create”          |
| Create matching sensor + valve from the same vendor                          | <b>Abstract Factory</b>        | “family of related objects”         |
| Object with many optional fields; unreadable constructors                    | <b>Builder</b>                 | “step by step construction”         |
| Third-party library with different method names; cannot modify               | <b>Adapter</b>                 | “incompatible interface”            |
| Many alert types × many channels                                             | <b>Bridge</b>                  | “two dimensions vary independently” |
| Select records by combinable conditions                                      | <b>Filter</b>                  | “AND / OR criteria”                 |
| Tree of city → zone → ward → bin, treat all alike                            | <b>Composite</b>               | “part-whole hierarchy”              |
| Add email/push/timestamp to a notifier at run time                           | <b>Decorator</b>               | “add responsibility dynamically”    |
| One simple method over many subsystems                                       | <b>Facade</b>                  | “simplified interface”              |
| Followers notified on new post; modules react to new reading                 | <b>Observer</b>                | “one-to-many notification”          |
| Request climbs Teacher → Chairman → Dean → VC                                | <b>Chain of Responsibility</b> | “pass along until handled”          |
| Buttons, queued jobs, undo                                                   | <b>Command</b>                 | “request as an object”              |
| Signals, buttons and sensors coordinate without knowing each other           | <b>Mediator</b>                | “central coordinator”               |
| Web app with data, pages and request handling separated                      | <b>MVC</b>                     | “model / view / controller”         |